

처음 배우는 컴퓨터공학, 면접에서 설명할 수 있을 때까지

한양대학교 컴퓨터소프트웨어학과 대학원 지원자를 위한 비전공자용 면접 교재. C의 자료형·연산·메모리부터
원본 노트의 자료구조·알고리즘·시스템까지, 예제를 풀고 이유를 말하는 49단계 과정.

학업계획서 제외

2026.09.16 C 개정 · 학습 재시작

11월 초 면접 · 사용자 제공 일정

오프라인 HTML

00. C에서 출발하는 김공 면접 학습 지도

코드를 처음 보는 사람도 값의 변화를 추적하고, 그 원리를 자료구조와 운영체제 설명으로 연결한다.

이 책은 문법 사전이 아니다. 먼저 프로그램이 실행되는 모습을 보고, 숫자 하나를 저장하고 계산한다. 조건과 반복으로 실행 순서를 바꾼 다음, 함수를 통해 작업을 나눈다. 그 뒤 변수의 범위와 수명, 비트 표현, 배열, 포인터, 구조체와 동적 할당을 배운다. 이 순서가 있어야 연결 리스트의 ‘주소를 연결한다’는 설명이 실제 코드로 이해된다.

값·자료형

연산·조건·반복

함수·범위·수명

배열·포인터·구조체

자료구조·알고리즘

OS·네트워크·DB·면접

시작일은 2026년 9월 16일이다. 49개 단계의 기본 마지막 날은 11월 3일이다. 밀리면 날짜가 지났다고 다음 단계가 자동으로 열리지 않는다. 위젯은 가장 앞의 미완료 단계를 우선한다. 하루의 다섯 항목을 모두 완료 체크하면 다음 단계가 열린다. 밀린 단계는 같은 날 이어서 보충할 수 있지만 미래 날짜의 단계는 미리 열리지 않는다. 완료 체크는 자기 점검이며 자동 실력 판정이 아니다.

기간	선행 지식 위에 쌓는 내용
1주 · 9/16-9/22	C 실행, 자료형, 연산자, 조건, 반복, 함수, 추적 복습
2주 · 9/23-9/29	지역·전역 변수, 비트, 배열, 포인터, 구조체, 동적 메모리, 통합
3주 · 9/30-10/6	복잡도, 선형 자료구조, 재귀, 이진 탐색, 트리, 기초 정렬
4주 · 10/7-10/13	고급 정렬, 해시, 그래프 탐색, 최단경로·MST, 선택 이유
5주 · 10/14-10/20	프로세스·스레드, 가상 메모리, 캐시, 스케줄링·동기화, 파일·I/O
6주 · 10/21-10/27	네트워크, DB, C/C++ 구분, 이산수학, 원본 확장 범위
7주 · 10/28-11/3	연구 연결, 분야별 구술, 모의 면접, 오답 보충

한 단계를 끝내는 기준

10분: 책을 덮고 전 단계의 핵심을 말한다. 30분: 오늘 절의 설명과 코드를 읽는다. 25분: 출력 결과를 먼저 종이에 예측하고 실행 또는 해설로 확인한다. 15분: '정의→예시→조건과 한계' 순서로 말한다. 10분: 막힌 질문을 기록한다. 처음 접하는 날은 90분을 넘길 수 있다. 이해가 안 되는데 체크만 채워 진도를 넘기지 않는다.

학교 공식 교재나 확인된 기출집은 아니다. 학업계획서는 다루지 않는다. 원본의 별표는 원본 강조 표시이며 출제 확률이 아니다. 원본 16쪽과 기존 해설을 부록에 보존했다. C++ 전용 개념은 C와 구분해서 공부한다.

▼ 연습 · 스스로 '이해했다'고 확인하려면 무엇을 할까?

예제를 보지 않고 변수 변화와 출력을 설명하고, 입력을 하나 바꿔 다시 풀고, 왜 그런지 말한다. 실행 결과만 맞힌 것과 원리를 설명하는 것은 다르다.

01. 프로그램과 C의 첫 실행

소스 코드·컴파일·실행을 구분하고, 첫 프로그램의 각 줄을 읽는다. 선수 지식은 없다.

코드는 컴퓨터가 할 일을 적은 글이다

소스 파일은 명령을 적은 텍스트다. C 컴파일러는 전처리와 번역을 거쳐 목적 코드를 만들고, 링커는 필요한 코드와 연결해 실행 파일을 만든다. 실행 파일을 실제로 시작하면 운영체제가 프로세스로 실행한다. ‘컴파일에 성공했다’는 문법과 일부 규칙 검사를 통과했다는 뜻이지, 의도대로 동작한다는 보장은 아니다.

```
#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

`#include <stdio.h>`는 표준 입출력 함수의 선언을 이용하게 한다. 여기서 `#`는 주석이 아니다. `main`은 일반적인 실행 환경의 시작 함수다. 괄호의 `void`는 이 형태에서 매개변수가 없다는 뜻이다. 중괄호는 여러 문장을 묶고, 세미콜론은 문장을 끝낸다. `printf`는 문자열을 출력한다. `\n`은 줄바꿈이다. `return 0;`은 운영 환경에 성공 종료를 알린다.

읽기 → 예측 → 실행 → 이유 설명

위 코드를 UTF-8 텍스트 `hello.c`로 저장한다. GCC가 설치된 터미널에서는 `gcc -std=c17 -Wall -Wextra -Wpedantic hello.c -o hello.exe`, Windows PowerShell에서는 이어서 `.\hello.exe`로 실행할 수 있다. 컴파일러는 위젯용 Python과 별개다. 이 교재는 C17 기준으로 설명하되 기초 예제는 C11과 공통인 문법을 사용한다. 특정 IDE나 유료 도구는 필요하지 않다.

터미널에 명령을 입력하는 것과 C 파일에 코드를 작성하는 것을 구분한다. `//` 설명은 줄 끝까지의 주석, `/*` 설명 `*/`은 구간 주석이다. 들여쓰기는 사람이 흐름을 읽기 위한 것으로, C의 블록은 들여쓰기 자체가 아니라 중괄호로 정해진다.

말로 설명하기 · 예시 답변

C 소스는 컴파일과 링크를 거쳐 실행 파일이 되고, 실행 중인 프로그램은 프로세스입니다. main에서 문장이 순서대로 실행되며 printf는 출력, return은 함수 종료와 결과 전달을 담당합니다.

▼ 연습 · Hello를 두 줄 출력하려면 무엇을 바꾸고, return 뒤의 출력은 실행될까?

printf 문장을 return 앞에 두 번 쓰거나 문자열에 줄바꿈을 넣는다. 실행 흐름이 return에 도달하면 그 함수가 끝나므로 그 뒤에 순서대로 놓인 출력문은 실행되지 않는다.

▼ 꼬리질문 · 컴파일 오류와 논리 오류는 어떻게 다른가?

세미콜론 누락처럼 번역을 막는 문제와, 합계를 구해야 하는데 곱셈을 한 것처럼 실행은 되지만 의도가 틀린 문제는 다르다. 경고도 살피고 작은 입력의 기대 결과를 미리 정해야 한다.

02. 자료형과 변수 — 어떤 값을 담는가

값·변수·자료형을 구분하고 초기화, 정수 나눗셈, 출력 형식을 설명한다. 먼저 첫 실행을 마친다.

변수는 저장된 값을 이름으로 다루는 수단이다. `int age = 25;`에서 `int`는 자료형, `age`는 이름, `25`는 초기값이다. 자료형은 값의 표현과 가능한 연산을 정한다. 선언은 이름과 형식을 알리고, 초기화는 객체가 만들어질 때 첫 값을 준다. 나중의 `age = 26;`은 대입이다.

형식	용도와 예	주의
int	정수 개수, <code>int count = 3;</code>	크기를 무조건 4바이트로 외우지 않는다.
unsigned int	음수가 아닌 정수, <code>5u</code>	범위를 벗어난 연산은 모듈러 규칙을 따른다.
double	실수 근삿값, <code>3.5</code>	모든 소수를 정확히 표현하지 못한다.
char	문자 코드 한 단위, <code>'A'</code>	한글 한 글자가 <code>char</code> 하나라는 뜻은 아니다.
bool	참·거짓, <code>true/false</code>	C17에서는 <code>stdbool.h</code> 를 포함한다.

```
#include <stdio.h>

int main(void) {
    int count = 3;
    double price = 2.5;
    char grade = 'A';
    printf("%d %.1f %c\n", count, price, grade);
    printf("%d %.1f\n", 5 / 2, 5 / 2.0);
    printf("%zu\n", sizeof count);
    return 0;
}
```

첫 줄은 3 2.5 A, 다음은 2 2.5다. 정수끼리 나누면 정수 몫이 된다. 마지막 출력은 현재 구현의 `int` 크기라 환경에 따라 다를 수 있다. `sizeof`의 결과는 `size_t`라서 `%zu`로 출력한다. `printf`의 `%d`는 `int`, `%u`는 `unsigned int`, `%f`는 `double`, `%c`는 문자를 출력하는 약속이다. 인자 형식이 약속과 맞아야 한다.

비트는 0 또는 1이고, C에서 `sizeof(char)`는 항상 1이다. 한 바이트의 비트 수는 `CHAR_BIT`로 확인하며 흔한 환경에서는 8이다. 자료형의 한계는 `limits.h` 등을 확인한다. 초기화하지 않은 자동 지역 변수의 값을 읽으면 안전하게 0이 나오는 것이 아니다. 처음부터 값을 정하자. `const int limit = 10;`은 해당 이름을 통한 수정을 금지한다.

말로 설명하기 · 예시 답변

자료형은 값의 표현 범위와 연산 규칙을 정합니다. `int`와 `double`은 나눗셈 결과부터 다를 수 있고, 정수 크기는 구현에 의존하므로 `sizeof`와 표준 한계값으로 확인합니다. 지역 변수는 사용 전에 초기화하겠습니다.

▼ 연습 · `double x = 5 / 2;`에서 `x`가 2.5가 아닌 이유는?

오른쪽에서 정수 나눗셈이 먼저 이루어져 2가 나온 뒤 `double`의 2.0으로 변환된다. `5.0 / 2` 또는 `(double)5 / 2`로 나눗셈 전에 실수 연산이 되게 한다.

▼ 꼬리질문 · `int`가 항상 32비트인가? 실수 비교도 정수처럼 생각해도 되나?

아니다. `int`의 크기와 범위는 구현을 확인한다. 부동소수점은 근삿값이므로 계산 결과의 비교에서는 문제의 단위와 오차 허용 범위를 따져야 한다.

03. 대입·산술·증감 연산자

각 문장 후 변수 값을 표로 적고, 전위·후위 증감의 차이를 설명한다.

=는 등식이 아니라 오른쪽 결과를 왼쪽 객체에 저장하는 연산이다. $x = x + 2$;는 이전 값을 읽어 2를 더하고 다시 저장한다. 산술 연산자는 $+$, $-$, $*$, $/$, $\%$ 다. 정수 나눗셈은 0 쪽으로 잘리며 $\%$ 는 정수 나머지다. 0으로 나누거나 나머지를 구하면 안 된다.

```
#include <stdio.h>

int main(void) {
    int x = 3;

    x += 2;           // x = 5
    int a = x++;       // a = 5, x = 6
    int b = ++x;       // x = 7, b = 7
    x--;              // x = 6
    printf("%d %d %d\n", x, a, b);
    return 0;
}
```

출력은 6 5 7이다. 후위 $x++$ 도 x 를 증가시킨다. 차이는 그 식이 내놓는 값이 증가 전 값이라는 점이다. 전위 $++x$ 의 식 값은 증가 후 값이다. 증감을 문장 하나로만 쓸 때에는 보통 최종 x 가 같지만, 다른 대입 안에서 사용하면 결과가 달라진다. $x += 2$ 처럼 복합 대입은 왼쪽을 한 번 평가한다는 특징도 있으므로 복잡한 식에서는 단순한 글자 치환으로 생각하지 않는다.

짧게 쓰는 것보다 평가 순서가 명확해야 한다

$i++ + ++i$, $i = i++$ 같은 식은 C에서 한 객체의 수정·접근 사이 순서가 보장되지 않아 정의되지 않은 동작을 일으킨다. 특정 컴파일러가 출력한 숫자를 정답으로 외우지 않는다. `printf`의 여러 인자도 왼쪽부터 평가된다고 가정하면 안 된다. 부수 효과가 있는 식은 별도 문장으로 나눈다.

곱셈이 덧셈보다 먼저 묶이지만, 헛갈리면 괄호로 의도를 드러낸다. $2 + 3 * 4$ 는 14, $(2 + 3) * 4$ 는 20이다. 부호 있는 정수 오버플로는 일반적인 순환을 보장하지 않는다. 예제 입력은 계산 결과가 해당 형식에 들어가는 범위로 제한한다.

말로 설명하기 · 예시 답변

전위와 후위 증감은 모두 객체를 1 증가시키지만, 식의 결과가 각각 증가 후와 증가 전 값입니다. 한 식에서 같은 변수를 여러 번 수정해 순서에 의존하지 않고 문장을 나누겠습니다.

▼ 연습 · `int n=4; int p=n--; int q=--n;` 뒤 `n,p,q`는?

첫 증감 뒤 `p=4, n=3`. 다음 증감 뒤 `n=2, q=2`. 따라서 `n,p,q`는 2,4,2다. 각 선언은 서로 다른 문장이므로 순서가 명확하다.

▼ 꼬리질문 · `printf("%d %d", i++, i++)`의 결과를 왜 외우면 안 될까?

C에서는 이 두 인자의 평가 사이에 요구되는 순서가 없고 같은 `i`를 수정하므로 정의되지 않은 동작이다. 각각 별도 문장으로 계산한 뒤 출력해야 한다.

04. 비교·논리 연산과 조건문

조건을 참·거짓으로 계산하고, 논리 AND와 비트 AND를 구분한다.

비교 연산자 ==, !=, <, <=, >, >=의 결과는 int의 0 또는 1이다. 조건에서는 0이 거짓이고 0이 아닌 값은 참이다. if (x == 3)은 비교지만 if (x = 3)은 대입 후 그 값으로 판단한다. 후자는 의도와 달라도 문법적으로 가능한 탓에 경고 확인이 중요하다.

```
#include <stdio.h>

int main(void) {
    int score = 75;
    int attendance = 1;
    if (score >= 60 && attendance != 0) {
        printf("pass\n");
    } else {
        printf("retry\n");
    }
    int divisor = 0;
    if (divisor != 0 && 10 / divisor > 1) {
        printf("large quotient\n");
    }
    return 0;
}
```

출력은 pass 한 줄이다. &&는 둘 다 참, ||는 하나라도 참, !는 참·거짓을 뒤집는다. &&는 왼쪽이 거짓이면 오른쪽을 평가하지 않고, ||는 왼쪽이 참이면 오른쪽을 평가하지 않는다. 이것이 단락 평가다. 예제에서 divisor가 0이므로 위험한 나눗셈을 실행하지 않는다.

A	B	A && B	A B	!A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

&와 |는 정수의 비트들을 연산하고 단락 평가하지 않는다. 논리 연산의 짧은 표기법이 아니다. $0 < x < 10$ 도 수학처럼 쓰지 않는다. C에서는 앞 비교 결과 0 또는 1을 10과 다시 비교한다. $0 < x \&\& x < 10$ 이라고 작성한다. else if는 앞 조건이 실패한 뒤 다음 조건을 판단한다. switch는 정수 계열 값의 경우를 나눌 때 쓸 수 있고 break가 없으면 다음 case로 흐를 수 있다.

말로 설명하기 · 예시 답변

논리 연산자는 참·거짓을 조합하고 결과는 0 또는 1입니다. AND와 OR의 단락 평가를 이용하면 0으로 나누는 상황처럼 선행 검사가 필요한 연산을 보호할 수 있습니다.

▼ 연습 · 2 && 4와 2 & 4는 각각 얼마인가?

2와 4는 둘 다 0이 아니므로 논리 AND는 1이다. 비트로 0010과 0100은 겹치는 1 자리가 없으므로 비트 AND는 0이다.

▼ 꼬리질문 · 입력값이 15일 때 $0 < x < 10$ 이 참이 되는 이유는?

$0 < 15$ 가 1이 되고, $1 < 10$ 도 참이다. 범위 조건 두 개를 &&로 연결해야 한다.

05. 반복문 — 실행 횟수와 상태 추적

for·while·do while의 순서를 이해하고, 반복 전후의 변화를 직접 적는다.

반복문은 같은 형태의 작업을 여러 번 수행한다. for의 순서는 초기화 1회→조건 검사→본문→증감→조건 검사다. 조건은 본문 전에 평가되므로 처음부터 거짓이면 본문이 한 번도 실행되지 않는다.

```
#include <stdio.h>
int main(void) {
    int total = 0;
    for (int i = 1; i <= 4; ++i) {
        total += i;
    }
    printf("%d\n", total);
    int n = 8;
    while (n > 1) {
        n /= 2;
        printf("%d ", n);
    }
    return 0;
}
```

본문 진입 i	기존 total	새 total
1	0	1
2	1	3
3	3	6
4	6	10

첫 출력은 10이고, while은 8→4→2→1로 바뀌며 4 2 1을 출력한다. 마지막에 n=1이 되면 다음 조건 검사가 거짓이다. 4번 더하는 반복과 3번 반감하는 반복을 비교하면 나중에 $O(n)$ 과 $O(\log n)$ 을 이해하는 기반이 된다.

do while은 본문 뒤에 조건을 검사해서 적어도 한 번 실행한다. break는 가장 안쪽 반복을 끝내고, continue는 이번 본문의 나머지를 건너뛴다. for의 continue는 증감식으로 이동하지만 while은 조건 검사로 이동한다. while에서 증가 코드를 continue 뒤에만 놓으면 반복이 끝나지 않을 수 있다.

배열을 배우면 인덱스는 보통 $0 \leq i \text{ \&\& } i < n$ 으로 제한한다. $i \leq n$ 과 혼동하는 한 칸 오류가 흔하다. 반복마다 무엇이 유지되는지 설명하는 문장을 불변식이라고 한다. 위 합계에서는 ‘다음 i를 더하기 직전 total은 1부터 i-1까지의 합’이 유지된다.

말로 설명하기 · 예시 답변

for는 초기화·조건·증감을 한곳에 모으고 while은 조건 중심으로 반복합니다. 종료 조건뿐 아니라 반복마다 상태가 종료 방향으로 변하는지 확인하고, 출력 전에 변수 변화를 추적합니다.

▼ 연습 · i=0부터 i<3인 동안 반복하면 본문과 조건 검사는 각각 몇 번인가?

본문은 i=0,1,2에서 세 번, 조건은 i=3에서 실패하는 검사까지 네 번이다. continue나 break가 없는 기본 반복을 가정한다.

▼ 꼬리질문 · 2회 반복 안에 3회 반복을 넣으면 작업은 몇 번인가?

안쪽 작업은 $2 \times 3 = 6$ 회다. 2회 반복과 3회 반복을 나란히 실행하면 5회다. 중첩과 순차 실행을 구분한다.

06. 함수와 값 전달

인자·매개변수·반환값을 구분하고, C가 값을 복사해서 전달한다는 뜻을 설명한다.

```
#include <stdio.h>

int twice(int value) {
    return value * 2;
}

void change(int value) {
    value = 99;
    printf("inside: %d\n", value);
}

int main(void) {
    int x = 3;
    int result = twice(x);
    change(x);
    printf("outside: %d, result: %d\n", x, result);
    return 0;
}
```

twice의 앞 int는 반환형이다. 괄호 안 int value는 매개변수 선언이다. 호출할 때 넘기는 x의 값 3은 인자다. value는 그 값을 복사받은 별도 객체다. twice는 6을 반환해 result에 저장하게 한다. change는 자신의 value만 99로 바꾸므로 최종 출력은 outside: 3, result: 6이다.

void 반환형은 돌려주는 값이 없다는 뜻이다. 화면에 출력하는 함수와 결과를 반환하는 함수는 다르다. 출력만 하면 다른 계산에 그 결과를 바로 사용할 수 없다. 함수를 호출하기 전에 선언이 보여야 한다. 큰 프로그램에서는 int twice(int value); 같은 함수 원형을 먼저 놓고 정의를 뒤에 둘 수 있다.

아직 포인터를 몰라도 확인할 수 있는 것

여기서는 ‘함수 안의 매개변수를 바꾸면 호출자의 int도 바뀐다’고 생각하지 않는 것이 중요하다. 호출자의 객체를 바꾸는 방법은 포인터 장에서 배운다. 그때에도 C의 전달 방식 자체가 참조 전달로 바뀌는 것은 아니다. 주소를 담은 값이 복사된다.

말로 설명하기 · 예시 답변

C는 인자 값을 매개변수에 복사합니다. 매개변수를 다른 값으로 바꿔도 호출자의 정수 변수는 그대로입니다. 반환값은 호출식의 결과이고, printf로 화면에 보여주는 것과는 별개입니다.

▼ 연습 · `int a=4; twice(a);`만 실행하면 a는 8이 될까?

a는 4다. 반환된 8을 저장하지 않고 버렸기 때문이다. `a=twice(a);`로 대입해야 a가 8이 된다.

▼ 꼬리질문 · 함수를 나누면 무엇이 좋아지고 무엇을 조심해야 할까?

역할이 분명해지고 작은 입력으로 따로 검증하기 쉽다. 입력 조건과 반환 의미를 정해야 하며, 함수가 보이지 않는 전역 상태를 너무 많이 바꾸면 결과를 추론하기 어려워진다.

07. 지역변수·전역변수 — 범위와 수명

‘어디서 이름을 쓸 수 있는가’와 ‘객체가 언제까지 존재하는가’를 분리한다.

블록 안에서 선언한 지역변수는 해당 블록 안에서 이름을 사용할 수 있다. 파일 범위에 선언한 변수를 흔히 전역변수라고 부른다. 파일 범위 선언이 뒤 파일이나 선언 전의 모든 코드에서 자동으로 보인다는 뜻은 아니다. 범위(scope), 저장 기간(storage duration), 연결(linkage)은 서로 다른 개념이다.

```
#include <stdio.h>

int total;                                // 정적 저장 기간, 0으로 초기화

void visit(void) {
    int local = 0;                        // 호출마다 새로 초기화
    static int calls = 0;                 // 함수가 끝나도 값 유지
    ++local;
    ++calls;
    ++total;
    printf("%d %d %d\n", local, calls, total);
}

int main(void) {
    visit();
    visit();
    return 0;
}
```

출력은 첫 호출 1 1 1, 둘째 호출 1 2 2다. local은 호출마다 새로 0이 된다. calls는 이름이 함수 블록 안에만 보이지만 수명은 프로그램 실행 동안 유지된다. total도 프로그램 실행 동안 존재한다. 따라서 ‘지역변수는 무조건 함수 종료 시 사라진다’는 말은 static 지역변수 때문에 정확하지 않다.

일반 자동 지역변수는 초기화하지 않으면 자동으로 0이 되는 보장이 없다. 정적 저장 기간 객체는 명시적 초기화가 없을 때 0에 해당하는 값으로 초기화된다. 파일 범위 static은 다른 번역 단위에서 그 이름으로 직접 연결되지 않게 하는 내부 연결과 관련된다. extern 선언은 다른 곳에 정의된 객체를 지칭할 때 쓰지만 선언과 실제 정의를 구분해야 한다.

지역변수를 ‘스택’, 전역변수를 ‘데이터 영역’에 그리는 것은 혼한 구현을 이해하기 위한 그림이다. C 언어가 모든 자동 변수를 반드시 물리 스택에 배치하라고 정하는 것은 아니다. 안쪽 블록에서 같은 이름을 선언하면 바깥 이름을 가릴 수 있으므로 초보 단계에서는 다른 이름을 권한다.

말로 설명하기 · 예시 답변

지역·전역은 이름이 보이는 범위이고, 자동·정적 저장 기간은 객체의 수명입니다. static 지역변수는 범위는 지역이지만 호출 사이에 값이 유지됩니다. 전역 상태는 공유가 쉽지만 함수 간 의존성과 동시 접근 문제를 늘릴 수 있습니다.

▼ 연습 · visit을 세 번 호출하면 세 번째 출력은? static을 지우면?

1 3 3이다. calls에서 static을 지우면 매번 0에서 시작해 세 번째 출력은 1 1 3이 된다.

▼ 꼬리질문 · 전역변수를 쓰면 편한데 왜 줄이라고 할까?

누가 언제 값을 바꾸는지 추적하기 어렵고, 함수 단독 테스트나 여러 스레드의 동시 접근이 복잡해진다. 필요한 상태를 인자와 반환값으로 드러내면 의존성을 이해하기 쉽다.

08. 비트 표현과 비트 연산

작은 이진수를 읽고, 마스크로 상태 비트를 켜고·끄고·검사한다.

이진수 각 자리는 오른쪽부터 1,2,4,8의 가중치를 갖는다. 0101은 $4+1=5$, 0011은 $2+1=3$ 이다. 아래 4자리 그림은 낮은 네 비트만 보여준다. unsigned int 전체가 4비트라는 뜻은 아니다. C17의 이식 가능한 예제에서는 0b 접두사를 사용하지 않고 십진 정수나 16진수로 쓴다.

연산	낮은 4비트 예	결과
$5u \& 3u$	0101 AND 0011	0001 = 1
$5u 3u$	0101 OR 0011	0111 = 7
$5u \wedge 3u$	0101 XOR 0011	0110 = 6
$5u \ll 1$	왼쪽으로 한 자리	1010 = 10
$5u \gg 1$	오른쪽으로 한 자리	0010 = 2

```
#include <stdio.h>

int main(void) {
    unsigned int flags = 0u;
    unsigned int read_mask = 1u << 0;
    unsigned int write_mask = 1u << 1;
    flags |= read_mask;           // 0001
    flags |= write_mask;          // 0011
    printf("%u\n", flags);        // 3
    printf("%d\n", (flags & read_mask) != 0u); // 1
    flags &= ~write_mask;         // 쓰기 비트만 끄
    flags ^= read_mask;           // 읽기 비트를 반전
    printf("%u\n", flags);        // 0
    return 0;
}
```

OR는 지정 비트를 켜고, AND와 반전 마스크는 지정 비트를 끈다. XOR는 지정 비트를 뒤집는다. ~는 모든 비트를 반전하므로 ~5u를 '4자리에서 10'이라고 단정하면 안 된다. 결과는 실제 unsigned int 폭 전체에 걸친다. 부호와 정수 승격 문제를 줄이기 위해 이 장은 unsigned int로 연습한다.

시프트 횟수는 음수이거나 승격된 왼쪽 피연산자 형식의 비트 폭 이상이면 안 된다. 부호 있는 값, 특히 음수의 시프트는 별도 규칙이 있어 '무조건 곱셈/나눗셈'으로 외우지 않는다. 작은 unsigned 예에서 관찰한 결과를 모든 int 입력으로 일반화하지 않는다. (flags & mask) != 0u처럼 괄호를 써 비교와 비트 연산의 묶임을 명시한다.

말로 설명하기 · 예시 답변

비트 연산은 정수의 각 자리 상태를 조작합니다. 권한 플래그처럼 여러 참·거짓 상태를 한 정수에 담을 때 마스크를 사용하며, 논리 연산과 달리 결과가 항상 0 또는 1인 것은 아닙니다.

▼ 연습 · flags=5u에서 두 번째 비트(1u<<1)를 켜고, 첫 비트를 끄면?

0101 OR 0010은 0111이므로 7. 여기에 ~0001 마스크를 AND하면 낮은 비트가 0110이 되어 6이다. ~는 실제 정수 폭 전체에서 계산된다.

▼ 꼬리질문 · !5u와 ~5u는 왜 다른가?

!는 논리 부정이라 0이 아닌 5u를 거짓 값 0으로 바꾼다. ~는 unsigned 값의 모든 비트를 뒤집어 형식의 폭에 따른 다른 정수를 만든다.

09. 배열과 문자열 — 같은 자료형을 순서대로

배열 길이와 인덱스를 구분하고, 반복문으로 원소를 안전하게 방문한다.

배열은 같은 자료형의 원소를 연속해서 저장한다. `int a[3]`에는 `int` 세 개가 들어가며 인덱스는 0,1,2다. `a[3]`은 마지막 원소가 아니라 범위 밖이다. C가 자동으로 범위를 검사해 주지는 않으므로 프로그램이 길이를 관리해야 한다.

```
#include <stdio.h>

int main(void) {
    int scores[] = {70, 80, 90};
    size_t count = sizeof scores / sizeof scores[0];
    int sum = 0;
    for (size_t i = 0; i < count; ++i) {
        sum += scores[i];
    }
    printf("sum=%d, average=%.1f\n", sum, (double)sum / count);
    char name[] = "CS";
    printf("%s, bytes=%zu\n", name, sizeof name);
    return 0;
}
```

`sum`은 0→70→150→240이며 평균은 80.0이다. `char name[] = "CS";`는 'C','S','\0' 세 원소를 갖는다. 문자열 끝을 나타내는 널 문자가 있으므로 `sizeof name`은 3이다. 'A'는 문자 상수이고 "A"는 널 문자까지 있는 문자열 리터럴이다. `%s`는 널 문자를 만날 때까지 읽으므로 반드시 유효한 종료와 접근 범위가 필요하다.

`sizeof`로 원소 수를 구하는 식은 여기처럼 실제 배열 객체가 보이는 자리에서만 쓴다. 함수 매개변수의 `int a[]`는 포인터 매개변수로 조정되므로 이 방법으로 원래 배열 길이를 복구할 수 없다. 길이를 별도 인자로 전달하는 이유를 다음 장에서 배운다.

2차원 배열 `int matrix[2][3]`은 'int 세 개짜리 배열 두 개'다. 각 행이 순서대로 연속 배치된다. 배열의 크기가 고정되어 있다는 것과 데이터가 연속 배치된다는 것은 별개 성질이다. 필요 크기가 변할 때에는 동적 할당과 용량 관리가 필요하다.

말로 설명하기 · 예시 답변

배열은 같은 형식의 원소를 연속 저장해 인덱스로 접근합니다. 길이 n 의 유효 인덱스는 0부터 $n-1$ 이며 C는 자동 범위 검사를 보장하지 않습니다. 문자열은 char 배열 안의 널 종료 약속을 함께 지켜야 합니다.

▼ 연습 · `int a[4]={1,2};`와 `char s[]="A"`의 나머지 원소와 길이는?

`a`는 {1,2,0,0}으로 나머지가 0 초기화된다. `s`는 {'A','\0'} 두 원소다. 배열 일부 초기화와 아무 초기화 없는 자동 배열은 다르다.

▼ 꼬리질문 · 문자열 리터럴을 수정해도 되나?

안 된다. 문자열 리터럴을 수정하려 하면 정의되지 않은 동작이다. 수정하려면 `char s[]="CS"`처럼 쓰기 가능한 배열에 복사한 형태를 사용한다. 포인터로 받는 읽기 전용 문자열은 `const char *`로 의도를 드러낸다.

10. 포인터 — 주소와 그곳의 값을 나누기

&x, p, *p를 구분하고 포인터 인자를 이용한 원본 수정과 배열 순회를 설명한다.

주소는 객체의 위치를 가리키는 값이고 포인터는 특정 형식의 객체 등을 가리킬 수 있는 값이다. `int *p`는 `int`를 가리키는 포인터 `p`를 선언한다. 식에서 `&x`는 `x`의 주소, `*p`는 `p`가 가리키는 객체를 뜻한다. 같은 기호라도 선언과 식에서의 역할을 구분한다.

```
#include <stdio.h>

void swap(int *left, int *right) {
    int temp = *left;
    *left = *right;
    *right = temp;
}

int main(void) {
    int x = 3;
    int *p = &x;
    *p = 8;
    int y = 10;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    printf("%p\n", (void *)p);
    return 0;
}
```

먼저 `p`가 `x`를 가리키므로 `*p=8`이 `x`를 8로 바꾼다. `swap`은 `x`와 `y`의 주소 값을 복사받는다.

`temp=8→x=10→y=8` 순서로 변해 최종 출력은 10 8이다. `p` 자체를 다른 주소로 바꾸는 것과 `*p`를 수정하는 것은 다른 작업이다. 주소 출력 `%p`에는 `void *`를 전달하며 실제 주소 숫자는 실행마다 달라질 수 있다.

배열 `a`는 많은 식에서 첫 원소를 가리키는 포인터로 변환된다. `a[i]`는 `*(a+i)`와 대응한다. `int` 포인터 `p+1`은 주소 숫자를 단순히 1바이트 늘린다는 뜻이 아니라 다음 `int` 원소를 가리킨다. 배열 객체와 포인터 변수는 동일한 것이 아니다. `sizeof` 배열과 `sizeof` 포인터가 다른 이유도 여기 있다.

```
int sum_array(const int *a, size_t n) {
    int sum = 0;
    for (size_t i = 0; i < n; ++i) {
        sum += a[i];
    }
    return sum;
}
```

이 함수 조각은 `stddef.h` 등에서 `size_t` 선언을 가져와야 한다. `a`는 읽을 수 있는 `int` `n`개를 가리켜야 하며 합은 `int` 범위 안이라고 가정한다. `const int *`는 이 포인터를 통해 원소를 수정하지 않겠다는 뜻이다. 길이는 포인터에 자동으로 붙어 오지 않는다.

세 가지 금지

초기화되지 않은 포인터를 역참조하지 않는다. `NULL`을 역참조하지 않는다. 이미 수명이 끝난 지역변수나 해제한 메모리의 주소를 사용하지 않는다. 배열 끝 바로 다음 포인터는 비교용으로 만들 수 있지만 역참조하면 안 된다. 포인터 산술은 같은 배열의 유효 범위 규칙 안에서 수행한다.

말로 설명하기 · 예시 답변

C에서는 포인터도 값으로 전달됩니다. 주소의 복사본으로 같은 객체를 가리키기 때문에 `*p`를 통해 호출자의 객체를 바꿀 수 있습니다. 포인터 값 변경과 대상 객체 변경을 구분하고 유효 수명과 범위를 확인합니다.

▼ 연습 · `int a=1,b=2; int *p=&a; p=&b; *p=9;` 뒤 `a,b`는?

`a`는 1, `b`는 9다. `p`를 옮긴 뒤 `*p`를 썼으므로 바뀐 대상은 `b`다. 포인터가 처음 가리키던 `a`가 계속 따라 바뀌는 것은 아니다.

▼ 꼬리질문 · 함수 안의 `int x` 주소를 `return`하면 왜 위험한가?

일반 자동 지역변수 `x`는 블록을 떠나면 수명이 끝난다. 반환된 주소로 나중에 그 값을 읽는 것은 유효한 객체 접근이 아니다. 호출자가 마련한 공간이나 수명이 충분한 저장소를 사용해야 한다.

11. 구조체 — 다른 종류의 데이터를 하나로

배열과 구조체의 역할을 구분하고 . 과 ->로 멤버에 접근한다.

배열은 같은 자료형 여러 개를 담고, 구조체는 서로 다른 역할의 멤버를 하나의 타입으로 묶는다. 학생 한 명의 학번과 성적처럼 함께 다루어야 하는 데이터를 표현한다. C 구조체는 C++ 클래스의 상속이나 virtual을 자동 제공하지 않는다.

```
#include <stdio.h>

struct Student {
    int id;
    double score;
};

void add_bonus(struct Student *student) {
    student->score += 5.0;
}

int main(void) {
    struct Student one = {101, 80.0};
    struct Student copy = one;
    add_bonus(&one);
    printf("%d %.1f %.1f\n", one.id, one.score, copy.score);
    return 0;
}
```

출력은 101 85.0 80.0이다. one.score는 객체 멤버 접근이고 student->score는 (*student).score와 같다. copy는 구조체 값의 복사본이라 one의 score만 바뀐다. 단, 멤버가 포인터이면 포인터 값이 복사되는 것이지 그 대상까지 자동 복제되지는 않는다.

struct Student group[3];는 학생 세 명의 배열이다. typedef struct Student Student;는 새 데이터 배치를 만드는 것이 아니라 기존 타입에 별칭을 붙인다. 구조체의 크기는 멤버 크기 합과 항상 같지 않다. 정렬 요구를 맞추는 패딩이 들어갈 수 있다.

```
struct Node {
    int value;
    struct Node *next;
};
```


이 노드는 값과 다음 노드의 주소를 묶는다. 자기 타입 객체 자체를 멤버로 포함하면 끝없이 크기가 필요하므로 안 되지만, 자기 타입을 가리키는 포인터는 가능하다. 여기까지 배워야 연결 리스트의 data와 next가 선언 가능한 실제 객체로 보인다.

말로 설명하기 · 예시 답변

구조체는 관련된 여러 멤버를 묶어 하나의 타입으로 만듭니다. 객체에는 점, 포인터에는 화살표로 접근합니다. 값 복사와 포인터 대상의 복사는 구분하고, 구조체 크기에는 패딩이 있을 수 있습니다.

▼ 연습 · 학생 배열과 학생 구조체는 어떻게 함께 사용할까?

구조체는 학생 한 명의 id·score를 묶고, 그 구조체의 배열은 여러 학생을 같은 형식으로 저장한다. `group[1].score`처럼 인덱스와 멤버 접근을 조합한다.

▼ 꼬리질문 · 포인터 멤버가 있는 구조체를 대입하면 깊은 복사가 될까?

아니다. 포인터가 담고 있던 주소 값이 복사된다. 두 구조체가 같은 대상에 접근할 수 있으므로 소유권과 해제 책임을 따로 설계해야 한다.

12. 동적 메모리와 C 기초 통합

지역변수와 동적 할당의 수명을 구분하고, 할당→검사→사용→해제를 끝까지 작성한다.

컴파일 시점에 정한 고정 배열 대신 실행 중 필요한 크기의 저장공간을 확보할 때 malloc을 쓴다. stdlib.h의 malloc은 요청한 바이트 수의 저장공간을 확보하고 주소를 반환하거나 실패하면 NULL을 반환한다. 이 공간은 초기화되어 있지 않다. calloc은 할당 공간의 모든 비트를 0으로 만들지만 모든 타입의 의미상 0 표현을 일반적으로 보장한다는 식으로 확장하지 않는다.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    size_t n = 3;                                // 이 예는 크기를 3으로 제한
    int *scores = malloc(n * sizeof *scores);
    if (scores == NULL) {
        return 1;
    }
    int sum = 0;
    for (size_t i = 0; i < n; ++i) {
        scores[i] = (int)(i + 1) * 10;
        sum += scores[i];
    }
    printf("%d\n", sum);                          // 60
    free(scores);
    scores = NULL;
    return 0;
}
```

C에서는 malloc의 void * 반환값을 객체 포인터로 변환하기 위해 별도 캐스트를 쓸 필요가 없다. scores라는 포인터 변수의 수명과 malloc으로 확보한 공간의 수명은 다르다. 자동 포인터 변수가 사라져도 할당 공간이 자동으로 free되지는 않는다. 주소를 잃으면 회수할 방법을 잃는 누수가 된다. 반대로 free 후 남은 주소로 읽으면 use-after-free가 된다.

scores=NULL은 그 포인터만 비울 뿐 다른 별칭 포인터까지 무효 접근을 막아 주지 않는다. 같은 할당 공간은 한 번만 해제하고, malloc 계열이 준 시작 주소가 아닌 중간 주소나 지역변수 주소를 free하지 않는다. 실제 외부 입력 n을 사용할 때는 0 처리, 상한, $n * \text{sizeof } *scores$ 의 크기 계산 오버플로까지 검사해야 한다. 위 예제의 고정 $n=3$ 을 무제한 입력 코드라고 생각하지 않는다.

realloc은 크기를 바꾸며 주소가 바뀔 수 있다. 양수 크기 요청이 실패하면 원래 공간은 유지되므로 임시 포인터에 결과를 받아 성공했을 때만 원래 포인터를 바꾸는 방식이 안전하다. 첫 단계에서는 malloc/free를 확실히 익힌 뒤 확장한다.

통합 과제: 학생 세 명의 평균

Student 구조체 배열 세 칸을 마련하고, 각 학생의 id와 score를 대입하고, 반복문으로 합을 구하고, 평균을 반환하는 함수를 작성한다. 포인터와 길이를 인자로 받게 하고 전역변수는 쓰지 않는다. 동적 배열을 쓴다면 실패 검사와 free까지 작성한다. ‘선언·초기화·반복·함수·주소·멤버’의 역할을 줄마다 말할 수 있어야 다음 자료구조로 간다.

말로 설명하기 · 예시 답변

동적 할당은 실행 중 필요한 공간을 마련하는 방법입니다. C에서는 소유권과 해제 책임을 직접 정해야 하며, 포인터 변수의 수명과 할당 객체의 수명은 다릅니다. 실패·범위·해제 후 접근을 모두 확인합니다.

▼ 연습 · free한 포인터가 NULL이 아니면 아직 사용할 수 있다는 뜻인가?

아니다. free는 포인터 변수의 비트 값을 자동으로 지우지 않는다. 값이 남아 있어도 대상 공간을 사용할 권리는 끝났다. 이후 접근하면 안 된다.

▼ 꼬리질문 · 메모리 힙과 자료구조 힙은 같은 것인가?

이름만 같다. 메모리 힙은 흔히 동적 할당에 쓰는 공간을 가리키고, 자료구조 힙은 부모·자식의 우선순위 조건을 유지하는 트리 기반 구조다.

C 규칙 확인: WG14 N1570의 6.2(범위·수명·형식), 6.5(표현식), 6.7(선언), 6.8(문장), 7.22.3(메모리 관리). N1570은 C11 공개 초안이며, C17 기초 학습과 공통인 규칙 확인에 사용했다. 컴파일 옵션은 GCC 공식 문서를 참고한다.

13. 복잡도와 로그를 외우지 않고 이해하기

$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ 를 연산 구조에서 설명한다. n 은 입력 데이터의 개수라고 정한다.

초 단위 대신 ‘입력이 커지면 얼마나 더 일하는가’

같은 코드도 컴퓨터에 따라 실행 시간이 달라진다. 그래서 크기 n 의 입력을 처리할 때 비교나 이동 같은 기본 연산을 몇 번 하는지 본다. 배열의 모든 값을 한 번 읽으면 n 번 수준이고, 모든 두 값의 조합을 비교하면 n^2 번 수준이다. 실제 성능에는 캐시·상수 비용도 영향을 주지만 증가율은 규모가 커질 때의 차이를 보여준다.

비용 모양	$n=8$	$n=1,024$	의미
1	1	1	크기와 무관한 일정 작업
$\log_2 n$	3	10	절반씩 줄이는 횟수
n	8	1,024	전체 한 번
$n \log_2 n$	24	10,240	전체 작업을 로그 단계
n^2	64	1,048,576	전체×전체

표는 함수값 비교이며 특정 프로그램의 실제 비교 횟수나 실행 시간이 아니다.

로그는 ‘몇 번 곱하면 그 수인가’

$2^3=8$ 이므로 $\log_2 8=3$ 이다. 역으로 8을 2로 세 번 나누면 1이 된다. 1,024개 후보가 $512 \rightarrow 256 \rightarrow \dots \rightarrow 1$ 로 줄면 열 번의 반감이다. 이진 탐색의 비교 횟수는 이와 비슷하게 증가한다. 후보가 1개 남은 뒤의 확인 등으로 정확한 횟수는 달라질 수 있다.

반복문 두 개가 있다고 항상 제곱은 아니다

```
// work()를 상수 비용의 작업이라고 가정한 함수 내부 조각
for (int i = 0; i < n; ++i) { work(); }
for (int j = 0; j < n; ++j) { work(); } // 2n -> O(n)

for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) { work(); }
} // n*n -> O(n^2)
```

여기서 work() 한 번은 일정한 비용이라고 가정한다. 안쪽 반복이 매번 n번이 아니라 10번이라면 10n이므로 O(n)이다. 코드의 줄 수보다 **실제로 각 작업을 몇 번 실행하는지**가 중요하다.

Big O는 충분히 큰 입력에서 비용의 증가를 위에서 제한하는 점근적 상한이다. O 자체가 최악이라는 뜻은 아니다. 최악 비용, 평균 비용 등 분석할 대상을 먼저 정한다. $3n^2+5n+7$ 은 $O(n^2)$ 이다. 이 식은 $O(n^3)$ 이기도 하지만, 비교할 때는 보통 의미 있는 타이트한 차수를 찾는다. $\Theta(n^2)$ 는 위와 아래를 같은 차수로 제한한다는 뜻이다.

공간복잡도는 필요한 메모리의 증가다. 새 배열 n개를 만들면 추가 공간 O(n), 숫자 변수 몇 개만 쓰면 O(1)이다. 재귀 호출도 저장공간을 쓴다. 입력 저장공간을 포함하는지, 추가 공간만 말하는지 구분한다.

말로 설명하기 · 예시 답변

시간복잡도는 입력 크기에 따라 연산량이 어떻게 증가하는지 나타냅니다. 이진 탐색은 매번 후보를 절반으로 줄여 $O(\log n)$ 이고, 전체를 한 번 훑으면 $O(n)$ 입니다. Big O는 점근적 상한이므로 평균인지 최악인지 조건을 함께 밝히겠습니다.

▼ 연습 · 바깥 n번, 안쪽 5번 반복이면? 바깥 n번마다 후보를 절반씩 줄이면?

첫 번째는 5n번 수준이라 $O(n)$. 두 번째는 매 바깥 회차에 n에서 시작해 1까지 절반씩 줄인다고 가정하면 $O(n \log n)$ 이다. 안쪽 반복의 시작값과 종료 조건을 확인해야 한다.

▼ 꼬리질문 · $O(n)$ 이면 $O(n^2)$ 보다 항상 빠른가요?

아니다. 작은 입력에서는 상수 비용이나 캐시 동작 때문에 역전될 수 있다. 복잡도는 충분히 큰 입력에서의 증가율 비교이며, 모든 입력 크기에서 실제 실행 시간을 보장하는 순위가 아니다.

▼ 꼬리질문 · 동적 배열 추가는 $O(1)$ 인데 가끔 전체 복사를 한다면 무슨 아닌가요?

한 번의 최악 비용과 여러 번의 총비용을 나눈 상각 비용을 구분한다. 용량을 두 배씩 늘리는 배열에서 복사량은 $1+2+4+\dots$ 처럼 증가해 n 번 추가 전체로는 $O(n)$ 이다. 따라서 추가 1회의 상각 비용은 $O(1)$, 확장하는 단일 회차의 최악은 $O(n)$ 이다. 상각은 입력을 확률적으로 평균내는 평균 분석과 다르다.

14. 배열·연결 리스트·스택·큐

‘무엇을 자주 할 것인가’에 따라 저장 방식을 고르고, 위치 찾기와 연결 변경 비용을 분리한다.

배열은 나란히, 연결 리스트는 다음 위치를 따라

배열은 같은 크기의 원소를 연속된 위치에 둔다. 시작 주소가 100, 한 원소가 4바이트라면 0번 원소는 100, 1번은 104, 2번은 108에 있다. 주소=시작+인덱스×크기이므로 중간 원소도 앞에서부터 찾아갈 필요가 없다. 이것이 인덱스 접근 $O(1)$ 의 이유다.

배열: [10][20][30] 위치는 연속
연결 리스트: [10|next] → [20|next] → [30|null]
실제 메모리 위치는 서로 떨어져도 됨

노드(node)는 값과 연결 정보를 묶은 단위다. next는 다음 노드의 주소, null은 다음 노드가 없다는 표시다. 두 번째 이후로 가려면 연결을 따라가므로 k번째 접근은 $O(k)$ 다. 이중 연결 리스트는 이전 노드를 가리키는 prev도 둔다.

20 뒤에 25를 넣어 보자

배열 [10,20,30]에서 충분한 용량이 있다고 하자. 30을 한 칸 뒤로 옮기고 빈칸에 25를 넣는다. 뒤에 원소가 많으면 이동도 많다. 리스트에서는 20의 노드를 이미 알 때 ①새 노드 25의 next를 30으로 ②20의 next를 25로 바꾸면 된다. 단, 20을 찾는 과정은 별도다.

연산	배열	단일 연결 리스트
i번째 접근	$O(1)$	$O(n)$ 최악
값으로 찾기, 정렬 없음	$O(n)$	$O(n)$
중간 삽입	$O(n)$ 이동	앞 노드를 알면 연결 $O(1)$, 찾기 포함 최악 $O(n)$
삭제	순서 유지 시 최악 $O(n)$	삭제할 노드와 앞 노드를 알면 $O(1)$, 검색 별도

배열은 이웃 데이터를 함께 읽기 쉬워 캐시에 유리한 경우가 많다. 리스트는 포인터 저장공간과 메모리 할당 비용이 든다. ‘리스트는 삽입이 빠르므로 항상 낫다’는 결론은 성립하지 않는다.

스택과 큐는 ‘꺼내는 규칙’이다

스택은 LIFO(Last In, First Out), 마지막에 넣은 것이 먼저 나온다. push는 넣기, pop은 맨 위 원소를 제거·반환하기다. 10→20→30을 넣고 pop하면 30이다. 되돌리기와 함수 호출 관리에 연결된다.

큐는 FIFO(First In, First Out), 먼저 넣은 것이 먼저 나온다. enqueue는 뒤에 넣기, dequeue는 앞에서 빼기다. 같은 입력에서 dequeue하면 10이다. 대기열과 BFS에 사용한다. 스택·큐는 제공할 연산의 약속인 추상 자료형(ADT)이고 배열이나 리스트로 구현할 수 있다.

배열 큐에서 앞을 뺄 때마다 전부 왼쪽으로 밀면 $O(n)$ 이다. 원형 큐는 front/rear 위치만 다음 칸으로 이동하고 끝에서는 처음으로 돌아가 $O(1)$ 로 처리한다. 짝 참과 비어 있음을 구분하는 크기 변수 등도 필요하다. 리스트 큐는 head와 tail을 함께 유지하면 양 끝 연산을 $O(1)$ 로 만들 수 있다.

말로 설명하기 · 예시 답변

배열은 연속 저장으로 인덱스 접근이 $O(1)$ 이지만 중간 삽입·삭제에는 이동이 필요합니다. 연결 리스트는 링크를 따라 접근하고, 필요한 노드 위치를 이미 알면 연결 변경이 $O(1)$ 입니다. 따라서 탐색 빈도와 수정 위치, 캐시와 메모리 비용까지 보고 선택하겠습니다.

▼ 연습 · 빈 스택과 큐에 A,B,C를 넣고 하나 빼고 D를 넣었다. 다음에 무엇이 나오는가?

스택: C를 뺀 후 A,B에 D가 쌓이므로 다음은 D. 큐: A를 뺀 후 B,C 뒤에 D가 붙으므로 다음은 B. '마지막 입력'만 보지 말고 남은 상태를 적는다.

▼ 꼬리질문 · 연결 리스트에서 특정 값을 찾아 삭제하면 $O(1)$ 인가요?

아니다. 머리에서 값을 찾아가면 최악 $O(n)$ 이다. $O(1)$ 은 삭제에 필요한 노드와 연결 정보를 이미 알고 있을 때의 변경 비용이다. 단일 리스트의 마지막 노드 삭제 등에서는 이전 노드가 필요하다.

원본 연결: 기존 01장. 확장: Open Data Structures, 배열·연결 리스트 장.

C로 연결 리스트의 실제 연결 보기

앞에서 배운 구조체와 포인터를 합친다. 아래는 동적 할당 대신 수명이 main 끝까지 유지되는 노드 세 개로 연결 변경만 보여 준다. 주소가 연속인지와 논리적 연결 순서는 별개다.

```
#include <stdio.h>
struct Node { int value; struct Node *next; };
int main(void) {
    struct Node tail = {30, NULL};
    struct Node head = {10, &tail};
    struct Node middle = {20, head.next};
    head.next = &middle; // 10 -> 20 -> 30
    for (const struct Node *p = &head; p != NULL; p = p->next) {
        printf("%d ", p->value);
    }
    head.next = middle.next; // 20을 논리적으로 연결에서 제외
    return 0;
}
```

middle.next를 먼저 기존 뒤 노드로 설정하고 head.next를 middle로 바꾼다. 반대로 순서를 잘못 쓰면 기존 연결을 잃을 수 있다. 삭제는 연결에서 제외하는 작업과 동적 할당 공간을 해제하는 작업을 구분한다. 이 예는 자동 객체이므로 free하면 안 된다.

C 배열 스택과 원형 큐 · 경계 검사까지

스택의 size는 현재 원소 수이며 다음 삽입 위치다. push 전에는 용량을, pop 전에는 빈 상태를 검사한다. 아래 함수 조각은 성공하면 1, 실패하면 0을 반환한다. out은 유효한 int 객체를 가리켜야 한다.

```
enum { CAPACITY = 4 };
struct Stack { int data[CAPACITY]; int size; };
int push(struct Stack *s, int value) {
    if (s->size == CAPACITY) { return 0; }
    s->data[s->size] = value;
    ++s->size;
    return 1;
}
int pop(struct Stack *s, int *out) {
    if (s->size == 0) { return 0; }
    --s->size;
    *out = s->data[s->size];
    return 1;
}
struct Queue { int data[CAPACITY]; int front; int size; };
int enqueue(struct Queue *q, int value) {
    if (q->size == CAPACITY) { return 0; }
    int rear = (q->front + q->size) % CAPACITY;
    q->data[rear] = value;
    ++q->size;
    return 1;
}
int dequeue(struct Queue *q, int *out) {
    if (q->size == 0) { return 0; }
    *out = q->data[q->front];
    q->front = (q->front + 1) % CAPACITY;
    --q->size;
    return 1;
}
```

사용 전 struct Stack s = {0}; struct Queue q = {0};처럼 초기화한다. 큐의 다음 삽입 위치는 (front+size)%용량이다. size로 빈 상태와 가득 찬 상태를 구분하므로 네 칸 모두 사용할 수 있다. 10,20 삽입

후 제거하면 스택은 20, 큐는 10을 반환한다. 이번 구현의 삽입·제거는 $O(1)$ 이고, 일반 연결 리스트에서 같은 연산 이름을 쓴다고 비용이 자동으로 같아지는 것은 아니다.

▼ 연습 · 네 칸 큐에서 $\text{front}=3, \text{size}=2$ 면 다음 삽입 위치는? 이미 $\text{size}=4$ 라면?

$(3+2)\%4=1$ 이다. $\text{size}=4$ 이면 가득 찼으므로 인덱스를 계산해 덮어쓰기 전에 실패를 반환한다. ‘순환’은 경계를 무시해도 된다는 뜻이 아니다.

15. 재귀·이진 탐색·분할정복

재귀를 '자기 자신을 부른다'에서 끝내지 않고, 작은 문제가 끝난 뒤 어떻게 돌아오는지 설명한다.

작은 답을 받아 큰 답을 만든다

```
// 입력은 0 ≤ n ≤ 100으로 제한한다.  
int sum_to(int n) {  
    if (n == 0) { return 0; }  
    return n + sum_to(n - 1);  
}
```

sum_to(3)은 3에 sum_to(2)의 결과를 더해야 하므로 기다린다. sum_to(2)는 sum_to(1)을 기다리고, sum_to(1)은 sum_to(0)을 기다린다. 마지막 호출이 0을 반환하면 차례로 1, 3, 6이 완성된다.

호출 3→2→1→0

종료 조건에서 0 반환

반환값 0→1→3→6

기다리는 호출의 지역 상태·복귀 위치를 보관하는 구조가 호출 스택이다. 이 예의 시간은 $O(n)$, 호출 스택 추가 공간도 $O(n)$ 이다. 반복문으로 합을 누적하면 추가 공간 $O(1)$ 이 가능하다. 종료 조건뿐 아니라 입력이 그 조건에 가까워지는지도 확인한다.

이진 탐색: 왜 절반을 버려도 되는가

오름차순 배열 [2,5,8,11,14,17,20]에서 17을 찾는다. 가운데 11보다 17이 크다. 배열이 정렬되어 있으므로 11의 왼쪽에는 17이 있을 수 없다. 오른쪽 구간만 남기면 가운데가 17이다. **정렬 조건**이 후보를 안전하게 버리는 근거다.

```
// a는 길이 n의 오름차순 int 배열. 0 <= n <= INT_MAX.
int binary_search(const int a[], int n, int target) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (a[mid] == target) { return mid; }
        if (a[mid] < target) { left = mid + 1; }
        else { right = mid - 1; }
    }
    return -1;
}
```

반복형 이진 탐색은 배열에서 최악 $O(\log n)$, 추가 공간 $O(1)$ 이다. 탐색이 끝났는데 못 찾으면 -1을 돌려주도록 약속했다. 중복값이 있으면 이 코드는 그중 하나를 찾으며 첫 위치를 보장하지 않는다.

직접 넘겨 보기: 위 배열에서 6 찾기

left=0, right=6 → mid=3, 값 11. 6은 작으므로 right=2.

1 / 4

인쇄·스크립트 미사용 시 추적 요약: (0,6)→mid=3/11 → (0,2)→mid=1/5 → (2,2)→mid=2/8 → (2,1), 탐색 실패.

재귀와 분할정복은 같은 말이 아니다

재귀는 함수 호출 방식이고, 분할정복은 문제를 작은 부분으로 나누어 해결하고 결합하는 설계 전략이다. 병합 정렬은 양쪽을 정렬한 뒤 합친다. 단순히 자기 자신을 호출한다는 이유만으로 모든 재귀가 분할정복이라고 하지는 않는다.

말로 설명하기 · 예시 답변

재귀는 작은 입력에 대한 같은 함수의 결과를 이용하는 방식입니다. 종료 조건과 문제 크기 감소가 필요하고 호출 스택 비용도 봐야 합니다. 이진 탐색은 정렬된 배열의 가운데를 비교해 절반을 제외하므로 최악 $O(\log n)$ 입니다.

▼ 연습 · 위 `sum_to(4)`의 반환값과 동시에 쌓이는 호출 수는?

반환값은 10. `sum_to(4)`, (3), (2), (1), (0)까지 최대 5개의 호출이 쌓인다. 일반 n 에서는 $n+1$ 개이므로 추가 공간 $O(n)$ 이다. 합을 구하는 반복문과 비교해 설명한다.

▼ 꼬리질문 · 정렬되지 않은 배열에 이진 탐색을 적용하면 왜 안 되나요?

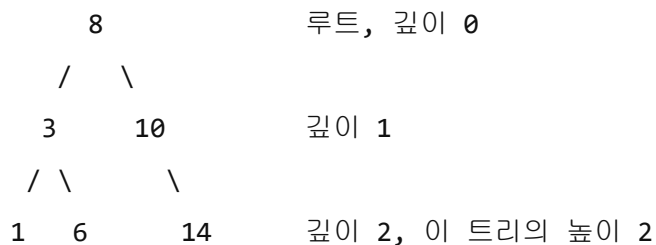
가운데 값보다 작다는 이유로 다른 절반에 답이 없다고 결론낼 수 없기 때문이다. 한 번만 찾는다면 $O(n)$ 순차 탐색이 정렬 후 탐색보다 저렴할 수 있다. 반복 검색이 많을 때는 정렬 비용을 여러 검색에 나누어 생각한다.

16. 트리·BST·힙·균형 트리

트리의 모양, 검색 규칙, 우선순위 규칙을 구분한다. 같은 '이진' 구조라고 같은 자료구조가 아니다.

계층을 그림으로 표현한다

트리는 연결되어 있고 사이클이 없는 무방향 그래프로 정의할 수 있다. 계층을 설명할 때 기준 노드인 루트(root)를 정한다. 바로 위 노드는 부모, 아래는 자식, 자식이 없는 노드는 잎(leaf)이다. 노드가 n 개인 트리의 간선은 $n-1$ 개다. 여기서는 깊이와 높이를 **간선 수**로 센다. 루트 깊이는 0이다.



이진트리는 각 노드의 자식이 최대 두 개이며 왼쪽과 오른쪽이 구분된다. **full**은 각 노드의 자식이 0개 또는 2개, **complete**는 마지막 층 전까지 꽉 차고 마지막 층은 왼쪽부터 채움, **perfect**는 모든 층이 꽉 찬 트리다. 한국어 번역이 혼용되기도 하므로 영어와 정의를 같이 말한다. 위 그림은 일반 이진트리이며 complete도 full도 아니다.

BST: 한 번 비교하면 갈 방향을 알 수 있다

중복 키가 없다고 할 때 이진 검색 트리(BST)의 규칙은 '왼쪽 서브트리 전체의 키 < 현재 키 < 오른쪽 서브트리 전체의 키'다. 위 그림에서 6을 찾으면 8보다 작아 왼쪽, 3보다 커 오른쪽으로 간다. 7을 삽입하면 6의 오른쪽 빈자리에 붙는다.

삭제는 잎이면 제거, 자식이 하나면 자식을 위에 연결, 자식이 둘이면 오른쪽 서브트리 최솟값 등으로 대체한 뒤 그 대체 노드를 제거한다. 예를 들어 3을 삭제할 때 6으로 대체하면 $1 < 6 < 8$ 이라는 순서가 유지된다.

검색·삽입·삭제는 내려가는 길의 길이인 높이 h 에 비례해 $O(h)$ 다. 1,2,3,4를 순서대로 넣으면 한쪽으로 길어져 높이 $n-1$ 이 될 수 있다. 따라서 일반 BST는 최악 $O(n)$ 이다. 균형을 유지하면 높이가 $O(\log n)$ 이 된다.

순회: 언제 현재 노드를 읽느냐

전위는 현재→왼쪽→오른쪽, 중위는 왼쪽→현재→오른쪽, 후위는 왼쪽→오른쪽→현재다. 위 그림의 중위는 1,3,6,8,10,14다. BST의 중위 순회라서 오름차순이지 모든 이진트리의 중위가 정렬 결과인 것은 아니다. 레벨 순회는 큐로 같은 깊이부터 읽는다.

힙: 가장 급한 일 하나를 빨리 고른다

최소 힙은 complete 형태와 ‘부모≤자식’ 조건을 유지한다. 최대 힙은 부등호가 반대다. 형제끼리의 순서는 정하지 않는다. 최소 힙은 최솟값 조회 $O(1)$, 삽입·최솟값 제거 $O(\log n)$ 이다. 이진 힙에서 임의의 특정 값을 찾는 일은 보통 $O(n)$ 이다. 우선순위 큐를 구현할 때 적합하다.

최소 힙 $[2,5,3]$ 에 1을 넣으면 마지막에 붙여 $[2,5,3,1]$. 부모 5와 바꾸어 $[2,1,3,5]$, 다시 부모 2와 바꾸어 $[1,2,3,5]$. 새 원소가 부모와 비교하며 올라가는 횟수는 높이가 이하다. 0부터 시작하는 배열에서 i 의 자식 위치는 $2i+1$, $2i+2$ 다.

AVL·Red-Black·B-tree는 왜 등장하는가

AVL은 각 노드의 왼쪽·오른쪽 높이 차이가 1 이하가 되도록 회전 등으로 조정한다. $3 \rightarrow 2 \rightarrow 1$ 을 넣어 왼쪽으로 기울면 2를 위로 올려 왼쪽 1, 오른쪽 3으로 만들 수 있다. 회전은 키의 중위 순서를 보존하면서 모양을 바꾼다. Red-Black Tree는 색 규칙을 통해 높이가 $O(\log n)$ 이 되도록 한다. 둘 다 균형 BST이지만 유지 규칙이 다르다.

B-tree는 노드 하나에 여러 키와 자식을 둔다. 저장장치에서 한 블록씩 읽을 때 많은 분기를 한 번에 확인해 트리 높이와 블록 접근을 줄이려는 목적이다. B+tree는 보통 실제 데이터 항목/레코드 참조를 앞에 두고 앞을 연결해 범위 검색에 유리하게 만든다. 노드의 최대 자식 수를 ‘차수’로 부르는 방식 등 정의 규약이 자료마다 다르니 공식을 외우기 전에 기준을 확인한다.

말로 설명하기 · 예시 답변

BST는 키의 순서를 이용해 검색하지만 시간은 높이에 비례해 편향되면 $O(n)$ 입니다. 균형 검색 트리는 높이를 $O(\log n)$ 으로 유지합니다. 힙은 전체 검색 순서보다 최솟값 또는 최댓값을 빠르게 꺼내는 데 적합합니다.

▼ 연습 · 위 BST의 전위·후위·레벨 순회를 적어 보세요.

전위: 8,3,1,6,10,14. 후위: 1,6,3,14,10,8. 레벨: 8,3,10,1,6,14. '현재 노드를 언제 읽는가'를 먼저 말하고 왼쪽 부분부터 같은 규칙을 적용한다.

▼ 꼬리질문 · 힙도 높이가 로그인데 임의 값 검색은 왜 로그가 아닌가요?

부모와 자식의 대소관계만 알 뿐, 목표가 왼쪽/오른쪽 어디에 있는지 BST처럼 한쪽을 배제할 수 없기 때문이다. 최소값이 루트라는 보장과 모든 키를 빨리 찾는 보장은 다르다.

▼ 꼬리질문 · B-tree 대신 이진 균형 트리를 DB 인덱스로 쓰면 안 되나요?

가능 여부보다 접근 비용을 비교한다. 블록 하나에 여러 키를 담고 분기 수를 키우면 같은 수의 키를 더 낮은 높이로 관리해 블록 I/O를 줄일 수 있다. 범위 조회 요구까지 있으면 B+tree의 앞 연결도 도움이 된다. 실제 엔진의 인덱스 구현은 확인해서 말한다.

대조: 기존 트리 기초, 균형 트리 상세·색 규칙. 블록 모델 검토: Open Data Structures §14.2.

17. 정렬: 과정에서 복잡도를 도출하기

삽입·병합·퀵 정렬을 손으로 실행하고, 속도뿐 아니라 안정성·추가 메모리·최악 경우를 비교한다.

‘안정적’은 오류가 없다는 뜻이 아니다

정렬은 키라는 기준값으로 순서를 바꾸는 것이다. (민수,80), (지수,80)처럼 키가 같은 두 기록의 원래 순서를 유지하면 안정 정렬이다. 이름순으로 정렬한 결과를 점수순으로 다시 안정 정렬하면 동점자는 이름순이 유지된다.

방법	작은 실행 예	핵심
선택 정렬	[3,1,2]→[1,3,2]→[1,2,3]	남은 전체에서 최솟값을 골라 앞자리 확정
삽입 정렬	[3 1,2]→[1,3 2]→[1,2,3]	정렬된 앞부분에 새 원소를 끼워 넣음
버블 정렬	[3,2,1]→[2,3,1]→[2,1,3]	이웃 교환으로 이번 최댓값을 뒤로

선택 정렬은 첫 회 $n-1$ 번, 다음 $n-2$ 번 ... 비교하므로 합이 $n(n-1)/2$ 다. 입력이 이미 정렬되어도 최솟값을 찾는 표준 구현의 비교는 $\Theta(n^2)$ 다. 삽입 정렬은 이미 정렬되면 새 값을 한 번 확인하고 멈춰 $\Theta(n)$ 이지만 역순이면 많은 이동이 생겨 $\Theta(n^2)$ 다.

병합 정렬: 먼저 나누고, 두 줄의 맨 앞을 비교한다

[4,1,3,2]
↓ 나눔
[4,1] [3,2]
↓ ↓ ↓ ↓
[4] [1] [3] [2]
↓ 합침
[1,4] [2,3]
↓ 두 맨 앞 중 작은 값부터
[1,2,3,4]

[1,4]와 [2,3]을 합칠 때 1 대 2 $\rightarrow$ 1, 4 대 2 $\rightarrow$ 2, 4 대 3 $\rightarrow$ 3, 남은 4를 붙인다. 각 원소를 한 번씩 옮기니 합치는 비용은 $O(n)$ 이다. 반씩 나누는 총수가 $\log_2 n$ 이고 각 층 전체 작업이 n 정도여서 $\Theta(n \log n)$ 이 된다.

$T(n)=2T(n/2)+\theta(n)$ 의 $T(n)$ 은 n 개를 정렬하는 비용이다. $2T(n/2)$ 는 절반 크기 정렬 두 번, $+\theta(n)$ 은 그 결과를 합치는 비용이다. 수식은 위 그림의 작업량을 적은 것이다. 일반적인 배열 구현은 $O(n)$ 추가 버퍼를 쓰며 같은 키에서 왼쪽을 먼저 고르면 안정적이다.

퀵 정렬: 기준값 양쪽으로 먼저 분리한다

[5,2,4,1,3]에서 기준값(pivot) 3을 정하면 작은 그룹 [2,1], 기준 3, 큰 그룹 [5,4]로 나누고 각 그룹을 정렬한다. 이는 원리 설명용 분할 예이며 실제 제자리 partition의 중간 배열은 구현마다 다르다.

균등 분할이면 각 층 n 정도 $\times$ 로그 층으로 $\Theta(n \log n)$ 이다. 매번 최솟값만 기준으로 골라 0개와 $n-1$ 개로 갈리면 $n+(n-1)+\dots$ 비용이 생겨 $\Theta(n^2)$ 이다. 무작위 pivot은 기대 $O(n \log n)$ 을 주지만 모든 실행의 최악 시간을 없애지는 않는다. 일반적인 제자리 퀵 정렬은 안정적이지 않고 재귀 스택을 쓴다.

힙 정렬과 비교 정렬 밖의 방법

힙 정렬은 최대 힙의 루트를 맨 뒤로 보내고, 남은 힙을 복구한다. n 개를 꺼내며 각 복구가 $O(\log n)$ 이므로 전체 $O(n \log n)$ 이다. 아래에서 위로 만드는 초기 힙 생성 자체는 $O(n)$ 이며, 대부분의 노드가 잎 가까이에 있어 움직일 거리가 짧기 때문이다.

계수 정렬은 [2,1,2,0]을 값별 개수 [1,1,2]로 세어 재배치한다. n 개 입력과 K 개 가능한 키 범위를 처리하므로 $O(n+K)$ 다. LSD 기수 정렬은 [21,13,12]를 일의 자리로 안정 정렬해 [21,12,13], 십의 자리로 안정 정렬해 [12,13,21]로 만든다. d 자리, 기수 b 에 대해 보통 $O(d(n+b))$ 다. 제한된 키 표현을 활용하므로 비교 정렬의 하한과 모순되지 않는다.

표준 구현	시간	추가 공간	안정성
삽입	최선 $\Theta(n)$, 평균·최악 $\Theta(n^2)$	$O(1)$	있음
선택, 교환형	$\Theta(n^2)$	$O(1)$	없음
버블, 조기 종료	최선 $\Theta(n)$, 평균·최악 $\Theta(n^2)$	$O(1)$	같은 키 미교환 시 있음
퀵	평균 $O(n \log n)$, 최악 $O(n^2)$	보통 평균 $O(\log n)$, 단순 재귀 최악 $O(n)$	보통 없음
병합, 기본 배열형	$\Theta(n \log n)$	$O(n)$	가능
힙, 반복 배열형	$O(n \log n)$	$O(1)$	없음
계수, 안정 출력형	$O(n+K)$	$O(n+K)$	있음

말로 설명하기 · 예시 답변

병합 정렬은 반씩 나눈 결과를 선형 시간에 합쳐 $\Theta(n \log n)$ 을 보장하고, 배열 구현은 $O(n)$ 추가 공간을 씁니다. 퀵 정렬은 기준값으로 분할하며 평균 $O(n \log n)$ 이지만 편향 분할이 반복되면 $O(n^2)$ 입니다. 안정성과 메모리, 최악 보장이 필요한지를 보고 선택하겠습니다.

▼ 연습 · $[2a, 2b, 1]$ 에 교환형 선택 정렬을 한 번 적용하면? 안정적인가요?

$2a$ 와 최솟값 1 을 교환해 $[1, 2b, 2a]$ 가 된다. 같은 키 2 의 상대 순서가 뒤집혔으므로 불안정하다. 안정성을 최종 값의 오름차순 여부와 혼동하지 않는다.

▼ 꼬리질문 · 병합 정렬은 왜 n^2 가 아니라 $n \log n$ 인가요?

각 층에는 작은 부분문제가 여러 개 있지만, 그 층의 원소 수를 모두 합치면 n 이다. 층의 수는 반감 횟수인 $\log n$ 이다. n 개의 각 원소가 n 번씩 움직이는 구조가 아니다.

▼ 꼬리질문 · 비교 정렬은 왜 일반적으로 $\Omega(n \log n)$ 의 최악 하한이 있나요?

서로 다른 n 개 키의 순서는 $n!$ 가지다. 두 키의 비교 결과로 구분하는 결정 트리는 모든 순서를 식별할 만큼 많은 잎을 가져야 하므로 높이가 최소 $\log_2(n!)$ 이고 이는 $\Omega(n \log n)$ 이다. Ω 는 아래쪽 경계다. 계수 정렬은 키를 직접 배열 위치로 쓰므로 이 비교 전용 모델이 아니다.

추가 정렬 전체: 기존 비교 정렬, 셸·MSD/LSD·계수 정렬과 조건. 셸 정렬의 복잡도는 간격 수열에 따라 달라지므로 하나의 숫자로 암기하지 않는다.

C 삽입 정렬 — 앞부분이 정렬되어 있다는 약속

```
// a는 길이 n의 유효한 배열이며 n >= 0이다.
void insertion_sort(int a[], int n) {
    for (int i = 1; i < n; ++i) {
        int key = a[i];
        int j = i;
        while (j > 0 && a[j - 1] > key) {
            a[j] = a[j - 1];
            --j;
        }
        a[j] = key;
    }
}
```

[3,1,2]에서 $i=1$ 이면 $key=1$ 을 따로 저장하고 3을 오른쪽으로 옮긴 뒤 1을 넣어 [1,3,2]가 된다. $i=2$ 에서 $key=2$ 를 저장하고 3을 옮겨 [1,2,3]이 된다. while은 $j>0$ 을 먼저 검사해 $a[-1]$ 접근을 막는다. 비교가 $>$ 라 같은 키는 앞의 원소를 지나치지 않아 안정적이다. $>=$ 로 바꾸면 같은 키의 순서가 바뀔 수 있다.

18. 해시: 빨리 찾기 위해 치르는 비용

해시 함수·충돌·부하율을 작은 테이블로 설명하고, 평균 $O(1)$ 의 조건을 말한다.

키를 칸 번호로 바꾼다

학번으로 학생 정보를 찾고 싶다. 모든 학생을 비교하는 대신 해시 함수 $h(k)=k \bmod 7$ 을 사용하자. mod는 나눈 나머지다. $10 \bmod 7=3$ 이므로 10은 3번 칸을 찾는다. ‘해시 값이 같으면 원본도 같다’는 뜻은 아니다. 17도 나머지 3이므로 **충돌**한다.

분리 연결법 (chaining)

칸 0: 비어 있음

칸 3: (10, 정보A) → (17, 정보B) → (24, 정보C)

선형 탐사 (linear probing), 위 순서대로 삽입

칸 3: 10 칸 4: 17 칸 5: 24

분리 연결법은 같은 칸에 들어온 항목들을 별도의 리스트 등으로 관리한다. 선형 탐사는 테이블 안에서 다음 칸을 확인하다 빈곳에 둔다. 17을 검색할 때는 계산된 3번에서 시작해 10은 다른 키이므로 4번까지 확인한다. 각 항목의 실제 키 비교가 필요하다.

평균 $O(1)$ 에도 관리 조건이 있다

부하율 $\alpha=n/m$ 은 항목 수 n 을 테이블 칸 수 m 으로 나눈 값이다. 7칸에 5개면 $5/7$ 이다. 균등하게 분산되는 해시와 적절한 부하율을 가정할 때 빠른 평균 검색을 기대한다. 분리 연결법은 평균 $O(1+\alpha)$, 충돌이 한 칸에 몰리면 기본 리스트 버킷에서 최악 $O(n)$ 이다. 선형 탐사 같은 개방 주소법은 빈칸이 줄면 긴 탐사가 생긴다.

테이블을 늘리면 칸 번호 계산도 바뀔 수 있어 항목들을 재배치하는 rehash가 필요하다. 확장 1회는 $O(n)$ 일 수 있고, 적절한 확장 정책 아래 삽입의 상각 비용을 따로 분석한다. 긴 문자열 키라면 해시 계산 자체에 키 길이 L 에 비례하는 비용도 생긴다. $O(1)$ 은 보통 키 처리 비용을 일정하게 보는 모델이다.

해시와 트리 중 무엇을 고를까

‘학번이 정확히 100인 학생’ 같은 일치 조회에는 해시를 고려한다. ‘학번 100부터 200까지’, ‘가장 작은 키부터’ 같은 범위·순서 조회에는 균형 검색 트리가 자연스럽다. 빨리 저장한다는 사실이 정렬 순서를 보장해 주지는

않는다.

말로 설명하기 · 예시 답변

해시 테이블은 키를 칸 번호로 계산해 접근합니다. 서로 다른 키가 같은 칸에 들어가는 충돌은 연결법이나 탐사법으로 처리합니다. 좋은 분산과 적절한 부하율에서 평균 $O(1)$ 을 기대하지만, 충돌이 몰리면 최악 $O(n)$ 이 될 수 있고 범위 검색에는 트리를 고려합니다.

▼ 연습 · 위 선형 탐사 테이블에서 10을 그냥 '빈칸'으로 지우면 24 검색에 무슨 문제가 생기나요?

검색이 원래 시작 칸 3에서 빈칸을 만나 '없다'고 멈추면 실제 5번의 24를 놓친다. 검색은 계속하되 삽입에 활용할 수 있는 삭제 표시(tombstone)를 두거나 적절히 재배치해야 한다.

▼ 꼬리질문 · 해시를 비밀번호 저장에도 그대로 사용하면 되나요?

테이블용 해시는 빠른 분산을 위한 것이고 비밀번호 저장은 공격 비용을 높이는 목적이다. 비밀번호에는 salt와 전용 비밀번호 해싱 방식 등이 필요하다. 빠른 테이블 해시를 보안 기능으로 대체하면 안 된다. 보안 목적의 구분은 27장에서 이어진다.

원본 대응: 검색·충돌 상세, 해시 함수 5종과 보안.

19. 그래프·BFS·DFS·최단경로·MST

탐색 순서와 경로 최적화 목적을 구분하고, 작은 그래프에서 직접 실행한다.

점은 대상, 선은 관계다

그래프의 정점(vertex)은 도시·사람 같은 대상, 간선(edge)은 도로·친구 같은 관계다. 방향이 있으면 $A \rightarrow B$ 와 $B \rightarrow A$ 가 다를 수 있고, 가중치(weight)는 거리·비용 등을 뜻한다. 경로는 간선을 따라 이동하는 순서, 사이클은 돌아오는 순환이다. 복잡도에서는 V 를 정점 수, E 를 간선 수로 쓰겠다.

무방향 그래프의 이웃 목록

A: B, C

B: A, D

C: A, D

D: B, C, E

E: D

인접 리스트는 이웃만 저장해 $O(V+E)$ 공간이다. 인접 행렬은 $V \times V$ 표에서 연결 여부를 저장해 $O(V^2)$ 공간이며 두 점의 간선 여부를 $O(1)$ 에 확인한다. 간선이 드문 그래프에서는 목록 방식이 메모리에 유리하다.

BFS: 가까운 층부터, 큐로 처리한다

꺼내는 정점	새로 발견해 넣는 정점	처리 후 큐
시작 전	A	[A]
A	B,C	[B,C]
B	D	[C,D]
C	없음, D는 이미 발견	[D]
D	E	[E]
E	없음	[]

큐에 넣을 때 방문 표시를 해 중복 삽입을 막는다. 발견 시 이전 정점을 기록하면 $E \leftarrow D \leftarrow B \leftarrow A$ 를 역으로 따라 $A \rightarrow B \rightarrow D \rightarrow E$ 경로를 얻는다. 모든 간선 비용이 같다면 먼저 발견한 경로의 간선 수가 최소다. 방문 순서가 같지 않아도 올바른 BFS일 수 있으며, 여기서는 이웃을 알파벳순으로 본다.

DFS: 한 방향을 끝까지 확인한 뒤 돌아온다

재귀 DFS로 위 그래프를 알파벳순 탐색하면 $A \rightarrow B \rightarrow D \rightarrow C$ 까지 간 뒤 D로 돌아와 E를 방문한다. 방문 순서는 A,B,D,C,E다. 재귀 호출 또는 명시적 스택을 사용한다. 사이클이 있어 방문 표시가 없으면 반복 방문할 수 있다. 인접 리스트에서 DFS/BFS 모두 각 정점·간선을 일정 횟수씩 처리해 $O(V+E)$, 탐색용 방문 배열·스택·큐는 $O(V)$ 다. 그래프 저장공간은 별도다.

가중치가 다르면 ‘적은 간선’이 ‘싼 경로’가 아니다

$A \rightarrow B = 10$, $A \rightarrow C = 1$, $C \rightarrow B = 1$ 이면 B로 직접 가는 한 간선의 비용은 10, C를 거치는 두 간선은 2다. 일반 BFS는 최소 간선 수를 찾으므로 최소 비용을 보장하지 않는다. Dijkstra는 음수 가중치가 없는 그래프에서 현재 알려진 누적 거리가 가장 짧은 정점을 확정하며 거리를 갱신한다.

Dijkstra 추적: 시작 거리 $A=0$, $B=\infty$, $C=\infty \rightarrow A$ 처리 후 $B=10, C=1 \rightarrow C$ 선택 후 $B=\min(10, 1+1)=2 \rightarrow B$ 선택. ∞ 는 아직 경로를 모른다는 표시다. ‘완화(relaxation)’는 새 경로가 더

짧으면 거리 추정값을 바꾸는 것이다. 음수 간선이 있으면 아직 안 본 경로가 이미 확정된 거리를 더 낮출 수 있어 이 확정 논리가 깨진다.

MST: 한 출발점의 거리가 아니라 전체 연결 비용

최소 신장 트리(MST)는 연결된 무방향 가중 그래프의 모든 정점을 사이클 없이 잇는 방법 중 간선 합이 최소인 것이다. 건물마다 전선을 연결하는 총비용을 최소화하는 문제로 생각한다. 연결되지 않았으면 전체의 신장트리가 없고 성분별 최소 신장 숲을 구할 수 있다.

Kruskal은 전체 간선을 싼 순서로 보며 사이클을 만들지 않는 간선을 고른다. Union-Find는 두 정점이 이미 같은 연결 성분인지 확인한다. **Prim**은 현재 트리와 바깥 정점을 잇는 간선 중 가장 싼 것을 골라 한 트리를 확장한다. Prim이 보는 값은 ‘붙이는 간선 하나’, Dijkstra가 보는 값은 ‘시작점부터 누적 거리’다.

A-B=2, A-C=2, B-C=1에서 Kruskal은 B-C(1), A-B(2)를 고르면 합 3인 MST다. 이 트리에서 A→C 경로는 비용 3이지만 원래 그래프에는 비용 2인 A-C가 있다. 전체 연결 비용 최소와 각 목적지 최단거리는 다른 목표다.

말로 설명하기 · 예시 답변

BFS는 큐로 가까운 층부터 탐색해 같은 간선 비용에서 최단경로를 찾습니다. DFS는 스택이나 재귀로 깊게 진행합니다. 가중치가 다르면 조건에 맞는 최단경로 알고리즘을 골라야 하며, MST는 개별 경로가 아니라 전체를 잇는 간선 합을 최소화하는 별도 문제입니다.

▼ 연습 · 위 그래프에서 A에서 E까지 최소 간선 수는? 경로가 하나뿐인가요?

3개다. A-B-D-E와 A-C-D-E가 모두 가능하다. BFS에서 어느 경로를 기록하는지는 이웃 처리 순서에 따라 달라질 수 있다. 최단경로가 유일하다는 보장은 없다.

▼ 꼬리질문 · 왜 탐욕적으로 고르면 언제나 최적이라고 할 수 없나요?

현재 최선이 전체 최선을 보존한다는 근거가 필요하다. 동전 1,3,4로 6을 만들 때 큰 동전부터 고르면 4+1+1의 3개지만 최적은 3+3의 2개다. MST에서는 현재 선택을 존중하는 절단을 가로지르는 최소 가중치 간선이 안전하다는 절단 성질이 근거다.

▼ 꼬리질문 · 그래프 알고리즘의 시간복잡도는 모두 $O(V+E)$ 인가요?

아니다. 인접 리스트 DFS/BFS가 $O(V+E)$ 다. Kruskal은 간선 정렬 때문에 보통 $O(E \log E)$, 배열·행렬 Prim은 $O(V^2)$, 이진 힙·인접 리스트 기반 Dijkstra는 보통 $O((V+E) \log V)$ 다. 어떤 자료구조로 무엇을 수행하는지 먼저 지정한다.

기존 상세·정정: 탐욕 선택, 그래프·MST·Borůvka.

20. 프로그램이 실행되면: 프로세스·스레드

운영체제의 역할을 자원 관리와 보호로 설명하고, 프로세스와 스레드의 공유 범위를 구분한다.

파일과 실행 중인 존재는 다르다

디스크의 프로그램 파일은 실행할 명령과 데이터다. 실행하면 운영체제(OS)는 주소 공간과 실행 상태 등을 준비해 프로세스로 관리한다. 같은 프로그램 파일을 두 번 실행하면 별개의 프로세스가 생길 수 있다. 운영체제는 CPU 시간·메모리·장치를 여러 프로그램에 배분하고 서로의 자원을 함부로 침범하지 않도록 보호한다.

커널(kernel)은 운영체제 핵심 부분이다. 일반 프로그램은 파일 읽기 같은 요청을 시스템 호출(system call)로 전달한다. 사용자 모드와 커널 모드는 권한을 구분한다. 시스템 호출로 권한 모드가 바뀌는 것과 다른 프로세스·스레드로 실행 대상이 바뀌는 문맥교환은 같은 사건이 아니다.

프로세스 안에서 여러 실행 흐름을 둔다

스레드는 프로세스 안의 실행 흐름이다. 일반적으로 같은 프로세스의 스레드는 코드·전역 데이터·힙·열린 파일 등의 자원을 공유하고, 각자의 레지스터 상태·프로그램 카운터·스택을 갖는다. 프로그램 카운터는 다음에 실행할 명령 위치를 나타낸다. 스택은 각 호출이 어디로 돌아갈지 등의 정보를 관리한다.

관점	프로세스	같은 프로세스의 스레드
주소 공간	보통 서로 분리	공유
데이터 교환	파이프·메시지·공유 메모리 등의 IPC	공유 메모리로 쉽게 접근, 동기화 필요
오류 영향	주소 공간 격리로 완화 가능	잘못된 메모리 접근 등이 프로세스 전체에 영향 가능
실행 상태	자원과 실행 흐름을 함께 관리	스레드별 스택·레지스터 등

프로세스도 명시적으로 공유 메모리를 사용할 수 있다. 스레드별 스택이 따로 있다는 말은 같은 주소 공간의 다른 스레드에서 해당 메모리가 절대 접근 불가능하다는 뜻은 아니다.

동시성·병렬성·문맥교환

동시성(concurrency)은 여러 작업이 겹친 기간에 진행되는 성질이다. 단일 코어에서도 A 일부→B 일부→A 나머지처럼 번갈아 진행할 수 있다. 병렬성(parallelism)은 실제로 같은 시각에 여러 작업을 실행하는 것이다. 여러 코어 등이 있어야 CPU 연산을 병렬로 실행할 수 있다.

문맥교환은 A의 실행 상태를 저장하고 B의 상태를 복원하는 과정이다. 같은 프로세스의 스레드 사이 전환은 주소 공간 전환 부담이 적을 수 있지만 캐시 등 다른 비용도 존재한다. '스레드 전환은 항상 빠르다'고 단정하지 않는다.

준비: CPU를 기다림

실행: CPU 사용

대기: I/O 등 사건 기다림

사건 완료 후 준비로

준비 상태와 대기 상태의 차이는 CPU만 받으면 실행할 수 있는지다. 대기 중인 작업에 CPU를 줘도 기다리는 I/O가 끝나지 않으면 진행할 수 없다.

말로 설명하기 · 예시 답변

프로세스는 실행 중인 프로그램으로 자원과 주소 공간의 관리 단위입니다. 스레드는 그 안의 실행 흐름으로 코드·힙 등을 공유하지만 각자의 실행 상태와 스택을 가집니다. 공유로 통신이 쉬운 대신 경쟁 상태를 동기화해야 하고, 동시 실행이 곧 병렬 실행을 뜻하지는 않습니다.

▼ 연습 · 한 코어에서 A와 B를 번갈아 실행한다면 동시성인가요, 병렬성인가요?

동시성은 있지만 CPU 실행의 병렬성은 아니다. 시간 전체로 보면 둘 다 진척하지만 한 시점의 해당 코어에서는 하나의 명령 흐름을 실행한다는 모델이다.

▼ 꼬리질문 · 스레드는 모두 공유하니까 스택도 하나만 쓰나요?

각 스레드는 서로 다른 함수 호출 위치에서 실행될 수 있으므로 별도의 호출 스택과 레지스터 상태를 가진다. 같은 주소 공간을 쓴다는 것과 호출 상태를 하나로 합친다는 것은 다르다.

▼ 꼬리질문 · fork()를 호출하면 무엇이 달라지나요?

호출한 프로세스를 바탕으로 자식 프로세스가 생긴다. 성공 시 부모에게 자식 PID, 자식에게 0이 반환되어 코드에서 두 흐름을 구분할 수 있다. 메모리는 논리적으로 독립하며 실제 복사는 copy-on-

write로 늦출 수 있다. 이는 같은 프로세스 안에 스레드를 만드는 것과 다르다.

기존 프로세스·스레드, fork. 추가 읽기: OSTEP의 Process·CPU Virtualization 장.

21. 가상 메모리·페이지 폴트·캐시

가상 주소를 물리 주소로 바꾸는 과정을 계산하고, 캐시 미스·TLB 미스·페이지 폴트를 구분한다.

왜 주소를 한 번 더 변환할까

프로세스마다 '내가 사용할 주소 공간'을 제공하면 다른 프로그램의 배치를 몰라도 코드를 실행하고, 접근 권한도 관리하기 쉽다. 이때 프로그램이 사용하는 것이 가상 주소, RAM의 실제 위치가 물리 주소다. MMU라는 하드웨어가 운영체제가 준비한 매핑을 이용해 주소를 변환한다. 가상 메모리를 단순히 'RAM이 부족하면 디스크를 쓰는 기능'으로만 정의하면 주소 변환과 보호 목적을 놓친다.

페이징: 같은 크기로 나누고 대응표를 둔다

가상 공간의 고정 크기 조각은 페이지(page), RAM의 같은 크기 조각은 프레임(frame)이다. 페이지 테이블은 어느 페이지가 어느 프레임에 놓였는지, 접근 가능한지 등을 기록한다. 한 프로세스의 페이지들이 RAM에 연속해 있을 필요는 없다.

단위부터 계산해 보자

교육용으로 페이지 크기를 1,024바이트라고 하자. 가상 주소 2,500은 $2 \times 1,024 + 452$ 이므로 페이지 번호 2, 페이지 안 위치(offset) 452다. 페이지 2가 프레임 7에 매핑되면 물리 주소는 $7 \times 1,024 + 452 = 7,620$ 이다. 페이지 번호만 프레임 번호로 바뀌고 offset은 유지된다.

TLB·캐시는 서로 다른 것을 가까이 둔다

TLB(Translation Lookaside Buffer)는 최근 주소 변환 정보를 저장한다. CPU 캐시는 실제 명령·데이터의 일부를 가까이 저장한다. TLB에 변환이 없으면 페이지 테이블을 조회할 수 있고, 해당 페이지가 RAM에 정상적으로 있으면 페이지 폴트 없이 계속 실행할 수 있다. CPU 캐시 미스도 더 아래 메모리 계층에서 데이터를 가져오는 것이며 곧바로 디스크 접근을 뜻하지 않는다.

가상 주소

TLB / 페이지 테이블로 변환

권한·매핑 확인

캐시 / RAM 접근

개념 이해를 위한 순서다. 실제 하드웨어에서는 일부 조회가 겹치거나 캐시 설계에 따라 순서가 달라질 수 있다.

페이지 폴트는 운영체제의 처리가 필요한 예외

필요한 페이지가 현재 RAM에 없거나 접근 권한·매핑 문제로 바로 사용할 수 없으면 페이지 폴트가 발생할 수 있다. 운영체제는 합법적인 접근인지 확인한다. 디스크에 있는 유효한 페이지라면 프레임을 확보하고 데이터를 적재한 뒤 매핑을 갱신해 명령을 재실행할 수 있다. 잘못된 주소 접근이면 프로그램에 오류를 전달하거나 종료할 수 있다. 처음 쓰는 페이지를 0으로 채우거나 copy-on-write를 처리하는 등 디스크 읽기가 없는 경우도 있다.

페이지 교환을 손으로 해 보기

프레임 3개에서 FIFO는 들어온 지 가장 오래된 페이지를 내보낸다. 참조열 1,2,3,1,4이면 1·2·3에서 폴트 세 번, 다시 1은 적중, 4에서 1을 내보내 폴트 한 번: 총 4회다. FIFO는 최근에 사용했어도 처음 들어온 1을 교체한다.

LRU는 가장 오랫동안 사용하지 않은 것을 내보낸다. 같은 참조열에서는 마지막에 2를 내보낸다. OPT는 미래에 가장 늦게 쓸 것을 내보내는 이론적 기준이다. LFU는 사용 횟수가 적은 것을 고른다. working set은 일정 구간에 활발히 참조하는 페이지 집합이고, 이를 RAM이 수용하지 못해 실행보다 교체에 시간을 많이 쓰면 thrashing(스래싱)이다.

단편화와 지역성

내부 단편화는 할당된 블록 *안*의 낭비다. 4KiB 페이지로 6KiB를 담으면 8KiB를 차지해 마지막 2KiB가 남는다. 외부 단편화는 빈 공간이 흩어져 필요한 연속 공간을 못 얻는 문제다. 가변 크기 논리 구간을 쓰는 세그멘테이션에서 대표적으로 나타난다.

시간 지역성은 최근 사용한 것을 다시 쓰는 경향, 공간 지역성은 가까운 주소를 이어 쓰는 경향이다. 반복문이 같은 누적값을 갱신하면 시간 지역성, 배열을 차례대로 읽으면 공간 지역성의 예다. 캐시는 이를 이용해 평균 접근 지연을 줄인다.

말로 설명하기 · 예시 답변

가상 메모리는 프로그램의 주소와 물리 메모리 위치를 분리해 주소 공간과 보호를 제공합니다. 페이징은 페이지를 프레임에 매핑합니다. TLB는 주소 변환 정보, 캐시는 데이터·명령을 저장하며, 페이지 폴트는 매핑이나 접근을 처리해야 하는 예외로 모든 경우가 디스크 읽기는 아닙니다.

▼ 연습 · 페이지 크기 1,024, 가상 주소 3,500, 해당 페이지가 프레임 9라면 물리 주소는?

$3,500 = 3 \times 1,024 + 428$. 페이지 번호 3, offset 428. 물리 주소는 $9 \times 1,024 + 428 = 9,644$ 다. 매핑이 유효하고 접근 권한이 있다는 가정이다.

▼ 꼬리질문 · 페이지 크기를 크게 하면 무조건 좋아지나요?

같은 주소 공간에 필요한 페이지 테이블 항목 수가 줄고 TLB 하나가 덮는 범위는 커질 수 있다. 반면 내부 단편화와 불필요하게 가져오는 데이터가 늘 수 있다. 접근 패턴과 하드웨어 조건에 따른 절충이다.

▼ 꼬리질문 · 자료구조 힙과 메모리 힙은 같은 뜻인가요?

아니다. 자료구조 힙은 우선순위 규칙을 가진 트리형 구조, 메모리 힙은 동적 할당 공간을 부르는 이름이다. 메모리 스택은 호출 수명에 맞춰 쓰는 영역이다. C의 동적 할당은 malloc/free, 다른 언어는 언어별 수명 관리 규칙을 따른다.

원본 대응: 메모리·캐시, 가상 메모리, 요구 페이지징.

22. 스케줄링·경쟁 상태·교착상태

실행 순서를 표로 추적해 잘못된 결과가 생기는 이유와 동기화가 해결하는 범위를 설명한다.

CPU를 누구에게 얼마나 줄까

FCFS는 먼저 도착한 작업부터 실행한다. SJF는 다음 CPU 사용 구간이 짧은 작업을 먼저 고르지만 길이를 알아야 하거나 예측해야 한다. 우선순위 방식은 높은 우선순위를 먼저 고르며, 낮은 작업이 계속 밀리는 기아(starvation)가 생길 수 있다. 기다릴수록 우선순위를 높이는 aging으로 완화할 수 있다.

선점은 실행 중인 작업에서 CPU를 회수해 다른 작업으로 바꿀 수 있다는 뜻이다. Round Robin(RR)은 time quantum이라는 시간 조각만큼 번갈아 실행한다. 시간 조각이 작으면 반응 기회는 자주 오지만 문맥교환 부담이 커진다.

A는 5, B는 2만큼 CPU가 필요하고 둘 다 시각 0 도착

문맥교환 비용은 무시하고 A부터 시작한다. FCFS: A(0-5), B(5-7). 대기시간은 A=0, B=5, 평균 2.5. RR, quantum=2: A(0-2), B(2-4 완료), A(4-6), A(6-7 완료). 완료시간은 A=7, B=4, 대기시간은 A=7-5=2, B=4-2=2, 평균 2다. 응답시간은 최초 실행까지이므로 A=0, B=2다.

반환시간은 도착부터 완료까지, 대기시간은 준비 큐에서 기다린 시간, 응답시간은 처음 실행·응답할 때까지다. 위의 반환-CPU시간 계산은 I/O 대기가 없는 예제의 조건에서 쓴다. 목표가 응답성인지 처리량인지에 따라 평가가 달라진다.

한 줄 코드도 여러 단계로 실행될 수 있다

$x=0$ 에 두 작업이 각각 $x=x+1$ 을 실행한다고 하자. 이를 읽기→더하기→쓰기로 나눈 교육용 모델을 보자.

순서	작업 A	작업 B	공유 x
1	x 읽음: 0		0
2		x 읽음: 0	0
3	읽은 값+1을 씀		1
4		읽은 값+1을 씀	1

기대한 2가 아니라 1이 됐다. 실행 순서에 따라 결과가 달라지는 경쟁 상태(race condition)의 예다. 특정 언어의 실제 메모리 모델에서 이 코드를 안전하다고 가정하면 안 되며, 예를 들어 C/C++의 동기화되지 않은 데이터 경쟁은 정의되지 않은 동작이 될 수 있다.

임계구역을 하나의 작업만 수행하게 한다

임계구역은 공유 상태를 다루어 보호가 필요한 코드 구간, 상호배제는 그 구간에 한 번에 하나만 들어가게 하는 성질이다. mutex를 잠그고 읽기·더하기·쓰기를 모두 수행한 뒤 풀면 위 순서의 겹침을 막을 수 있다. 원자적 연산(atomic operation)은 다른 실행에서 중간 상태가 보이지 않는 하나의 연산처럼 처리되도록 제공되는 연산이다.

세마포어(semaphore)는 원자적인 wait/signal로 자원 수나 실행 순서를 제어한다. 자원 3개라면 최대 세 작업이 자원을 획득하도록 모델링할 수 있다. mutex는 보통 잠금 실행 주체가 풀어야 하는 소유권 규칙이 있고, 세마포어와 용도가 완전히 같지 않다. monitor는 공유 데이터와 접근 절차를 묶고 상호배제를 제공한다. 조건 변수는 조건을 기다리고 알리는 데 쓰며, 깨어났다고 조건이 여전히 참이라는 보장은 없어 일반적으로 조건을 반복 확인한다.

잠금이 새 문제를 만들 수도 있다

A는 잠금 L1을 잡고 L2를 기다리고, B는 L2를 잡고 L1을 기다리면 둘 다 진행하지 못한다. 이것이 교착상태(deadlock)다. 대표 필요 조건은 상호배제, 점유 대기, 비선점, 순환 대기다. 모든 작업이 L1→L2처럼 같은 순서로 잠금을 획득하도록 하면 이 예의 순환 대기를 깨뜨릴 수 있다.

예방은 조건 중 하나가 생기지 않도록 설계, 회피는 앞으로도 완료 순서가 가능한 안전 상태를 유지하며 자원 할당, 탐지·복구는 생긴 교착을 찾아 해소하는 접근이다. CPU 스케줄링의 선점과 교착 조건에서 말하는 자원 비선점은 구분한다.

말로 설명하기 · 예시 답변

공유 변수의 읽기·수정·쓰기가 겹치면 갱신이 사라지는 경쟁 상태가 생길 수 있습니다. 임계구역 전체를 mutex 등으로 보호해 상호배제를 확보합니다. 다만 여러 잠금을 서로 다른 순서로 잡으면 교착상태가 생길 수 있어 일관된 잠금 순서 등을 고려하겠습니다.

▼ 연습 · A=4, B=1, 둘 다 시각 0 도착. A부터 FCFS로 실행하면 평균 대기시간은?

A는 0, B는 4만큼 기다리므로 평균 $(0+4)/2=2$ 다. B부터 SJF로 실행하면 B=0, A=1로 평균 0.5다. 실행 길이를 알고 I/O와 전환 비용을 무시한 예다.

▼ 꼬리질문 · 교착상태와 기아는 어떻게 다른가요?

교착은 관련 작업들이 서로 자원을 기다려 진행하지 못하는 상태다. 기아는 다른 작업은 진행하지만 특정 작업이 계속 선택되지 않는 상태다. 우선순위 aging은 기아 완화책이지 모든 교착의 해결책이 아니다.

▼ 꼬리질문 · 비교적 작은 코드인데 잠금을 빼도 되지 않나요?

코드 길이보다 공유 상태의 접근 방식과 원자성·실행 순서가 중요하다. 안전한 원자 연산이나 공유하지 않는 설계로 대체할 수는 있지만, 짧다는 이유로 동기화를 생략할 근거는 없다.

기존 스케줄링·동기화·모니터·분산. 추가 읽기: OSTEP의 Concurrency·Locks·Semaphores 장.

23. 네트워크: 주소를 입력한 뒤 생기는 일

프로토콜·계층·주소의 역할을 하나의 통신 흐름으로 연결하고 TCP/UDP를 요구 조건에 따라 비교한다.

프로토콜은 서로 이해하기 위한 약속

프로토콜(protocol)은 주고받는 메시지의 형식·의미·순서에 대한 약속이다. HTTP는 웹 요청과 응답의 규칙, IP는 네트워크 간 전달에 필요한 주소와 패킷 규칙, TCP는 프로세스 간 순서 있는 바이트 전달을 다룬다. 계층은 역할을 나눠 아래쪽 구현이 달라도 위쪽 기능을 사용할 수 있게 한다.

OSI 계층, 위→아래	처음 이해할 역할	TCP/IP 4계층과 대략 대응
응용·표현·세션	응용 통신·표현 방식·대화 관리	응용
전송	프로세스 사이 전달, TCP/UDP	전송
네트워크	네트워크 사이 패킷 전달, IP	인터넷
데이터링크·물리	한 링크의 프레임과 실제 신호	링크/네트워크 접근

이 대응은 학습용이며 실제 프로토콜이 OSI의 모든 경계에 정확히 들어맞는 것은 아니다. 캡슐화는 상위 데이터를 담고 각 계층에 필요한 제어 정보를 덧붙이는 과정이다.

이름·IP·MAC·포트는 다른 질문에 답한다

DNS는 도메인 이름에 대한 IP 주소 등 레코드를 조회하는 시스템이다. **IP 주소**는 네트워크 간 전달에, **MAC 주소**는 해당 링크의 프레임 전달에 쓰는 인터페이스 주소다. **포트 번호**는 전송 계층에서 통신 끝점을 구분하는 데 쓰인다. 하나의 기기에도 여러 서비스가 있으므로 IP만으로 어떤 서비스와 통신할지 충분하지 않다.

IPv4 LAN에서 ARP는 다음 홉의 IPv4 주소에 대응하는 링크 주소를 알아내는 데 사용한다. 외부 서버에 보낸다면 보통 원격 서버 MAC을 찾는 것이 아니라 같은 링크에 있는 기본 게이트웨이의 MAC을 찾는다. IP/MAC 변환과 가상/물리 메모리 주소 변환은 별개다.

간단한 웹 요청을 따라가기

1. 브라우저가 서버 이름을 해석한다. DNS 캐시가 있으면 조회 일부가 생략될 수 있다.
2. 목적지 IP로 가기 위한 경로와 다음 홉을 정한다. 같은 LAN의 다음 홉 주소가 필요하다면 ARP 등을 사용한다.
3. HTTPS를 TCP 기반으로 사용하는 예에서는 TCP 연결 후 TLS로 서버 인증과 암호화 통신을 준비한다.
4. HTTP 요청을 보내고 응답을 받는다. 응답에는 HTML·다른 리소스를 요청할 정보 등이 들어갈 수 있다.
5. 브라우저가 받은 내용을 해석하고 화면을 구성한다.

기존 연결·캐시·프록시 여부에 따라 생략되거나 달라지는 단계가 있다. 모든 HTTP가 TCP 위에서 동작하지는 않으며 HTTP/3는 QUIC을 사용한다. 처음에는 위 TCP 기반 예를 이해한 뒤 예외를 구분한다.

TCP와 UDP: 무엇을 보장해 주는가

TCP는 연결 지향의 신뢰성 있는 순서 있는 **바이트 스트림**을 제공한다. 순서 번호·확인 응답·재전송 등을 사용한다. 애플리케이션이 ‘안녕’과 ‘반가워’를 두 번 보냈다고 수신도 정확히 두 번에 나뉘는 것은 아니다. 메시지 경계는 길이 정보 등의 응용 규칙으로 정해야 한다. 연결이 실패하면 데이터 전달 완료가 항상 보장되는 것도 아니다.

UDP는 데이터그램 단위로 보내고 자체적으로 도착·순서·재전송을 보장하지 않는다. 필요하다면 응용이 보완한다. 낮은 지연이나 최신 정보가 중요하다고 해서 무조건 UDP를 고르지 말고, 손실 시 동작과 신뢰성 요구를 설명한다. **흐름 제어**는 수신자가 감당하는 양을, **혼잡 제어**는 네트워크가 감당하는 양을 고려한다.

TCP 연결 설정의 기본 예: 클라이언트 SYN(x) → 서버 SYN(y)+ACK(x+1) → 클라이언트 ACK(y+1). 서로의 초기 순서 번호를 확인하고 양방향 통신 상태를 준비한다. SYN은 연결 동기화 요청, ACK는 확인 응답이다. 세 메시지의 암기보다 ‘누가 무엇을 확인했는가’를 말한다.

말로 설명하기 · 예시 답변

TCP는 순서 있는 바이트 스트림의 신뢰성 있는 전달을 제공하고 UDP는 데이터그램을 보내지만 자체적으로 도착과 순서를 보장하지 않습니다. 데이터 누락을 허용할 수 있는지, 지연과 응용의 복구 요구가 무엇인지에 따라 선택하겠습니다. IP·MAC·포트는 각각 전달 범위와 통신 끝점을 구분하는 역할이 다릅니다.

▼ 연습 · 다른 네트워크의 서버로 보내는데 왜 공유기 MAC 주소가 필요한가요?

현재 링크에서 프레임을 전달할 직접 상대가 기본 게이트웨이이기 때문이다. 다음 링크에서는 그 링크에 맞는 전달이 이루어진다. 인터넷 전체 경로를 원격 서버 MAC 하나로 전달하는 것이 아니다.

▼ 꼬리질문 · TCP면 암호화까지 되는 건가요?

아니다. TCP의 신뢰성과 순서 보장은 기밀성과 별개다. HTTPS에서는 TLS가 암호화와 상대 인증 등을 담당한다. UDP 위에서도 별도 보안 프로토콜을 사용할 수 있다.

▼ 꼬리질문 · 흐름 제어와 혼잡 제어는 왜 따로 필요한가요?

수신 버퍼가 넉넉해도 경로의 라우터나 링크가 과부하일 수 있고, 네트워크가 여유로워도 수신 프로그램이 느릴 수 있다. 제한이 생기는 위치가 다르므로 각각을 고려해야 한다.

기존 OSI·IP·ARP. 정의 확인: RFC 9293 TCP, RFC 768 UDP, RFC 826 ARP. 위 흐름과 예제는 학습용으로 구성했다.

24. 객체지향을 배운 뒤 가상 함수를 읽자

클래스·객체·상속을 먼저 이해하고, 다형성과 동적 바인딩을 실제 호출 결과로 설명한다.

데이터와 동작을 함께 묶는다

객체(object)는 상태와 동작을 함께 다루는 대상이다. 상태는 계좌의 잔액, 동작은 입금·출금처럼 생각한다. 클래스(class)는 어떤 상태와 동작을 갖는 객체를 만들지 정의한 타입이다. 같은 Account 클래스에서 만들어도 A계좌 잔액은 10, B계좌 잔액은 20으로 다를 수 있다. 메서드는 객체에 관련된 함수다.

캡슐화는 관련 상태와 동작을 묶고 접근 방식을 관리하는 것이다. 잔액을 아무 값으로 덮어쓰게 두지 않고 출금 메서드에서 잔액 부족을 검사한다. **추상화**는 사용에 중요한 인터페이스를 드러내고 세부 구현을 감추는 것이다. ‘출금’ 동작을 쓰는 사람이 내부 저장 형식까지 알 필요는 없다. 둘은 관련되지만 같은 정의는 아니다.

상속과 다형성은 어떻게 연결될까

상속(inheritance)은 기존 타입을 바탕으로 새 타입을 정의하고 확장하는 방식이다. Dog는 Animal의 한 종류라는 관계로 생각할 수 있다. 다형성(polymorphism)은 공통 인터페이스로 서로 다른 타입을 다룰 때 각 타입에 맞는 동작을 수행하는 성질이다. 모든 Animal에 speak()를 요청하면서 Dog는 다른 방식으로 말하도록 할 수 있다.

```
// C++ 예제: 문법을 모두 암기하기보다 호출 대상을 추적한다.
struct Animal {
    virtual const char* speak() { return "animal"; }
    virtual ~Animal() = default;
};
struct Dog : Animal {
    const char* speak() override { return "dog"; }
};
Dog d;
Animal* p = &d;
// p->speak() 의 결과는 "dog"
```

struct는 타입을 정의한다. Dog : Animal은 상속, &d는 d의 주소, Animal*은 Animal을 가리키는 포인터 타입이다. ->는 포인터가 가리키는 객체의 멤버에 접근한다. 변수 p의 선언 타입은 Animal*이지만 실제

객체는 Dog다. virtual은 이런 호출에서 실제 객체 타입에 맞는 재정의의 선택하도록 한다. override는 올바르게 재정의했는지 컴파일러가 확인하게 한다. 소멸자 줄은 이 예의 기반 클래스 사용을 안전하게 하기 위한 선언이며 첫 회독에서는 speak 호출에 집중한다.

이 절은 원본 노트의 C++ 내용을 보존한 비교 학습이다. 아래 class·virtual·int&·STL은 C 문법이 아니다. C는 함수 이름에 따른 일반적인 오버로딩, 클래스 상속, 참조 매개변수를 제공하지 않는다. C 기반 객체 표현과 상태 조작은 구조체와 포인터·함수로 구분해서 설계한다.

오버로딩과 오버라이딩 · C++ 비교

오버로딩(overloading)은 같은 이름에 서로 다른 매개변수 목록을 두는 것이다. C++의 print(int), print(double)이 예다. 반환형만 달리해서 같은 매개변수 함수 둘을 구분할 수는 없다. 오버라이딩(overriding)은 상속받은 가상 함수를 파생 타입에 맞게 재정의하는 것이다. 단순히 같은 이름의 함수를 새로 만들었다고 모든 언어에서 항상 오버라이딩이 되는 것은 아니다.

바인딩(binding)은 이름·호출과 대상의 연결이다. 정적 바인딩은 실행 전 결정, 동적 바인딩은 실행 중 실제 객체 타입 등에 따라 결정한다. vtable은 가상 함수 호출을 구현하는 흔한 방식이지만 모든 컴파일러가 반드시 사용해야 하는 유일한 구현은 아니다.

값 전달·참조 전달

```
// C++: a는 처음에 1
void by_value(int x) { x = 9; }    // 복사된 x만 변경
void by_ref(int& x) { x = 9; }     // 호출자 변수의 별칭을 변경
// by_value(a) 이후 a=1, by_ref(a) 이후 a=9
```

참조 전달은 원본을 수정할 수 있다는 뜻이지 반드시 훼손한다는 뜻이 아니다. const 참조로 수정을 제한할 수도 있다. C에서 포인터 값을 전달하는 방식과 C++의 참조 매개변수는 구분한다. C에는 int& 참조 매개변수 문법이 없다.

말로 설명하기 · 예시 답변

객체지향은 상태와 동작을 객체로 묶어 프로그램을 구성하는 방식입니다. 캡슐화로 상태 접근을 관리하고 추상화로 필요한 인터페이스를 제공합니다. 상속과 가상 함수 기반 다형성을 사용하면 공통 인터페이스 호출이 실제 객체 타입에 맞게 동작할 수 있습니다.

▼ 연습 · 위 코드에서 포인터 p의 타입과 실제 객체 타입은 각각 무엇인가요?

p의 선언 타입은 Animal*, 실제 객체 d의 타입은 Dog다. 가상 함수 호출이므로 Dog::speak가 선택된다. '포인터가 Animal이니 항상 Animal 함수'라고 답하면 선언 타입과 실제 객체를 혼동한 것이다.

▼ 꼬리질문 · 코드 재사용을 위해 항상 상속을 쓰면 좋나요?

아니다. 'is-a' 관계와 대체 가능성이 타당한지 봐야 한다. 단지 기능을 이용하는 관계라면 객체를 멤버로 갖고 동작을 위임하는 합성(composition)이 더 적절할 수 있다. 상속은 기반 타입 변경의 영향을 크게 받을 수도 있다.

▼ 꼬리질문 · 순수 가상 함수와 일반 가상 함수는 어떻게 다른가요?

C++의 `virtual void f() = 0;`처럼 순수 가상 함수를 선언하면 해당 클래스는 추상 클래스가 되어 직접 객체를 만들 수 없다. 구체 타입으로 사용하려면 필요한 순수 가상 함수를 구현해야 한다. 일반 가상 함수는 기본 구현을 제공하며 재정의할 수 있다.

원본 순서를 바꾸어 객체지향 기초를 먼저 배우고 바인딩, 참조:순수 가상 함수·STL로 연결했다.

25. 데이터베이스: 표를 나누는 이유부터

키·함수 종속·정규화·JOIN·트랜잭션을 작은 데이터로 설명한다. 정의에 나오는 단어를 이해한 다음 정규형을 읽는다.

파일로 저장하면 되는데 왜 DBMS가 필요한가

파일에 기록하는 것만으로 데이터를 저장할 수는 있다. 여러 사용자의 동시 수정, 조건 검색, 제약 검사, 장애 복구 등을 프로그램마다 직접 구현하려면 복잡해진다. 데이터베이스는 조직된 데이터의 모음이고 DBMS는 이를 관리하는 소프트웨어다. 관계형 DB는 데이터를 테이블로 다루며 행(row/tuple)은 한 기록, 열(column/attribute)은 속성이다. 스키마는 테이블 구조와 제약 등의 정의다.

학번	이름	학과	학과전화	과목	점수
101	민수	컴공	1000	OS	90
101	민수	컴공	1000	DB	85
102	지수	컴공	1000	OS	95

예제 가정: 학생은 학과 하나에 속하고 학과마다 대표 전화 하나가 있으며 학생은 같은 과목을 한 번 수강한다. 실제 제도가 그렇다는 뜻이 아니라 종속성을 분석하기 위한 조건이다. 한 행은 (학번,과목)으로 구분된다.

키: 행을 구분하는 최소한의 근거

슈퍼키는 행을 유일하게 식별하는 속성 집합이다. 위에서 {학번,과목}도, {학번,과목,이름}도 슈퍼키다. 후보키는 속성을 더 빼면 유일성이 깨지는 최소 슈퍼키다. 여기서는 {학번,과목}이다. 후보키 하나를 대표로 고르면 기본키다. ‘최소’는 전체 후보키 중 열 수가 가장 적다는 뜻이 아니라 **그 집합 안에서 더 뺄 수 없다**는 뜻이다.

외래키는 다른 행의 적절한 유일 키를 참조하는 속성 집합이다. 예를 들어 수강.학번이 학생.학번을 참조한다. 기본키는 NULL과 중복을 허용하지 않으며, 외래키의 NULL 허용 여부는 별도 제약에 달린다. 존재하지 않는 학생 번호를 수강에 넣지 못하게 하는 것이 참조 무결성의 예다.

함수 종속: 같은 X면 Y도 같아야 한다

$x \rightarrow y$ 는 X값이 같은 행들은 Y값도 같아야 한다는 의미다. 위 가정에서 학번 $\rightarrow$ 이름, 학번 $\rightarrow$ 학과, 학과 $\rightarrow$ 학과전화, (학번,과목) $\rightarrow$ 점수가 성립한다. 데이터 몇 줄이 우연히 그렇게 보이는 것이 아니라 관리할 데이터의 규칙으로 정해야 한다. X는 결정자, 후보키에 포함되는 속성은 주요 속성이라 부른다.

학과 전화가 바뀌면 여러 행을 바꿔야 하는 갱신 이상, 학생이 아직 수강하지 않으면 정보를 넣기 어려운 삽입 이상, 마지막 수강을 지우다 학생·학과 정보도 잃는 삭제 이상이 생길 수 있다.

분해 과정을 한 단계씩

1. **1NF**: 한 칸에서 모델이 정한 하나의 값을 다룬다. 과목 칸에 'OS,DB' 목록을 넣었다면 수강 행을 나누는 식으로 정리한다. 위 표는 이미 그 형태다.
2. **2NF**: 후보키 일부에만 의존하는 비주요 속성을 분리한다. 이름·학과·학과전화는 (학번,과목) 중 학번만으로 결정되므로 학생(학번,이름,학과,학과전화)과 수강(학번,과목,점수)으로 나눈다.
3. **3NF**: 학생 표에서 학번 $\rightarrow$ 학과 $\rightarrow$ 학과전화의 관계를 보자. 학생(학번,이름,학과)과 학과(학과,학과전화)로 나누면 전화번호를 학과당 한 곳에서 관리한다.

학생: 학번·이름·학과

수강: 학번·과목·점수

학과: 학과·전화

분해 후 원래 정보를 정확히 복원할 수 있어야 하는 것이 **무손실 조인**이다. 아무렇게나 쪼개면 없던 행 조합이 생길 수 있다. 제약을 각 표에서 검사할 수 있는 종속성 보존도 고려한다.

▼ 심화: 3NF와 BCNF의 정확한 조건

비자명 함수 종속 $X \rightarrow A$ 에서 '비자명'은 A가 X에 이미 포함된 속성이 아니라는 뜻이다. 3NF는 모든 이런 종속에 대해 X가 슈퍼키이거나 A가 주요 속성이어야 한다. BCNF는 모든 비자명 함수 종속의 결정자 X가 슈퍼키여야 한다. 그래서 BCNF가 더 강하다. 모든 3NF를 단순히 '비키끼리 종속 없음'이라고 정의하면 복합 후보키 사례를 놓친다. 4NF·5NF는 별도 다차·조인 종속을 다루며 기존 상세를 참고한다.

JOIN: 나눈 표를 필요할 때 연결한다

```
SELECT s.name, e.course, e.score
FROM Student AS s
JOIN Enrollment AS e ON s.id = e.student_id
WHERE e.score >= 90;
```

영어 열 이름을 쓰는 위 SQL은 Student=학생, Enrollment=수강이다. SELECT는 출력할 열, FROM은 대상 표, JOIN ... ON은 연결 조건, WHERE는 선택할 행 조건이다. 위 데이터라면 (민수,OS,90), (지수,OS,95) 두 행이다. 출력 순서는 ORDER BY가 없으면 보장되지 않는다. INNER JOIN은 조건에 맞는 행만, LEFT JOIN은 왼쪽 행을 보존하며 짝이 없으면 오른쪽을 NULL로 채운다. JOIN이 항상 나쁜 연산은 아니다.

트랜잭션과 ACID

A에서 10을 빼 B에 더하는 이체가 중간에 실패하면 돈이 사라지면 안 된다. 트랜잭션은 이런 여러 조작을 하나의 논리적 작업으로 묶는다. 원자성(Atomicity)은 전부 반영 또는 전부 취소, 일관성(Consistency)은 정의한 무결성 규칙을 지키는 상태 변화, 격리성(Isolation)은 동시 작업의 간섭을 정해진 수준으로 제어, 지속성(Durability)은 커밋된 결과의 보존이다. 실제 격리 수준에 따라 허용하는 현상은 다르다.

COMMIT은 확정, ROLLBACK은 취소다. 일관성이 응용의 모든 현실적 오류를 자동으로 고친다는 뜻은 아니다. DB 잠금은 동시성 제어 수단 중 하나이며 MVCC처럼 여러 버전을 사용하는 방식도 있다.

인덱스는 검색을 돕는 보조 구조다. 조회가 빨라질 수 있지만 추가 공간과 삽입·갱신 시 유지 비용이 든다. 테이블 전체를 읽는 편이 유리한 쿼리도 있어 모든 열에 인덱스를 추가하는 것이 답은 아니다. B+tree는 16장의 범위 검색 설명과 연결된다.

말로 설명하기 · 예시 답변

정규화는 함수 종속 등을 바탕으로 테이블을 분해해 중복과 갱신·삽입·삭제 이상을 줄이는 과정입니다. 학생의 학과 전화가 수강마다 반복된다면 학생·수강·학과 표로 나누고 필요할 때 JOIN합니다. 정보가 손실 없이 복원되는지와 조인·인덱스 비용도 함께 고려하겠습니다.

▼ 연습 · 위 표에서 학번만 기본키로 삼으면 왜 문제가 되나요?

한 학생이 여러 과목을 수강해 101이 여러 행에 나타나므로 수강 기록을 유일하게 식별하지 못한다. 예제 가정에서는 (학번,과목)이 후보키다. 분리된 학생 표에서는 학번 하나가 키가 될 수 있다.

▼ 꼬리질문 · 원자성과 격리성은 무엇이 다른가요?

원자성은 한 트랜잭션의 일부만 남지 않게 하는 성질이고, 격리성은 여러 트랜잭션이 동시에 실행될 때 서로의 중간 상태·간섭을 어떻게 다루는가다. 하나의 작업을 전부 취소할 수 있어도 다른 작업이 잘못된 중간 상태를 읽는 문제는 별도로 고려해야 한다.

▼ 꼬리질문 · 잠금 단위를 작게 하면 무조건 빨라지나요?

서로 다른 행에 대한 작업이 동시에 진행할 여지는 늘지만 관리할 잠금 수와 비용도 커질 수 있다. 큰 단위는 관리가 단순한 대신 더 넓은 범위의 작업을 막는다. 실제 성능은 접근 패턴·경합·DB 구현에 달린다.

원본 파일·스키마, 정규형·키·로킹. 연산 의미 확인: PostgreSQL JOIN 튜토리얼, 트랜잭션 튜토리얼. 학생 데이터와 문제는 새로 구성했다.

26. 이산수학: 기호를 문장으로 읽기

집합·함수·논리·관계의 기호를 한국어 문장으로 풀고, 간단한 증명의 근거를 말한다.

집합은 ‘어떤 원소가 들어 있는가’

$A=\{1,2\}$ 에서 $1\in A$ 는 1이 A의 원소라는 뜻이다. $A\subseteq B$ 는 A의 모든 원소가 B에도 들어 있다는 뜻이다. $\emptyset$ 는 원소가 없는 공집합이다. $A\cup B$ 는 합집합, $A\cap B$ 는 교집합이다. 집합은 순서·중복을 따로 세지 않아 $\{1,2\}=\{2,1,1\}$ 이다. $|A|$ 는 원소 수다.

멱집합 $P(A)$ 는 모든 부분집합을 모은 집합이다. $A=\{a,b\}$ 이면 $P(A)=\{\emptyset,\{a\},\{b\},\{a,b\}\}$ 다. 원소마다 포함·미포함 두 선택이 있어 n 개 원소의 부분집합 수는 2^n 이다. $\emptyset$ 와 $\{\emptyset\}$ 는 다르다. 전자는 원소가 0개, 후자는 공집합을 원소로 갖는 1개짜리 집합이다.

명제·조건·대우

명제는 참·거짓을 판단할 수 있는 문장이다. $P\wedge Q$ 는 둘 다 참, $P\vee Q$ 는 적어도 하나가 참, $\neg P$ 는 P의 부정이다. $P\Rightarrow Q$ 는 ‘P이면 Q’다. P가 참인데 Q가 거짓인 경우에만 이 함의가 거짓이다. $Q\Rightarrow P$ 라는 역은 원래 명제와 다르고, $\neg Q\Rightarrow\neg P$ 라는 대우는 원래 명제와 논리적으로 동치다.

‘4의 배수이면 짝수다’는 참. 역인 ‘짝수이면 4의 배수다’는 6 때문에 거짓. 대우 ‘짝수가 아니면 4의 배수가 아니다’는 참이다. 충분조건·필요조건을 말할 때 원래 화살표 방향을 먼저 적어 본다.

$\forall x$ 는 ‘모든 x에 대해’, $\exists x$ 는 ‘어떤 x가 존재해’다. ‘모든 입력에서 성공’의 부정은 ‘모든 입력에서 실패’가 아니라 ‘실패하는 입력이 하나라도 존재’다. 알고리즘의 정확성 주장과 반례를 이해할 때 중요하다.

함수: 입력마다 출력 하나

$f:X\rightarrow Y$ 에서 X는 정의역, Y는 공역, 실제 출력 $f(X)$ 는 치역이다. $X=\{1,2\}$, $Y=\{2,4,6\}$, $f(x)=2x$ 라면 치역은 $\{2,4\}$. 서로 다른 입력이 같은 출력으로 합쳐지지 않으면 단사, 공역의 모든 값을 실제 출력으로 만들 수 있으면 전사, 둘 다면 전단사다. 이 예는 단사이지만 6을 만들지 못해 전사가 아니다.

Y 전체에서 원래 입력을 되찾는 역함수가 존재하려면 전단사여야 한다. 위 예는 공역을 $\{2,4\}$ 로 제한하면 역함수를 만들 수 있다. ‘단사이면 원래 공역 전체의 역함수가 있다’고 외우면 안 된다.

관계: 두 대상을 연결하는 조건

$A \times B$ 는 가능한 순서쌍 전체이고 관계 R 은 그 일부다. $A=\{1,2\}$, $B=\{a,b\}$ 이면 $A \times B = \{(1,a),(1,b),(2,a),(2,b)\}$ 다. 함수는 각 입력에 출력이 정확히 하나씩 있는 특별한 관계다.

성질	뜻	예
반사성	모든 x 에 xRx	$x \leq x$
대칭성	xRy 이면 yRx	같은 나머지 관계
추이성	xRy, yRz 이면 xRz	$x \leq y \leq z$ 이면 $x \leq z$
반대칭성	xRy 와 yRx 이면 $x=y$	$\leq$
비대칭성	xRy 이면 yRx 가 아님	$<$

반사·대칭·추이를 만족하면 동치관계다. 정수를 3으로 나눈 나머지가 같은 관계에서는 1,4,7이 같은 동치류에 속한다. 반사·반대칭·추이를 만족하는 관계는 부분순서다. 모든 두 원소가 비교 가능해야 한다는 조건은 따로 추가해야 한다.

수학적 귀납법: 도미노의 시작과 연결

$1+\cdots+n=n(n+1)/2$ 를 $n \geq 1$ 에서 보이자. **기본 단계:** $n=1$ 에서 $1=1$. **귀납 단계:** k 에서 맞다고 가정하면 $k+1$ 까지의 합은 $k(k+1)/2+(k+1)=(k+1)(k+2)/2$ 이므로 다음에서도 맞다. 기본 단계가 첫 도미노, 귀납 단계가 ‘하나가 쓰러지면 다음도 쓰러진다’는 연결이다. 몇 개의 숫자를 대입해 본 것만으로 모든 n 에 대한 증명이 되지는 않는다.

귀류법은 결론의 부정을 가정하고 모순을 얻는 방식이다. 가장 큰 자연수 m 이 존재한다고 가정하면 $m+1$ 이 더 크므로 모순이다. 집합 상등 $A=B$ 는 $A \subseteq B$ 와 $B \subseteq A$ 를 각각 증명할 수 있다.

말로 설명하기 · 예시 답변

함수는 정의역의 각 입력에 공역의 출력 하나를 대응시킵니다. 공역과 실제 치역은 다를 수 있고, 공역 전체에서 역함수를 가지려면 전단사여야 합니다. 보편적인 명제는 예를 몇 개 보는 것보다 귀납법이나 반례·논리 구조로 확인해야 합니다.

▼ 연습 · $A=\{1,2,3\}$ 의 부분집합 수와 공집합의 역집합은?

$|P(A)|=2^3=8$. $P(\emptyset)=\{\emptyset\}$ 로 원소 수는 1이다. ‘없음을 고르는 방법’ 한 가지가 있다. $2^0=1$ 과 일치한다.

▼ 꼬리질문 · 반대칭이면 대칭성이 성립하지 않는다는 뜻인가요?

아니다. 반대칭은 서로 다른 두 원소가 양방향으로 관계를 가지면 안 된다는 조건이다. 등호 관계는 대칭이면서 반대칭이다. ‘반대칭’과 ‘대칭의 부정’을 혼동하지 않는다.

▼ 꼬리질문 · ‘모든 경우 $O(n)$ 이다’라는 주장에 반례 하나면 충분한가요?

어떤 보편 주장인지 정확히 해야 한다. ‘어떤 입력에서도 비용이 특정 상한 이내’라는 주장에는 그 상한을 어기는 입력이 반례다. 그러나 점근적 $O(n)$ 은 충분히 큰 n 과 상수의 존재를 말하므로, 작은 n 하나에서 느렸다는 실측만으로 깨지지 않는다. 입력 집합에 따른 비용 증가를 구분한다.

원본 함수·집합·증명·관계를 기호 입문부터 재구성하고 명제·대우·양화사를 보충했다.

27. 남은 원본 범위와 자주 혼동하는 연결

앞 장에서 깊이 다루지 않은 원본 항목을 위치와 의미까지 알고, 모르는 용어를 다른 개념과 섞지 않는다. 자세한 조건·변형은 하단 원본 대응표에서 찾는다.

검색·정렬의 변형

Trie는 문자열의 문자 등 키의 조각을 따라 내려가는 구조다. cat과 car는 c→a 경로를 공유하고 다음 t/r에서 갈라진다. 접두사 검색에 자연스럽다. 한 단계 자식 접근 비용을 일정하게 보면 길이 L의 키 검색은 $O(L)$ 이다. 키 개수 n에 대한 $O(\log n)$ 과 입력 기준부터 다르다.

보간 탐색은 정렬된 수치 키의 값 분포로 위치를 추정한다. 균등 분포 등 조건에서 평균 $O(\log \log n)$ 을 기대할 수 있지만 최악은 $O(n)$ 이다. **셸 정렬**은 간격을 둔 삽입 정렬을 하고 간격을 줄여 마지막 1로 마무리한다. 성능은 간격 수열에 따라 달라진다. **MSD 기수 정렬**은 높은 자리부터 그룹을 나누고, **LSD**는 낮은 자리부터 안정 정렬을 반복한다.

외부 정렬은 데이터가 RAM에 다 들어가지 않을 때 저장장치 I/O를 고려하는 정렬이다. 큰 파일을 메모리에 들어갈 조각으로 정렬해 저장하고, 정렬된 여러 조각의 맨 앞을 보며 병합할 수 있다. 정렬된 조각을 run이라 부른다. CPU 비교 수뿐 아니라 블록을 얼마나 읽고 쓰는지 중요하다.

해시·암호화·서명은 목적부터 구분

암호화는 키를 가진 쪽이 복호화할 수 있도록 내용을 숨기는 데 사용한다. 대칭키 방식은 공유한 비밀키를 쓰고, 공개키 방식은 공개키·개인키 쌍을 사용한다. 암호학적 해시는 메시지에서 고정 길이 요약값을 만들며 역상·충돌 공격에 대한 저항성 등이 요구된다. 일반적으로 원문 복원 기능을 제공하지 않는다.

디지털 서명은 개인키로 서명하고 공개키로 검증하여 메시지 출처·변경 여부를 검증하는 데 사용한다. 서명만으로 메시지 내용이 숨겨지는 것은 아니다. 서명을 단순히 ‘개인키로 암호화’라고 일반화하지 않는다. 키 합의는 양측이 공유 비밀을 정하는 절차이고, 인증 없는 키 합의만으로 상대의 신원까지 보장되지는 않는다.

기밀성은 허가되지 않은 쪽에서 내용을 읽지 못하게, 무결성은 무단 변경을 탐지·방지하게, 가용성은 필요할 때 사용할 수 있게 하는 목표다. 해시값만 메시지 옆에 붙이면 공격자가 둘 다 바꿀 수 있으므로 인증까지 자동 보장하지 않는다. 비밀번호 저장용 해시는 빠른 일반 해시와 목적이 다르다.

장치·파일·스키마

인터럽트는 장치 등의 사건에 대응하도록 CPU에 알리는 메커니즘이다. 폴링은 소프트웨어가 상태를 반복 확인하는 방식이다. 대기 부하·응답 지연·사건 빈도에 따른 절충이 있다. **daisy chain**은 장치들을 따라 승인 신호를 전달해 인터럽트 우선순위를 정하는 하드웨어 방식의 예다. 네트워크 패킷의 경로 선택과 같은 뜻이 아니다.

HDD 스케줄링의 SSTF는 현재 헤드에서 가까운 요청부터, SCAN은 한 방향으로 이동하다 끝에서 방향을 바꾸는 방식이다. SSD는 움직이는 헤드가 없으므로 같은 기계적 모델을 그대로 적용하지 않는다.

파일 시스템은 이름·디렉터리·저장 위치·접근 권한 등을 관리한다. DB의 **외부 스키마**는 사용자별 뷰, **개념 스키마**는 전체 논리 구조, **내부 스키마**는 물리 저장 관점이다. DDL은 구조 정의, DML은 데이터 조회·변경, DCL은 권한 제어에 관한 분류다. 관계대수는 선택·투영·조인 같은 연산으로 질의를 표현하고, 관계해석은 결과가 만족할 조건으로 표현한다. 관계대수의 selection은 행 선택, projection은 열 선택이다.

분산 처리와 동적 계획법: 새로 보충하는 연결

분산 시스템은 여러 컴퓨터가 네트워크로 협력하는 시스템이다. 한 컴퓨터 내부 스레드와 달리 통신 지연과 일부 노드만 실패하는 부분 장애를 고려해야 한다. 컴퓨터 수를 늘린다고 항상 처리 시간이 줄거나 가용성이 높아지는 것은 아니다.

동적 계획법(DP)은 겹치는 부분문제의 결과를 저장해 재사용하는 설계법이다. 피보나치 $F(n)=F(n-1)+F(n-2)$ 를 그대로 재귀 계산하면 같은 $F(k)$ 를 반복 계산한다. $F(0)=0, F(1)=1$ 에서 1,2,3,5...를 차례로 저장하면 n 번째까지 $O(n)$ 에 계산할 수 있다. 직전 두 값만 필요하면 추가 공간 $O(1)$ 도 가능하다. 최적화 문제에서는 최적 부분구조 등 DP로 풀 수 있는 근거와 상태·전이를 설명해야 한다.

말로 설명하기 · 예시 답변

익숙한 이름이 비슷해도 목적과 비용 기준을 먼저 구분하겠습니다. 예를 들어 해시 요약은 원문 복호화가 목적이 아니고, 디지털 서명은 내용의 기밀성을 자동으로 제공하지 않습니다. 자료구조도 키 개수·키 길이·블록 I/O 중 무엇을 비용으로 보는지에 따라 선택이 달라집니다.

▼ 연습 · 파일 내용과 해시값을 함께 받았다. 이것만으로 발신자가 진짜인지 알 수 있나요?

아니다. 믿을 수 있는 경로의 기준 해시인지, 인증된 MAC(메시지 인증 코드)이나 서명 검증이 있는지 등이 필요하다. 공격자가 파일과 해시를 모두 바꾸면 단순 재계산 비교만으로 발신자를 인증할 수 없다. 이 MAC은 네트워크 인터페이스의 MAC 주소와 다른 약어다.

▼ 꼬리질문 · 메모이제이션을 사용한 재귀는 추가 공간 $O(1)$ 인가요?

일반적으로 아니다. 저장한 부분문제 결과와 호출 스택 공간을 모두 본다. 피보나치를 n 개 상태로 저장하면 $O(n)$ 공간이다. 직전 두 값만 쓰는 반복형 구현과는 별도다.

▼ 심화 자료로 넘어갈 기준

Red-Black 삽입의 각 경우, B-tree 분할·병합, Borůvka, 4NF·5NF, 기수별 상세 복잡도는 원본 상세를 보존했다. 먼저 '무엇을 해결하는가'와 앞 장의 기본 예제를 설명한 후 확장한다. 관심 연구실이나 공식 안내에서 해당 영역을 강조하면 우선순위를 올린다. 본 교재의 확장 분류가 실제 시험에서 제외된다는 뜻은 아니다.

28. 내 연구를 컴퓨터공학 질문으로 연결하기

연구 경험을 말할 때 정의·실험 조건·관측 결과·한계를 구분한다. 학업계획서를 작성하는 장은 아니며, 연구에서 전공 질문으로 이어지는 구술 연습이다.

현재 작업 공간은 로봇 형상·보행 학습 연구와 관련되어 있다. 아래는 그 맥락을 활용한 **연습 질문과 설명**들이다. 본인의 실제 사용 알고리즘이나 성과 수치를 대신 확정하지 않는다. ‘제가 구현했습니다’와 ‘기존 구현을 사용해 평가했습니다’도 본인의 역할에 맞게 구분한다.

연구 설명을 네 덩어리로 준비한다

①무슨 문제인가: 비교할 대상·목표. ②어떤 방법인가: 입력·출력·알고리즘·본인의 기여. ③어떻게 평가했나: 비교 기준·동일 조건·평가 지표. ④무엇을 아직 모르는가: 일반화 범위·실물 검증·추가 실험. 이 네 부분이 연결되어야 꼬리질문에서 설명이 흔들리지 않는다.

연습 틀: “제가 다루는 문제는 [문제]입니다. [입력]을 이용해 [출력]을 얻고, [방법]으로 비교했습니다. 성능은 [같은 평가 조건]에서 [측정 지표]로 확인했습니다. 다만 [아직 검증하지 못한 조건]은 남아 있어 [추가 검증]이 필요합니다.” 대괄호는 실제 기록으로 채우며 관측하지 않은 성과는 넣지 않는다.

강화학습을 말한다면 먼저 정의할 것

강화학습은 환경과 상호작용하며 얻는 보상에 따라 행동 전략을 개선하는 문제를 다룬다. 상태(state)는 환경의 상황, 관측(observation)은 에이전트가 받는 정보, 행동(action)은 선택하는 제어, 정책(policy)은 관측·상태 등에 따라 행동을 정하는 규칙이다. 즉시 보상과 미래 보상을 합친 누적 보상은 다르다. 할인율 γ 는 미래 보상을 현재 목표에서 얼마나 반영할지 조절한다.

지도학습은 보통 입력과 목표 정답의 쌍으로 예측을 학습한다. 강화학습에서는 행동이 이후 데이터와 보상에 영향을 주고 결과가 늦게 드러날 수 있다. ‘정답이 없으니 아무 기준도 없다’는 뜻은 아니다. 보상 설계가 곧 목적을 완전히 대변하는지도 검토해야 한다.

보상·실제 지표·일반화를 분리한다

높은 보상이 실제로 더 빠른 이동이나 더 안정적인 보행과 같은 뜻인지 확인해야 한다. 속도만 보면 넘어지기 직전의 짧은 돌진을 좋게 평가할 수 있어 평가 구간·낙상·추종 오차 등의 정의가 중요하다. 특정 실험에서

그렇게 나타났다고 주장하려면 본인의 기록이 필요하다.

여러 설계를 비교할 때 학습 예산·평가 시간·초기 조건·명령·환경 조건을 통제하지 않으면 다른 요인이 결과에 섞인다. 난수 시드(seed)를 여러 개 사용하고 평균뿐 아니라 변동도 확인하는 이유는 한 번의 우연을 일반 성능으로 오해하지 않기 위해서다. 같은 데이터를 반복 보며 설계를 골랐다면 별도 최종 평가 조건이 필요할 수 있다.

연구 이야기에서 이어질 질문	필요한 CS 기초	먼저 준비할 근거
많은 환경을 동시에 돌렸나요?	동시성·병렬성·프로세스/스레드	어떤 장치에서 어떤 연산을 병렬화했는지
데이터를 어떤 구조로 저장했나요?	배열·큐·해시·메모리	실제 데이터 모양과 접근 패턴
왜 이 알고리즘을 썼나요?	복잡도·목적함수·탐색 전략	비교한 대안과 선택 기준
모든 조건에서도 잘 되나요?	일반화·실험 설계	학습 조건과 별도 평가 조건의 차이
명령이 늦게 도착하면요?	통신·버퍼·시간 정보	지연·손실·오래된 명령 처리 방식

▼ 연습 · 학습 보상이 높아졌으면 연구 목표를 달성했다고 말해도 되나요?

보상과 실제 목표의 관계를 확인해야 한다. 학습에 사용한 보상 외에 목표와 직접 연결되는 평가 지표를 정의하고, 동일 조건의 비교와 별도 평가로 검증한다. 자신의 실험에서 확인한 것과 일반 가능성을 구분해 답한다.

▼ 꼬리질문 · 왜 단순 탐색 대신 학습 기반 방법을 사용했나요?

문제의 상태·행동 공간, 평가 비용, 모델 오차 등 실제 선택 이유를 설명한다. 'AI가 더 좋기 때문'은 근거가 아니다. 사용한 방법이 어떤 가정 아래 이점이 있고, 무작위 탐색·기존 제어·고정 정책 등 어떤 대안과 비교했는지 본인 기록으로 답한다.

▼ 꼬리질문 · 본인의 기여와 아직 검증하지 못한 것은 무엇인가요?

직접 설계·구현·분석한 부분을 구분해 한 문장씩 준비한다. 사용한 라이브러리나 기존 알고리즘 자체를 자신이 개발했다고 표현하지 않는다. 예를 들어 실물 전이를 아직 확인하지 않았다면 시뮬레이션 결과라고 범위를 명시한다.

이 강의 예시 문장을 그대로 본인 성과로 외우지 않는다. 본인 연구에 없는 알고리즘·실험·수치는 말하지 않고, 실제 기록으로 검증 가능한 설명만 준비한다.

29. 과목을 섞어 말하는 모의면접

익숙한 순서 없이 질문을 받아도 핵심을 먼저 답하고, 근거·예시·한계를 이어 말한다. 아래는 제작한 연습 세트이며 한양대 실제 면접 문항·시간표가 아니다.

30초 답변과 90초 설명을 구분한다

30초 답변은 정의 한 문장→핵심 동작→중요 조건 정도로 시작한다. 더 물으면 예제를 그리거나 대안을 비교한다. 모든 질문에 외운 네 문장을 기계적으로 말할 필요는 없다. 비교 질문이면 차이를 먼저, 실행 질문이면 상태 변화를 먼저 보여준다.

질문: “해시와 BST를 비교해 보세요.”

첫 답변: “해시는 키로 위치를 계산해 평균적으로 빠른 일치 조회에, 균형 BST는 키 순서를 유지하는 검색에 적합합니다. 해시는 충돌과 부하율을 관리해야 하고, BST는 높이를 관리해야 합니다. 범위 검색이 필요하면 트리를 고려하겠습니다.”

고리질문 “항상 $O(1)$ 인가요?” → “아닙니다. 적절한 분산과 부하율을 가정한 평균 비용이며 충돌이 물리면 최악 $O(n)$ 입니다. 긴 문자열이면 해시 계산의 키 길이 비용도 구분합니다.”

세트 A · 기초를 연결하는 15분

1. 배열과 연결 리스트의 차이, 검색 후 삽입 비용까지 설명. → 14. 배열·연결 리스트·스택·큐
2. 병합 정렬의 $n \log n$ 을 층별 작업으로 설명. → 17. 정렬: 과정에서 복잡도를 도출하기
3. 프로세스와 스레드의 공유·분리 범위. → 20. 프로그램이 실행되면: 프로세스·스레드
4. TCP/UDP를 선택할 기준과 보장하지 않는 것. → 23. 네트워크: 주소를 입력한 뒤 생기는 일
5. 기본키와 외래키를 학생·수강 표로 설명. → 25. 데이터베이스: 표를 나누는 이유부터

세트 B · 종이에 그리는 20분

1. BST에 4,2,6,1,3 삽입 후 중위 순회, 2 삭제. → 기대 순회 1,2,3,4,6; 2는 3으로 대체하고 원래 3 노드 제거 가능.
2. [3,1,2] 삽입 정렬을 단계별로 실행. → [1,3,2]→[1,2,3].

3. $A-B=5, A-C=1, C-B=1$ 에서 BFS와 최소 비용의 차이. → 직접 1간선/비용5, 경유 2간선/비용2.
4. 페이지 크기 1,024, 주소 2,100, 프레임 4로 변환. → 페이지2, offset52, 물리주소4,148.
5. 학생 정보와 수강 점수가 반복되는 표에서 갱신 이상을 찾고 분해. → 25장의 종속성과 가정을 먼저 적는다.

세트 C · 반례와 연구 연결 20분

1. ‘힙은 정렬된 트리다’에 반론. → 부모·자식 조건만, 전체 검색 순서 보장 없음.
2. ‘TLB 미스는 디스크 읽기다’에 반론. → 변환 캐시 미스일 뿐 페이지가 RAM에 있을 수 있음.
3. ‘상속하면 무조건 유지보수가 쉬워진다’에 반론. → 관계·결합도·합성 대안 고려.
4. ‘보상이 증가했으니 실제 목표가 개선됐다’에 필요한 검증. → 지표·평가 조건·반복 실험.
5. 현재 연구에서 사용한 CS 개념 두 개와 본인 기여를 설명. → 실제 코드·기록으로 확인한 내용만.

막혔을 때 사용할 문장

“정의와 기본 동작부터 말씀드리겠습니다.” / “배열 기반 구현을 가정하면 …입니다.” / “정확한 최악 복잡도는 지금 기억나지 않지만, 이 구현에서는 …번 반복합니다.” / “그 구현 세부는 확인하지 못했습니다. 제가 검증한 범위는 …입니다.”

모르는 것을 아는 척하는 대신 확실히 아는 범위를 먼저 밝힌다. 단, 외운 회피 문장만 반복하지 말고 질문과 관련된 실제 근거를 설명한다.

자가 평가표

항목	0	1	2
정의	다른 개념과 혼동	핵심 일부만	핵심과 구분이 정확
동작·예시	예시 없음/틀림	도움을 받아 추적	새 숫자도 직접 추적
조건·한계	무조건 단정	조건 일부 언급	적용 조건·반례 설명
전달	질문과 무관	핵심이 늦게 나옴	핵심 먼저, 요청에 맞게 확장

8점 척도는 복습 우선순위를 정하기 위한 자체 기준이며 학교 채점표가 아니다. 두 번 연속 6점 이상이어도 틀린 정의가 있으면 먼저 고친다. 하루 뒤 같은 질문에 다시 답하지 못하면 ‘설명 가능’ 체크를 되돌린다.

▼ 연습 · “이진 탐색과 BST는 같은 건가요?”에 30초로 답해 보세요.

이진 탐색은 정렬된 배열 등의 후보 범위를 절반씩 줄이는 알고리즘이고, BST는 왼쪽·오른쪽 서브트리의 키 순서 규칙을 가진 자료구조다. 둘 다 비교로 방향을 좁히지만 BST의 높이는 삽입 순서·균형 유지에 따라 달라져 항상 로그가 아니다.

▼ 꼬리질문 · “전체 내용을 공부했나요?”라는 질문에 어떻게 답할까요?

읽은 범위와 설명·구현·검증할 수 있는 범위를 구분한다. 예: ‘기초 자료구조와 운영체제는 예제를 직접 추적하며 학습했고, B-tree 삭제 같은 세부 구현은 추가 학습 중입니다.’ 실제 본인의 준비 상태에 맞게 바꾼다.

원본 범위를 잃지 않는 대응표

새 본문은 C 실행→문법→메모리→자료구조→알고리즘→시스템 순서로 재배치했다. 지역·전역·비트·증감·포인터·구조체는 앞의 C 기초 장에서 보충했다. 원본의 C++ 기능은 C 문법과 구분한 비교 장으로 보존했다. 기존 해설 18개 번호 장·준비 장과 원본 이미지 16쪽은 아래 접힌 부록에 보존했다. 새 본문에서 요약한 변형·세부 규칙은 ‘기존 상세’에서 이어 읽는다.

원 본 쪽	주요 범위	새 교재 / 기존 상세
1	리스트·스택·큐·트리·순회·정렬 입문	14. 배열·연결 리스트·스택·큐, 16. 트리·BST·힙·균형 트리, 17. 정렬: 과정에서 복잡도를 도출하기 / 기존 01
2	스택·힙 메모리·캐시·교체 정책	21. 가상 메모리·페이지 폴트·캐시 / 기존 02
3	비교 정렬·안정성·구현별 조건	17. 정렬: 과정에서 복잡도를 도출하기 / 기존 03
4	셀·계수·MSD/LSD 기수·전체 표	17. 정렬: 과정에서 복잡도를 도출하기, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 04
5	순차·이진·보간 탐색·BST·해시	15. 재귀·이진 탐색·분할정복, 16. 트리·BST·힙·균형 트리, 18. 해시: 빨리 찾기 위해 치르는 비용, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 05
6	해시 함수 종류·보안·암호·서명	18. 해시: 빨리 찾기 위해 치르는 비용, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 06
7	Trie·B/B+·m-way·AVL·Red-Black	16. 트리·BST·힙·균형 트리, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 07, 규칙 상세
8	탐욕·MST·Prim	19. 그래프·BFS·DFS·최단경로·MST / 기존 08
9	그래프 표현·DFS/BFS·Kru	19. 그래프·BFS·DFS·최단경로·MST / 기존 09, 전체 비교

	skal/Prim/Bo růvka	
10	프로세스· 스레드·가상 메모리·폴트· 스래싱	20. 프로그램이 실행되면: 프로세스·스레드, 21. 가상 메모리·페이지 폴트·캐시 / 기존 10
11	요구 페이징 ·CPU/HDD 스케줄링· 동기화·분산	22. 스케줄링·경쟁 상태·교착상태, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 11
12	네트워크 ·ARP· 인터럽트 ·daisy chain	23. 네트워크: 주소를 입력한 뒤 생기는 일, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 12
13	바인딩·가상 함수·vtable	24. 객체지향을 배운 뒤 가상 함수를 읽자 / 기존 13
14	함수·먹집합· 증명·관계	26. 이산수학: 기호를 문장으로 읽기 / 기존 14
15	파일·스키마· 객체지향	24. 객체지향을 배운 뒤 가상 함수를 읽자, 25. 데이터베이스: 표를 나누는 이유부터, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 15
16	DB·정규형· 잠금·인자 전달·Big O·fork·STL	13. 복잡도와 로그를 외우지 않고 이해하기, 20. 프로그램이 실행되면: 프로세스·스레드, 24. 객체지향을 배운 뒤 가상 함수를 읽자, 25. 데이터베이스: 표를 나누는 이유부터, 27. 남은 원본 범위와 자주 혼동하는 연결 / 기존 16, 4NF·5NF·STL 포함

찾아보기: 이름이 비슷한 개념

혼동하기 쉬운 말	구분 / 복습
값 / 주소 / 인덱스	내용 / 메모리 위치 / 배열 순번 · 01. 프로그램과 C의 첫 실행
평균 / 최악 / 상각	입력 분포 / 가장 비싼 입력 / 연산열 총비용 배분 · 13. 복잡도와 로그를 외우지 않고 이해하기
이진트리 / BST / 힙	자식 수 / 키의 검색 순서 / 부모·자식 우선순위 · 16. 트리·BST·힙·균형 트리
안정 정렬 / 안정적 프로그램	동일 키 순서 보존 / 일반적 신뢰성 표현 · 17. 정렬: 과정에서 복잡도를 도출하기
MST / 최단경로	전체 연결 합 / 출발점에서 목적지 비용 · 19. 그래프·BFS·DFS·최단경로·MST
동시성 / 병렬성	겹친 기간에 진척 / 같은 시각 실행 · 20. 프로그램이 실행되면: 프로세스·스레드
메모리 힙 / 자료구조 힙	동적 할당 공간 / 우선순위 구조 · 21. 가상 메모리·페이지 폴트·캐시
캐시 미스 / TLB 미스 / 페이지 폴트	데이터 / 주소 변환 정보 / 예외 처리 · 21. 가상 메모리·페이지 폴트·캐시
교착 / 기아	서로 기다려 멈춤 / 특정 작업이 계속 밀림 · 22. 스케줄링·경쟁 상태·교착상태
IP / MAC / 포트	네트워크 간 주소 / 링크 주소 / 전송 끝점 구분 · 23. 네트워크: 주소를 입력한 뒤 생기는 일
오버로딩 / 오버라이딩	매개변수별 같은 이름 / 상속 가상 함수 재정의 · 24. 객체지향을 배운 뒤 가상 함수를 읽자
원자성 / 격리성	작업 전체 반영·취소 / 동시 작업 간섭 · 25. 데이터베이스: 표를 나누는 이유부터

공역 / 치역	출력 대상으로 정한 집합 / 실제 출력 집합 · 26. 이산수학: 기호를 문장으로 읽기
암호화 / 해시 / 서명	내용 보호 / 요약 / 출처·변경 검증 · 27. 남은 원본 범위와 자주 혼동하는 연결

자료의 근거와 검토 범위

2026-09-16 C 개정판. 첫 12개 C 기초 장을 선수지식 순서로 새로 작성하고, 자료구조·알고리즘의 코드 예제를 C로 바꾸었다. 기초 자료는 사용자가 제공한 「대학원면접_기초해설서.html」이다. 해당 파일 안의 기존 해설·표·원본 16쪽 이미지를 보존하고 새 학습 본문을 작성했다. 이번 C 개정에서도 기존 HTML에 포함된 16쪽 이미지를 원본 대조 기준으로 보존했다. Downloads의 PDF 파일은 별도 전수 판독하지 않았다. 손글씨의 판독 유보 항목은 기존 정정·판독 안내에 남아 있다.

개념 설명·계산 예제·면접 연습 문제는 이 교재를 위해 구성했다. 아래는 정의 확인과 추가 학습에 사용한 공식 문서·저자 공개 교재다. 모든 문장을 이 링크로 전수 검증했다는 뜻은 아니며, 학교의 실제 출제 빈도나 합격 가능성의 근거로 사용하지 않았다. 확인일: 2026-09-12.

- 한양대학교 컴퓨터소프트웨어학과 공식 홈페이지 — 학과와 공식 공지 경로 확인. 이번 세부 구술 범위·배점·기출은 미확인.
- Pat Morin, Open Data Structures — 배열·리스트·해시·트리·정렬·그래프 추가 학습. B-tree 장에서 블록 기반 모델 확인.
- Arpaci-Dusseau & Arpaci-Dusseau, Operating Systems: Three Easy Pieces — 프로세스·가상화·동시성·영속성 후속 읽기.
- RFC 9293 TCP, RFC 768 UDP, RFC 826 ARP — 통신 기능과 주소 해석 확인.
- PostgreSQL JOIN, Transactions — 조인·트랜잭션의 기본 의미 확인.

파일 사용 안내

인터넷 없이 본문·원본 이미지·연습을 볼 수 있다. 외부 출처 링크를 여는 경우만 인터넷이 필요하다. 각 장의 메모와 학습 상태는 현재 브라우저에만 저장될 수 있으므로 ‘학습 기록 백업’을 주기적으로 사용한다. 파일을 이동하거나 브라우저를 바꾸면 저장 기록이 따라오지 않을 수 있다.

‘교재 인쇄 / PDF’는 새 교재와 해설을 펼쳐 인쇄하고 이미지가 많은 기존 부록은 제외한다. ‘기존 자료 인쇄’는 기존 본문·원본 이미지를 인쇄한다. 브라우저 인쇄 창에서 PDF 저장을 선택할 수 있다. JavaScript가 꺼져 있어도 본문과 접힌 해설은 읽을 수 있지만 검색·기록·무작위 연습 기능은 작동하지 않는다.

무작위 구술 연습실

모든 새 교재의 연습·꼬리질문에서 뽑습니다. 해설을 보기 전, 정의→동작 예→조건·한계 순서로 말하세요.
자동 채점이나 합격 예측 기능은 없습니다.

질문 뽑기를 눌러 시작하세요.

▼ 해설·자기 점검 보기

자가 평가: 0점 = 설명 못함 / 1점 = 용어만 기억 / 2점 = 예제로 설명 / 3점 = 조건·한계와 꼬리질문까지 설명.
같은 질문을 다음 날과 3일 뒤에도 다시 답하세요.

- 알고리즘 문제해결 전략
1. Linked List 과목과 과목 연결
- 노드 : 실제 정보를 담고 있는 데이터 / 링크 : 노드간의 연결을 나타내는 포인터
 - 동적인 자료구조(필요할 때 할당, 미리 크기 지정할 필요X) / 순차적 접근 가능한 자료구조(임의 접근 불가능) / 불연속 메모리 사용(물리적 순서 없고 불연속적으로 저장, 링크에 의해 순서 유지)
 - 단일 연결 리스트(단방향 : data, next pointer) / 이중 연결 리스트(양방향 : data, next pointer, prev pointer)

2. Stack 용량 증가
- LIFO(Last In Last Out) / 입구와 출구가 하나인 자료구조
 - push : 데이터 추가 / pop : 가장 최근에 추가한 데이터 뺌
 - 배열로 구현 : 스택 크기가 미리 정해져 있음, 들어가는 데이터 개수의 변동 폭이 적을 때(즉, 크기 가늠 가능할 때), 적절할당
 - 리스트로 구현 : 연결리스트의 특성 상 스택 크기가 자유로움, 들어가는 데이터 개수 가늠 어려울 때 (성능 면에서 배열 보다 우수- 훨씬 간단함), 동적할당, 단일 연결리스트로 구현

3. Queue 용량 증가
- FIFO(First In First Out) / 입구와 출구가 따로 있는 자료구조 / front(출구) rear(입구)
 - put : 데이터 추가 / get : 가장 나중에 추가된 데이터 뺌
 - 배열 / 리스트로 구현 시 스택과 동일 / 차이점 : 이중 연결리스트로 구현

4. Tree 사이클이 없다
- 노드(vertex/점)와 링크(edge/간선)로 구성된 비선형 자료구조 / ex. 윈도우 디렉터리구조(가장 전형적인 트리구조)
 - 특징 : root (최 상위 node) 제외한 모든 node는 하나의 부모를 가짐 / node는 여러 개의 자식 가질 수 있음 / 한 node로부터 다른 node로 가는 경로(Path)는 유일함
 - Leaf node : 자식 없는 최하위 노드(=terminal, external) / Internal node : leaf node 아닌 노드(non-terminal)
 - subtree : 트리 부분집합 / Level : root에서 특정노드로 가는 경로의 노드 수 / Height : 가장 높은 레벨

5. Binary Tree (노드 = 2 이하)
- 모든 Internal node가 두 개 이하의 자식을 갖는 트리(최대 두 개 : left child, right child)
 - 가장 쓰임새가 많은 트리 (Parse Tree(수식계산) / Heap(정렬에 사용) / Binary Search Tree(검색에 사용))

Full Binary Tree : 마지막 레벨을 제외한 모든 레벨에 노드가 꽉 차있음(자식이 모두 둘)
 $2^{d-1} \leq N \leq 2^d - 1$ 이면(log2 취해서 증명) $d = \lceil \log_2 N \rceil + 1$ (레벨: d, 노드: n개)

Complete Binary Tree : 모든 레벨에 노드가 꽉 차있음

6. Tree Traversal

- 트리가 비선형구조이기 때문에 전체 노드를 방문하는 방법이 필요(트리의 모든 노드 방문) - 방법에 차이가 있음
- stack 기반 : pre-order / post-order / in-order queue 기반 : level-order
- pre-order traversal : root -> left -> right 123
- in-order traversal : left -> root -> right 213
- post-order traversal : left -> right -> root 312
- Parse Tree: 수식을 연산순서에 따라 트리로 구성. root(operator: 모두 non-terminal) / child(operand: 모두 leaf)

7. 정렬 알고리즘

- 데이터 집합을 정해진 순서대로 재배열 하는 것. 오름차순(ascending order) / 내림차순(descending order)
- 키 값(비교 대상, 정렬 기준) / 레코드(교환 대상, 하나의 정보 단위(여러 개 필드가 합쳐진 것))
- 장소에 따라 : 내부정렬(컴퓨터 메모리 내에서 작동): 정렬할 개수가 적어서 주기억장치에 모두 적재시켜 놓고 주기억장치 내에서 정렬하는 방법 / 특정 인덱스로 바로 읽어오기 가능
- 외부정렬(테이프(순차적 접근), 하드디스크): 자료개수를 주기억장치가 수용하지 못해서 보조기억장치에 저장해두고 일부만 차례로 적재해서 사용하는 방법

* 메모리 영역, ← 가상메모리 영역 → 프로그램 동작시 메모리에 프로그램이 동작하기 위한 가상메모리 공간 생성
 목적에 따라 이 영역은 상위 / 하위 메모리를 나뉨

① Stack : 임시변수를 저장하는 특정영역 / 순서있다 (LIFO)

(하위영역) 함수에서 새로운 변수 선언시 stack이 저장, 함수의 종료시 stack이 null로 자동으로 메모리 해제함.

② Heap : stack과 달리 CPU에서 자동으로 관리해주지 않는 영역 / 프로그래머가 직접 영역 할당

(하위영역) 메모리 주소값만으로 참조 / malloc(), calloc() 등으로 메모리 할당 하고 직접 해제 → 안하면 메모리 누수 발생 (memory leak)

장점: 상황에 따라 원하는 크기만큼 메모리 할당 가능. & 메모리 크기 조절 가능.

CF. 프로그램에서
 힙 영역과 스택
 영역의 차이가
 중요하게 작용
 되고
 이걸 프로그램이
 명시적으로 해
 리스
 해당 프로그램에서
 스택과 힙의
 메모리 주소는
 다른 메모리 해

CF.

메모리에 고려해야 할 문제.

① 속도적인 문제 → 캐시 메모리 등장.

② 용량적인 문제 (메모리 부족 문제 해결 방법)

강제적인 속도적 문제.
 ex. CPU < 메모리 < 하드디스크
 1 150
 (하드디스크가 가장 느린
 메모리는 4배 정도 빠름)

* 캐시 메모리 (cache memory) ← SRAM으로 많음

= 메인 메모리에 있는 명령어와 데이터 중 일부를 가져와 임시적으로 저장하여 CPU에서 필요할 때 해당 정보의 신속한 제공이 가능도록 만들어진 장치.

- CPU와 메인메모리 사이에 데이터 처리 속도 차이가 크기 때문에 CPU 활용 제어로 온다. ∴ CPU 캐시 캐시 메인 배를 처리

- 교체 알고리즘 → 작동: CPU가 현재 필요한 정보를 캐시 메모리에서 찾을 때 있을 경우.

캐시
 메모리
 데이터
 데이터 속도 차이 극복
 데이터

캐시 메모리에서 공간 마련을 위해 데이터 제거, 어떤 데이터 제거할지 결정하는 것.

→ LRU, LFU, FIFO 등 존재. → 데이터 교체 알고리즘 (캐시에 정보 없으면 주 메모리에서 찾아야 함)

Least

- 교환 방식에 따라 : 직접정렬(레코드 전체 이동하여 정렬) / 간접정렬(별도로 존재하는 키 값의 인덱스만을 정렬)
- Stability : 같은 키 값을 가지는 여러 레코드가 정렬 후에도 그 상대적 순서를 유지하는지 여부

1. Selection Sort 최소

- 최소값 찾아 앞으로 이동시킴 / 정렬되지 않은 부분에서 최소값을 찾아 정렬된 부분 뒤에 더함(앞부분부터 완전정렬)
- complexity : $O(n^2)$ / Worst case = Best case = $O(n^2)$ ($n*(n-1)/2$)
- 입력 자료에 둔감 : 교환 횟수가 적으므로(메모리 복사가 적으므로) 큰 레코드에 적합

2. Insertion Sort

- 현재 레코드를 일부 정렬된 부분의 적절한 위치로 옮김 / 현재 수와 바로 앞의 수 비교(앞부분부터 상대적 정렬)
- complexity : $O(n^2)$ / Worst case = $O(n^2)$ (reverse) / Best case = $O(n)$ ((n-1)->sorted)
- 입력 자료에 민감 : 거의 정렬된 문제에서는 빠름

3. Bubble Sort 최대

- 인접 요소의 비교/교환을 통해 최대값을 제일 뒤로 보냄(뒷부분부터 완전정렬 되는 모양)
- complexity : $O(n^2)$ / Worst case = Best case = $O(n^2)$
- 비효율적 : 교환이 3번의 operation으로 구성(swap) / 잘 사용하지X / 성능 좋지않음 / 느림 / 시간 오래걸림

4. Quick Sort

- 분할(partition) 알고리즘: 축값(pivot value)을 중심으로 왼쪽은 축값보다 작은 값, 오른쪽은 축값보다 큰 값으로 배열
- 크기가 1보다 클 동안(1 될 때까지) partition을 재귀적으로 반복(재귀적으로 분할되어 가는 과정) / 멀리 떨어져 있는 요소를 교환하기 때문에 효율적임
- Worst case = $O(n^2)$ (분할이 N-1 : 1로 이루어질 때 (순차배열, 역순배열)=Degenerated Case)
=> 문제 : (Recursive Quick Sort일 때) Stack Overflow Error 발생 메모리 사용량 증가 → 넘어서 → 에러 발생
해결법 : Non-recursive로 사용하여 stack overflow 방지 / 개선방법 3가지 사용
- 개선법 1. 난수분할: pivot value를 임의로(random하게) 정함 <<1,2,3 모두 순차/역순(최악)도 $O(N \cdot \log N)$ 나옴>>
- 2. Median-of-Three Partition: 처음, 가운데, 끝 값 중 중위값을 찾아 그것을 pivot value로 하는 방법
- 3. Subfiles(2번 + Insertion Sort): 퀵정렬은 큰 데이터 집합에 유리한데 이때 작은 데이터 집합에는 (↑최적방법) 단순한 삽입정렬이 유리하므로 파티션을 해가며 작은 구간이 되면 삽입정렬로 대체하여 재귀의 깊이 줄임(스택사용 줄임=메모리 사용 줄임)
/ Best case = $O(N \cdot \log N)$ (분할이 정확히 절반씩 나누어질 때) ($T(N) = 2 * T(N/2) + N$) 재귀 호출 시 Stack 사용 줄임
- 결국 분할을 절반에 가깝게 하는 것이 성능의 관건

5. Heap Sort

- 힙: 키 값의 크기가 우선순위가 됨 / ~~Binary Tree~~ complete Binary Tree로 구성 / 부모노드 > 자식노드 / root에 가장 큰값
- 힙을 생성하여 힙에 자료가 하나도 남지 않을 때까지 extract함(역순저장 = 뒷쪽부터 최대값 저장)
- complexity : $O(N \cdot \log N)$ => 삽입, 선택정렬 보단 성능 좋음 / 퀵정렬에 비해서는 느림 / 입력자료에 둔감
- 힙정렬은 부가메모리를 전혀 사용하지 않음(힙부분과 입력배열, 결과배열의 경계구분이 명확하므로) space complexity = 0
- 하향식(top-down)힙 생성(N개 자료insert) < 상향식(bottom-up)힙 생성(N/2개 자료downheap) 더 효율적(빠름)

6. Merge Sort

- 두 개 이상의 정렬된 자료집합을 하나의 정렬된 자료집합으로 합치는 것
- 대상 자료집합이 정렬된 상태이므로 순차적으로 비교하며 작은자료를 뽑음(병합 대상이 되는 자료와 순차적 비교)
- complexity(time) : $O(N \cdot \log N)$ => 크기M과 크기N의 단위를 병합하는데 M+N비교 필요 $T(N)=2T(N/2)+N$ (퀵정렬)
- complexity(space) : $O(N)$ => 병합 결과를 저장하기 위한 N개의 공간이 필요(비효율적) / 연결리스트는 $O(1)$ 이유: 이전노드만 참조하면 됨, 다음노드는 다음노드 주소에 저장되어 있음
- 입력 자료에 둔감(역순/순차 정렬 등과 무관)
- 순차적 접근만 사용하기 때문에 (즉, Random Access X) 순차적 접근만 가능한 장치(tape같은)의 외부정렬에 사용
- Random Access의 성능이 떨어지는 경우 효과적(Sequential Access 이므로)
- Stability 있음 (값이 같더라도 정렬 후 상대적 위치를 유지하는 것)

7. Shell Sort

- 삽입정렬의 문제점(비교/교환 횟수 많음 $O(N^2)$)을 개선한 것
/ 어떤 수열을 선택할 것인가가 성능의 관건 ($h(N) = 3h \cdot (N-1) + 1$)
- complexity : $O(N(\log N)^2)$ or $O(N^{1.25})$ ($O(N \cdot \log N)$ 보다는 크나 단순한 알고리즘이므로 속도 빠름, 입력자료 둔감)
- space complexity = 0 => 부가 메모리를 사용하지 않음(입력배열 그 자체 제외하고 다른 부가메모리 사용X)

8. Distribution Counting

- 같은 키가 많이 중복되는 정수 자료집합에 적용 가능한 매우 단순한 알고리즘
- 특정 키가 출현하는 빈도와 누적분포 이용
- complexity(time) : $O(N)$ => 단일루프로만 이루어짐(성능 우수하나 메모리 공간 사용 많이함)
- complexity(space) : $N+M$ => 결과를 저장할 배열 크기 N + 키 값의 분포를 저장할 count배열 크기 M
- 중복된 키가 많고 키의 범위가 비교적 작은 경우 유리(이때 메모리 낭비X. but 크면 낭비多)
- 양의 정수 자료만 정렬 가능 => 정수 값이 그대로 인덱스 요소가 되기 때문

9. Radix Exchange Sort

- Quick Sort와 유사한 partition의 재귀적 반복 / partition의 기준은 비트(0 or 1) / 양의 정수만 정렬 가능
/ MSB(Most Significant Bit: 가장 자릿수 큰값) -> LSB(가장 자릿수 작은값) ex) 010101을 000111로 만드는 것
- 최상위 비트부터 bit partition 실행해서 분할된 영역에 대해 재귀호출
- 퀵정렬과 유사 / 차이점 : 분할의 축값(pivot value)이 없음
- complexity(time) : $O(N \cdot \log N)$ / complexity(space) (스택 사용량) : $O(W)$ $3W+3$ (W =자료의 비트 수)

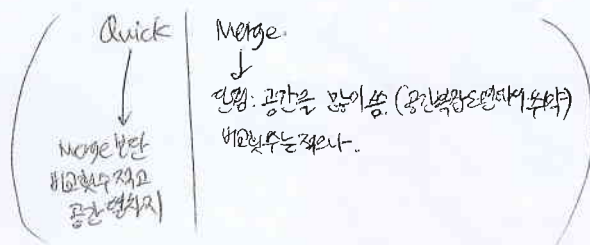
10. Straight Radix Sort

- 기수교환정렬과 반대로 최하위자리(LSB)부터 최상위자리(MSB)로 올라가면서 Stability 있게 정렬 (LSB->MSB)
- 기수교환정렬과 마찬가지로 양의 정수에서만 가능
- 메모리카운트(비트 수)를 얼마씩 잡느냐에 따라 분포수세기 실시횟수 달라짐(메모리 늘리느냐 or 속도 빠르게하냐)
- complexity(time) : $O(N)$ => $(W/M) \cdot \text{분포수세기 시간}$ -> 선형 정렬 알고리즘(Linear Sorting Algorithm)
- complexity(space) : $N + 2^M$ => 분포 결과 저장 배열 크기 N + 분포 저장할 count배열 크기 2^M
- M 크게 하면(분포수 세기 횟수 적어지면) 성능 좋아짐. but 메모리 차지 많아짐

* Radix(기수)를 기반으로 하는 정렬은 입력배열에 둔감함
random(난수)배열, sorted배열, reverse배열 차이 없음 => 입력배열에 상관없이 비트만 보고 따지기 때문

[정렬 알고리즘 정리]

Algorithm	time complexity	space complexity	worst case	best case	stability
Selection Sort	$O(N^2)$	0	$O(N^2)$	$O(N^2)$	Y
Insertion Sort	$O(N^2)$	0	$O(N^2)$	$O(N)$	Y
Bubble Sort	$O(N^2)$	0	$O(N^2)$	$O(N^2)$	Y
Shell Sort 퀵 개선	$O(N(\log N)^2)$ or $O(N^{1.25})$	0			N
Quick Sort	$O(N \cdot \log N)$	$2N+2$	$O(N^2)$	$O(N \cdot \log N)$	N
Heap Sort	$O(N \cdot \log N)$ 개선	0			N
Merge Sort 외부정렬 극히 중요	$O(N \cdot \log N)$	X			Y
Radix Exchange Sort 외부정렬	$O(N \cdot \log N)$ 기수교환 MSB->LSB	$3W+3$			N
Distribution Counting	$O(N)$	$N+M$			Y
Straight Radix Sort	$O(N)$ 적용기법 LSB->MSB	$N+2^M$			N < Y



8. 검색 알고리즘

- 검색 : 자료집합에서 원하는 데이터를 찾아내는 것 / 세가지 요소 : 검색, 삽입, 삭제
- 자료의 조직화 : 재조직 비용이 듦. 삽입/삭제 비용 $\uparrow$ but 검색 속도 $\uparrow$ $\rightarrow$ 자료를 조직화 하는 것이 좋음
- 자료집합의 조직화여부에 따라 성능이 달라짐 / 조직화 방법에 따라 알고리즘이 달라짐

1. Sequential Search worst $O(N)$

- 무순서로 저장된 자료집합에서 검색하고자 하는 키를 처음부터 차례로 비교해서 찾아내는 방법
- 배열로 구현 $\Rightarrow$ complexity : 삽입= $O(1)$ / 검색= $O(N)$ (worst) / 제거= $O(N)$
- 리스트로 구현 $\Rightarrow$ complexity : 삽입= $O(1)$ / 검색= $O(N)$ / 제거= $O(1)$ $\rightarrow$ 배열로 구현할 때 보다 효율적

2. Binary Search

- 정렬된 자료집합을 가지고 구간크기 > 0 일 때, 키 값과 중앙값을 비교 / 같: 찾음 / 키 > 중앙 : 오른쪽 / 키 < 중앙 : 왼쪽
- complexity : 삽입 = $O(N)$ (삽입정렬의 한 스텝과 동일한 방식 / worst = 최소값 삽입할 때)
- / 검색 = $O(\log N)$ (구간 이등분하여 줄여나감 / worst = 자료에 없는 값 찾을 때)
- / 제거 = $O(N)$ (삭제시간 = 검색시간($O(\log N)$) + 자료가동시간($O(N)$))

3. Interpolation Search

- 자료집합이 정렬된 선형분포를 가진다(linear하다)고 가정하고 내분법(Interpolation)에 의해 mid 값을 찾음
- 나머지 과정은 이분검색과 동일 / 산술연산 가능한 숫자 키만 가능(정수, 실수만 가능)
- complexity : 삽입 = $O(N)$ (이분검색과 동일)
- / 검색 = $O(\log(\log N))$ (linear만 성능 $\uparrow$ / 실수연산 성능 $\downarrow$ / 환경에 따라 다름)
- / 제거 = $O(N)$ (이분검색과 동일)

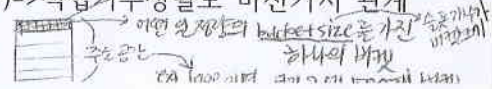
9. Binary Search Tree (BST)

- 이진트리를 이용하는 검색방법 / 부모노드는 왼쪽 subtree의 모든 노드보다 큼 / 부모노드는 오른쪽 subtree의 모든 노드보다 작음 (즉, 부모기준 좌:小 우:大)
- 여기서도 In-order / Pre-order / Post-order Traversal $\Rightarrow$ In-order Traversal했을 때 정렬된 순서
- 방법 : 검색할 키 값을 가지고 root부터 크기비교를 하면서 leaf node까지 탐색 / 같으면 찾음. 없으면 자료존재X
- 트리의 높이는 이진트리 성능에 중요한 요소 $\Rightarrow$ 좌우 균형이 맞을 때 가장 성능 좋음(+트리 높이 낮음)
- 자료의 입력순서에 의해 이진트리 모양이 결정 $\Rightarrow$ 순차/역순 일 때 트리의 불균형 심해져 순차검색과 동일한 성능
- complexity : 삽입(항상 leaf에 삽입) = 검색 = 삭제(leaf면 간단, but 내부노드면 대체노드 선택) = $O(\log N)$ (평균)
- complexity : 삽입 $\Rightarrow$ worst = $O(N)$ (한 줄로 늘어설 때) best = $O(\log N)$ $\Rightarrow$ 삽입 시 트리의 높이만큼 비교
- / 검색 $\Rightarrow$ worst = $O(N)$ (한 줄로 늘어설 때) best = $O(\log N)$ $\Rightarrow$ 검색 시 트리의 높이만큼 비교
- / 삭제 $\Rightarrow$ $O(\log N)$ $\Rightarrow$ 삭제될 노드를 대체할 후보를 찾는 데 드는 비용(오히려 불균형한 경우 찾기 쉬움)
- 결국 트리의 균형을 맞추는 것이 중요(성능이 트리의 균형에 좌우됨) $\rightarrow$ 때문에 Balanced Tree가 연구
- * 이진트리 정렬 : In-order Traversal하면 정렬된 순서가 나옴을 이용하여 정렬
- complexity : best case= $O(N \log N)$ (자료 입력 순서가 균형이 맞춰진 경우) / worst case= $O(N^2)$ (정렬/역순)

10. Hash

- hash function: 키 값을 수치(양의 정수)로 변환하는 함수 (클러스터가 적고 자료가 퍼진 것=가장 잘 만든 해쉬함수)
- hash table: 자료를 해쉬 함수의 결과 위치에 배치하는 방법(ex. 주소와 우편번호, 우체국 우편물 분류)
- best= $O(1)$ 의 성능 가지는 효율적인 검색구조(자료 개수에 상관없이 일정) / worst= $O(N)$
- $\Rightarrow$ 찾고자 하는 자료는 해쉬함수 연산으로 한번에 찾기 가능 / 해쉬함수의 선택과 해쉬테이블 크기가 중요한 요소
- collision: 서로 다른 키의 해쉬연산 결과가 같을 때 하나의 해쉬테이블에 들어가면서 충돌이 일어남 $\Rightarrow$ 선형탐사법(linear probing=open addressing) or 연결법(separate chaining)으로 해결
- 선형 탐사법: 충돌발생시 순차적으로 탐사하여 빈 곳을 찾음 / 입력자료 $\uparrow \rightarrow$ 연속된 cluster $\uparrow$, 비효율적 (hash map)
- 무조건 자료수 < 해쉬테이블 / 예상 입력 자료수의 2배 이상의 해쉬테이블 필요 / 정적인 자료 검색에 유리 (cluster=동일데이터 묶음)
- 연결법: 충돌발생시 해당 위치의 리스트에 추가 (해쉬테이블이 연결리스트의 배열로 이루어짐) (chain map)
- 자료수 > 해쉬테이블 상관X / 입력자료 수보다 적은 해쉬테이블 가능 / 동적인 상황에 적합 / 범용 사용 가능
- memory/performance trade-off: 메모리 높게 잡으면 성능 $\uparrow$ (클러스터 적어지니까) $\Rightarrow$ 직접가수정렬도 마찬가지 관계

버킷(Bucket): 해쉬함수로 계산된 값에 데이터를 저장하기 위한 기법(동적인 해쉬함수로 계산된 값에 따라 버킷이 변함)



* 해시 함수: 해시 함수 종류와 테이블 크기가 중요하므로 각함, 목적점: 많은 공간 활용, 양비 / 충돌

양도가 적고, 빠른 계산은 한수되어야 하며, 테이블 내 데이터(주소)가 고르게 분포해야 좋은 해시함수.
(크기성 ↓) 가장 중요한 것은 해시함수에서 필요한 것!!!

① 제한 함수: 가장 일반적인 함수.

정수인 키를 버킷 크기로 나눈 나머지를 해시주소로 사용.

$h(Key) = Key \% M$ (이때 M 값은 '소수'인 것이 좋다. 배열 내 고른 부분을 위해)

② 표정 함수: 탐색 키 전체를 해시주소로 함.

주소를 표에서 나눠서 수를 더해서 (ex. $\square + \square + \square$) 해시주소를 만듦.

③ 공간 제곱 함수: 탐색 키를 제곱해 그 값의 중간 부분에 있는 숫자로 해시주소 생성.

④ 숫자 분석: 탐색 키의 특징은 알 때 사용. (ex. 학번. 앞은 동일하고 아는 특징이 나뉘는 뒤를 이용함)

⑤ 탐색 키가 문자열: 모든 문자열을 가지고 아스키 코드로 변환해 사용.

이때 곱을 더하면 숫자만 다른 같은 문자열인 경우 구분 함수 없으므로

각 아스키 문자에 상수를 거듭제곱한 식 사용. (ex. $\square \cdot C^N + \square \cdot C^{N-1} + \square \cdot C^{N-2} \dots$)

제곱의 횟수에.

* 특징: 가장 기본 특징 = 두 해시값이 다르다면 원래 데이터도 다르다.

(같은 해시값을 갖더라도 원래 데이터가 같음은 보장하지 않는다.)

① * 장의: 양의의 적의 데이터로 고정된 길이의 데이터로 매핑하는 알고리즘

* 응용: 암호학에서 일방향 해시 함수로 무결성 확인 한 예 사용. (메시지 내용에 변경이 없을)

→ 해시 함수로 생성된 해시값 < 이므로 메시지 내용 > 불충분함

→ 메시지 데이터도 알고리즘 (MD) → 이 과정에서 "인출"이 발생. (이것이 암호학의 핵심)

② DB에서 인덱스 만들 때 (해시 인덱스) 사용.

→ 인덱스 생성 시 데이터베이스에 저장

→ 해시 테이블 기반 / 인덱스 구성하는 모든 작업 이용됨. (해시 인덱스 이용함)

ex. B-Tree 인덱스 → 키값 이용함

보안의 4대 요소	Authenticity [정확성]	Confidentiality [기밀성]	Integrity [무결성]	Access Control [제한성]
가장 중요한 것 (가장 보편적으로 사용) 인증 (ID + Password 사용) (자신인증 → MAC 활용, 오류 등 생략해 데이터도 필요함) 보안	정확한 데이터 전송 (ID + Password 사용)	정확한 기밀이 어느 정도 보장될 수 있는가. 암호화/복호화 기술이 이루어졌는지. (TCI/IT 공개 표준으로 보았을 때)	원래 데이터가 데이터 전송 후 변경 되었는지 등의 여부 확인.	데이터 전송을 위한 데이터로 접근 시 인증과 정도에 제한 두는 것. 보안 키 관리.

암호화 알고리즘에서... ← 평문 데이터를 암호화 한 후 암호된 사용자 측은 사용하여 해당 암호문을 복호화해 기밀성 유지.

ex. DES (Data Encryption Standard) 대칭키 암호화
암호화 시키는 Key = 복호화 시키는 Key 같을 때 →

→ 비대칭키 암호화 ex. RSA
= 공개키 암호화 방식

평문을 복호화한 형태로 변환해 기밀성 유지. 암호화 복호화가 분리되어 있음. / 단: 암호 공개가 외부로 알려지면 안됨.

보안에서 무엇을 꼭 지켜야 할지 같이 꼭 조건 먼저 말할 것!

ex: Diffie-Hellman의 암호 교환 방식: 타인에게 알려지지 않은 정보를 두 사람이 교환하는 것만으로 공통의 비밀번호를 만들어내는 방법.

* Radix Tree, Radix Trie 모두 높이 = $\log N + 1$ / complexity: 삽입, 검색, 삭제 모두 = $O(\log N)$

→ 매우 좋은 성능을 나타내는 검색구조 / 비교적 균형이 유지되는 효율적 검색구조 / 비트배열 이용하는 트리구조
→ 기억장소가 충분할 경우 가장 효율적 (기수정렬) (키이진, 기수정렬) = Bucket Sort 같은도 함

11. B-Tree → DB에서 인덱스 구성 방법 중 하나

- Balanced Tree: 이진트리의 문제점(좌우균형 맞지 않으면 비효율적) = degenerated case를 해결한 균형적 트리 / 삽입, 삭제 시 필요하면 스스로 균형을 유지 / 항상 $O(\log N)$ 검색 성능 / 종류 = AVL Tree, B-Tree, Red-Black Tree 등
- B-Tree: 하나의 노드에 여러 자료가 배치되는 트리구조 / 정렬된 데이터를 보존함 / 한 노드에 M개의 자료가 배치되면 M차 B-Tree (한 노드에 들어갈 수 있는 최대 자료개수 = M개) / 한 노드에 M개의 자료, M+1개의 자식을 가질 수 있음 모든 leaf node는 같은 레벨에 위치하여 균형을 유지 (노드 개수 같은 거임)
- M차 B-Tree 높이 = $\log_{M/2} N$ 이하 / 각 노드는 최소 M/2개의 자료를 가짐
- 검색 시 각 노드 당 최대 M번의 순차검색
- complexity: 삽입, 검색, 삭제 = $O(\log N)$ $T(N) < c * M * \log_{M/2} N$
- B-Tree는 외부검색에 효율적인 검색구조 => 하나의 노드 크기를 디스크 입출력 단위(블록)의 크기로 함 / 순차검색과 트리검색의 이점을 취함 => B-Tree는 Balanced Tree로 Worst case가 없다 / 쿼리정렬에서 소구간에 대한 삽입정렬이 성능이 좋음
- * B+ Tree: 모든 레코드들이 트리의 최하위 레벨에 정렬되어 있고 트리 내부 블록에는 키들만 저장됨 / 파일시스템 같은 블록기반 스토리지에서 저장데이터의 효율적 검색에 유용함

12. multi-way search tree (m-way search tree)

- 2-원 탐색 트리라고 부르기도 함 / 이진탐색트리(=2-원 탐색 트리라고도 부름)의 일반화된 형태
- 서브트리 개수: 최대 m개 / 각 노드의 키 개수: 최대 m-1개 / 각 노드마다 서브트리 m개 가질 수 있음
- 장점: 분기율을 높임으로써 트리의 높이가 낮아지게 되고 탐색시간이 줄어듦
- 단점: 노드를 삽입, 삭제할 때 트리의 균형을 유지하기 위해 복잡한 연산이 필요함
- m-원 트리: 높이가 낮고 폭이 넓음 / 한 단계 탐색과정 진행할 때 마다 노드의 탐색 수가 약 $1/m$ 로 줄어듦 BUT binary tree: 높이 높고 폭 좁음 / 한 단계 탐색과정 진행할 때 마다 노드의 탐색 수가 $1/2$ 로 줄어듦
- 목작: 값을 가로로 많이 배치하여 디스크 블록 읽기를 통해 한번에 읽어들이므로 디스크 액세스 수를 최대한 줄임 방문해야 할 노드 수를 줄이는 것은 결국 디스크 액세스 수를 줄이는 것임
- m-원 트리 종류(예가 균형탐색트리 Balanced Search Tree): AVL tree, B-tree, B+ tree 등 $O(\log N)$

13. AVL Tree

- Adelson-Velskii and Landis가 제안 / 각 노드의 왼쪽 서브트리와 오른쪽 서브트리의 높이가 서로 균형을 이루는 트리
- 모든 노드의 서브트리 높이 차이가 1 이하인 트리 / 높이 차이가 2 이상이 되면 노드들 스스로 재배치하여 균형인 상태를 만드는 트리
- 균형인수(balance factor) = 왼쪽 서브트리 높이 - 오른쪽 서브트리 높이 = 1 인 트리
- balance factor가 2이상 인 경우 4가지: LL, RR, LR, RL 방법을 통해 회전시켜 균형을 맞춰줌
- complexity: $O(\log N)$

14. Red-Black Tree

- 이진 탐색 트리 중 하나(좌<root<우)로 가장 효율 좋은 트리 / 노드의 색이 Red or Black인 트리 / root와 leaf node(nil값) 는 모두 Black / Red 노드의 자식은 모두 Black / Black 노드의 자식은 Red or Black / root 노드부터 leaf 노드까지의 Black 노드 개수는 어느 경로에서나 동일
- root node부터 가장 먼 경로까지의 거리가 가장 가까운 경로까지의 거리의 두 배 보다 항상 작다 => 균형이 잡힌 트리 / 모든 worst case에서 complexity가 트리의 높이에 따라 결정되므로 보통의 BST보다 효율적
- rotation은 AVL tree의 기본 operation / 삽입 경우 3가지 있음
 1. 삼촌노드 Red : 부모노드, 삼촌노드 -> Black, 할아버지 노드 -> Red, 할아버지를 삽입노드로 생각하고 루트까지 1번 과정 반복
 2. 삼촌노드 Black, 삽입노드가 부모의 오른쪽자식 : 부모 기준 좌회전, 부모였던 노드가 삽입한 노드로 생각하고 3번 과정 실행
 3. 삼촌노드 Black, 삽입노드가 부모의 왼쪽 자식 : 부모노드 -> Black, 할아버지 노드 -> Red, 할아버지 노드 우회전
- complexity: $O(\log N)$

MST 만드는 방법 ← 무작정 알고리즘 → 모든 신장트리를 고려, 그 중 최소비용이 되는 것을 선택 $\Rightarrow$ 매우 비효율적
탐욕 알고리즘 (Greedy Algorithm) $O(n^2)$

- ↓
- 최종해답을 얻지 못하는 동안 다음의 3단계 반복작업.
 - 선정 절차: 적절한 후보해 선정 절차에 따라 하나의 이음선 선정.
 - 적정성 점검: 선정된 이음선을 E에 추가해도 사이클이 생기지 않으면 E에 추가.
 - 해당 점검: 안 들어본 트리가 신장트리이면 문제 해결.
- ↓

$\therefore$ 다른 방법.

탐욕 알고리즘 사용한 대표작인 알고리즘 = Prim's Algorithm.

15. Graph

- 정점(vertex=node)과 간선(edge)로 이루어진 자료구조 / 객체간의 연결을 모델링하는 자료구조
- 경로(path) : 한 노드에서 다른 노드까지의 길 / 순환(cycle) : 처음 정점과 끝 정점이 같은 경로
- 차수(degree) : 하나의 정점에 달아있는 간선의 수 => 진입차수(in-degree)/진출차수(out-degree) (방향그래프만)
- 방향(directed)그래프: 출발노드와 도착노드가 정해짐 / 무방향(undirected)그래프: 출발노드와 도착노드의 구분없음
- n개 꼭짓점 가진 무방향 그래프의 최대 변의 수 = $n(n-1)/2$ = 완전그래프(complete graph)
- 인접하다(adjacent) = 두 꼭짓점(정점)이 변으로 이어져 있는 상태
- Spanning Tree : 신장트리 = 그래프의 부분 그래프 중 모든 정점과 연결된 부분 그래프(cycle이 없는 그래프)
- Minimum Spanning Tree(최소신장트리) : 신장트리에서 가중치(간선 비용의 합) 합이 최소인 트리
/ (ex. 지하철역 최단거리, 최소비용으로 통신망연결 등) / 이걸 위해 Dijkstra 알고리즘 존재(시작점 제외한 모든 정점 시작점 가리킨다 가정 / 이어지지 않은 정점은 무한대 표시(무한대 비용)하고 가중치 작은 정점을 찾아 잇는 것)
- 그래프 순회/탐색 방법 : DFS(깊이 우선 탐색) / BFS(너비 우선 탐색)
/ (현재 위치를 스택에 넣어 기억하고, 방문하지 않은 인접리스트에 대해 계속 처리함)
- Depth First Search : 시작점을 기준으로 가장 아래까지 이동하면서 탐색하는 방식 (↑↓ 인접점탐색)
- Breadth First Search : 인접한 꼭짓점을 순서대로 방문하는 방식
/ 인접한 꼭짓점 순서대로 방문하기 때문에 큐를 만들어서 넣는 순서대로 방문함 (→ 순차진행)
- 그래프 표현방법 : 인접리스트(Adjacency List) / 인접행렬(Adjacency Matrix)
- 인접리스트 : 한 정점에 간선으로 연결된 다른 정점들을 연결리스트로 연결해 놓는 방법
/ 필요한 만큼만 메모리를 사용하므로 희소그래프(sparse: 정점에 비해 적은 간선수)의 표현에 유리함
/ 적은 메모리 공간을 요구하나 행렬 구조에 비해서 느림
- 인접행렬 : 정점의 수가 V인 그래프를 $V \times V$ 행렬을 이용하여 표현
/ $V \times V$ 만큼의 메모리를 사용하므로 밀집그래프(그래프가 조밀하게 구성)의 표현에 적합함
/ 많은 메모리 공간을 요구하나 리스트 구조에 비해서 빠른 접근 제공(random access 가능한 것)

	Adjacency list	Adjacency Matrix
저장공간	$O(V+E)$	$O(V^2)$
Vertex 추가	$O(1)$	$O(V^2)$
Edge 추가	$O(1)$	$O(1)$
Vertex 삭제	$O(E)$	$O(V^2)$
Edge 삭제	$O(E)$	$O(1)$

노드의 개수를 V라고 하고 링크의 개수를 E라고 해보자. 이 때, 인접 행렬은 $O(V^2+E)$ 의 시간을 가지고, 인접 리스트는 $O((V+E) \cdot \log V)$ 의 시간을 가진다. 밀집 행렬일 경우에는 V에 비해 E가 커지기 때문에 인접 행렬의 $O(V^2+E)$ 가 더 유리하고, 반면 희소 행렬일 경우에는 E에 비해 V가 크기 때문에 인접 리스트의 $O((V+E) \cdot \log V)$ 가 더 유리하다. ✓ 설명이랑 위의 표랑 참고하기

MST 표현 알고리즘

① Kruskal

- 최소비용을 가진 edge들을 차례로 탐색해 나가면서 cycle이 없는 경로를 찾는 방법.
- $O(E \cdot \log V)$ (= $O(E \log E)$)
(= 트리 정렬하는데 걸리는 시간 $O(E \log E)$ + cycle생성 여부 검사 시간 $O(E)$ = $O(E \log E)$ = $O(E \log V)$) dense tree: $O(N^2 \log N)$

② Prim

- 임의의 정점을 기준으로 비용이 가장 적은 edge를 탐색해 나가는 방법, 역시 cycle이 없는 경로를 찾음.
- 밀집그래프(dense graph)에서 Optimal하다. ($|E| \approx |V|^2$ 인 경우 Kruskal 보다 빠름) dense tree: $O(N^2)$

이진 힙(Binary Heap)이름한 경우 = $O(E \cdot \log V)$

① 인접리스트 = $O(E \log V)$

② 인접행렬, (전산) = $O(V^2)$

③ 피보나치 힙 ① 인접리스트 = $O(E + V \log V)$

④ Sollin 알고리즘: 최소한의 비용을 갖는 subset을 조합하는 방법 (= forset)
→ $O(N^2 \log N)$ → 병렬 알고리즘에서 가장 높은 효율을 가짐.
PPL 병렬 컴퓨터(여러개의 프로세서들 사용하여 여러개의 명령어를 동시에 실행)에서 실행되도록 고안된 알고리즘.
Parallel Pattern Library

운영체제

1. Thread

- Thread : 프로세스 내에서의 작업 단위 / 시스템의 여러 자원을 할당받아 실행하는 프로그램의 단위
- 장점: 시스템 자원을 공유해서 기억장소 낭비 줄임 / 프로그램 응답 시간 단축됨 / 자원 공유가 용이함

② 하나의 프로세스를 여러개의 스레드로 생성 → 병행성 증진 (parallel) (concurrency)
③ multi-process system일 때, 각 스레드는 서로다른 프로세서를 병렬적으로 실행, 동시성 향상

④ 단점: 병행성 문제 → 두 개 이상의 스레드가 어떤 객체에 있는 하나의 데이터에 접근할 때,

/ 해결법 ⇒ lock = 상황에 따라 한 스레드가 사용중인 자원을 특정작업이 끝날 때까지 다른 스레드에서 쓸 수 없게 만들
synchronized = 어떤 메소드를 한 번에 한 스레드만 접근가능 하도록 하는 것

- Process : 일반적으로 실행중인 프로그램이라고 함

PCB((Process Control Block): 프로세스에 대한 중요 정보 저장)를 가짐(프로세스 생성시마다 PCB생성)

Context-switch(문맥교환) : 프로세스가 다음에 실행할 프로세스로 전환되는 과정. 이전 프로세스에 대한 정보저장

스레드와 프로세스 비교

- 공통점 : 모두 여러 흐름이 동시에 진행된다는 것

- 차이점 : 프로세스 = 독립적으로 실행되며 각각 별개의 메모리를 차지하고 있음

스레드 = 프로세스 내 메모리를 공유해 사용 = 경제적

/ 문맥교환의 속도가 빠르다(프로세스에 비해 보존해 두어야 할 정보가 상대적으로 적어 시간이 짧음)

2. 가상 메모리 운영방법

1. 페이징 기법

- 프로그램을 동일한 크기로 나눈 단위를 페이지라 하며 이 페이지를 블록으로 사용하는 기법

⇒ 동일한 크기로 나뉜 프로그램을 주기억장치의 영역(페이지 프레임)에 적재시켜 실행 / 내부 단편화 발생

2. 세그멘테이션 기법

- 프로그램을 가변적인 크기로 나눈 단위를 세그먼트라 하며 이 세그먼트를 블록으로 사용하는 기법

⇒ 기억공간 절약하기 위해 사용 / 외부 단편화 발생

- 내부 단편화 : 분할된 영역 크기 > 할당될 프로그램 크기 → 사용되지 않고 남은 빈 공간

- 외부 단편화 : 분할된 영역 크기 < 할당될 프로그램 크기 → 사용되지 못하고 남은 빈 공간

3. Paging

- 페이지 크기: 페이징 기법 사용시 프로그램을 페이지 단위로 나눌 때 시스템에 미치는 영향

小: 페이지 단편화 ↓ (감소) / 매핑속도 ↓ (테이블 크기 ↑) / 입출력 효율 ↓ (자주 접근, 시간 ↑) / 적재이동시간 ↓
/ 효율적인 working set (locality 일치-가까운 시간 내(temporal), 근처 페이지(spatial) 계속 참조), 장치의 효율 ↑

- 大: 페이지 단편화 ↑ / 매핑속도 ↑ (테이블 크기 ↓) / 입출력 효율 ↑ (디스크 접근수 ↓, 시간 ↓) / 적재이동시간 ↑

/ 불필요한 내용까지 적재될 수 있음

- Locality(국부성) : 프로세스가 실행되는 동안 주기억장치를 참조할 때 일부 페이지만 집중적으로 참조하는 성질

- Working Set : 프로세스가 일정 시간동안 자주 참조하는 페이지들의 집합 / 주기억장치에 상주시켜서 효율 ↑

- Page Fault(페이지 부재) : 프로세스 실행시 참조할 페이지가 주기억장치에 없는 현상

/ 페이지 부재율에 따라 주기억장치에 있는 페이지 프레임 수를 조절해 적정 수준으로 유지

- 페이지 교체 알고리즘 : 페이지 부재 발생 시 어떤 페이지 프레임을 선택하여 교체할 것인가 결정

- OPT(OPTimal replacement: 최적 교체) : 앞으로 가장 오랫동안 사용하지 않을 페이지 교체 / 미리예측=가능성희박

- FIFO(First In First Out) : 적재된 시간 기억해 먼저 들어온 순서대로 페이지 교체

/ 페이지 프레임수 ↑ 시켰지만 페이지 부재 ↑ 더 많이 발생 = 모순 발생

- LRU(Least Recently Used) : 최근에 가장 오랫동안 사용하지 않은 페이지를 교체 / 오버헤드 발생 / 구현 어려움

- LFU(Least Frequently Used) : 사용 빈도가 가장 적은 페이지를 교체 / 사용되지 않은 페이지가 프레임 계속 차지

- Thrashing(스래싱) : 프로세스의 처리 시간 < 페이지 교체에 소요되는 시간 ⇒ 전체 시스템 성능 저하됨

⇒ 방지법: working set 유지, 페이지 부재 빈도 조절, 다중프로그래밍 정도 적정수준 유지 등

네트워크

1. OSI(Open System Interconnection) 7계층

: 통신이 일어나는 과정을 7단계로 구분한 것 (개발의 참조모델, 프로토콜의 표준적 뼈대 제공)

- 1 Physical layer(물리 계층)
 - = 두 컴퓨터 간 물리적 연결을(정의)하는 계층 / 데이터를 물리매체 상으로 전송 / 장비들 간 Bit 전송
- 2 Data link layer(데이터링크 계층)
 - = 물리적 계층을 통한 데이터 전송에 신뢰성 제공 / 물리적 주소 지정(MAC address) / Frame 안전하게 전송
- 3 Network layer(네트워크 계층)
 - = 경로 선택 및 라우팅, 논리적 주소(IP) 정의하는 계층 / 패킷 이동경로 결정 / Packet으로 분할하여 전송
- 4 Transport layer(전송 계층)
 - = 상/하위 계층간 인터페이스 기능 및 메시지 전달 / 흐름제어 및 오류제어 / TCP, UDP 프로토콜 / Segment
- 5 Session layer(세션 계층)
 - = 통신하는 두 호스트 간 최초연결 및 연결유지 담당 / 세션을 열고 닫고 관리 / dialogue관리(= 대화 = 통신)
- 6 Presentation layer(표현 계층)
 - = 전송 데이터 포맷 결정 / 데이터 포맷 상호 변환, 압축(compression) 및 암호화(encryption)수행 / ex. jpeg, avi
- 7 Application layer(응용 계층)
 - = 사용자가 이용하는 네트워크 응용프로그램 / 응용프로세스 상호간 정보교환 창구/telnet(23), http(80), ftp(21) protocol

2. TCP/IP 모델

: OSI7계층=장비 개발자가 어떻게 표준을 잡을지 결정 / TCP/IP=실질적으로 사용되는 프로토콜에 대한 것(신뢰도↑)

- 1, 2 = Network Access layer(네트워크 접속 계층) / - 3 = Internet layer(인터넷 계층)
- 4 = Transport layer(전송계층) ICMP, 흐름 / 오류제어 / - 5, 6, 7 = Application layer(응용 계층)

3. 가상주소 물리주소 차이

- MAC address: 각 장비들이 정확한 통신을 위해 네트워크상에서 서로 구분하는 역할을 해주는 것 / 각각의 네트워크 장비의 인터페이스는 고유의 MAC 주소를 가짐

← TCP/IP 프로토콜 사용하는 네트워크에서 IP 주소 사용하여 통신. but, 최종적으로 MAC 주소 사용하여 데이터 전달

- ARP(Address Resolution Protocol) : IP → MAC ↔ RARP(reverse) : MAC → IP (인터넷계층3)
- 논리적인 가상주소를 물리적인 실기억주소로 변환하는 것 = 주소 사상 = 매핑(Mapping)

4. TCP 역할 / IP 역할

- TCP(Transmission Control Protocol) 신뢰성 ↑
 - : 연결 지향성 프로토콜 / 신뢰적 / 장: 보장된 세그먼트 전달 / 단: 연결을 위한 초기 설정 시간 소요
- UDP(User Datagram Protocol) 신뢰성 ↓
 - : 비연결형 프로토콜 / 비신뢰적(데이터 도착여부 모름) / 빠름 / 신뢰성보다는 고속성 요구되는 응용에 사용
- IP(Internet Protocol) 3
 - : 비연결형 프로토콜 / 데이터 전송 / 발신지(송신지)부터 목적지(수신지)까지의 라우팅 경로를 결정하는데 사용

5. 데이터 체인 방식

- 인터럽트 발생 시 우선순위 판별방법: S/W=폴링(Polling)기법 / H/W=데이터 체인(daisy-chain)기법

S/W 폴링: 인터럽트 요청한대로 차례대로 검사하는 방식 / 응답속도 느리다

H/W 데이터 체인: 인터럽트 우선순위에 따라 직렬로 연결하는 방식 / 응답속도 빠르다

= 연결방식이 데이터체인!

S/W	H/W
지연	고속
간단	복잡
정기적	비정기적

polling: 프로그램이 이연신에만이 지속적으로

daisy-chain: 연속적으로 판별되어있는 하드웨어가

*바인딩 (Binding): 함수 호출과 실제 함수를 연결하는 방법 // 프로그램 실행 시 각종 변수나 함수를 특정으로 해결짓는 과정.
기본적으로는 정적 바인딩 실행.

종류

정적 바인딩 (Static Binding) [컴파일 타임에 호출될 함수를 결정. (컴파일 시)
객체나 타입을 보고 호출할 함수 결정.

동적 바인딩 (Dynamic Binding) [런타임(실행 중)에 호출될 함수를 결정. (프로그램 실행 시)
- virtual 메서드를 사용해 가상 함수를 정의해 주함.
- 호출 함수의 결정은 객체의 가상 함수 table 사용.

*가상 함수 (Virtual function): virtual 메서드를 붙여 정의된 함수는 런타임 시 포인터가 가리키는 타입에 따라 호출할 함수 결정.
포인터가 어떤 객체를 가리키는가에 따라 호출될 함수 결정.
이를 통해 다형성 구현.

이산수학

* X에서 Y로의 함수인 $f: X \rightarrow Y$

- 정의역(Domain): 함수의 값이 정의된 집합 / x값 전체의 집합
- 공역(Codomain): 함수의 값들이 속하는 집합 / y값 전체의 집합
- 치역(Range): 정의역에 대응하는 공역의 원소를 모아둔 집합 / 함수의 모든 출력 값의 집합 / 치역 $\subset$ 공역

1. 역함수

- 함수 $f: X \rightarrow Y$ 가 Y에 대해 일대일대응일 때, 함수 Y의 각 원소 y에 대해 $f(x) = y$ 인 원소 x는 단 하나 존재합니다. 이 원소 x에 대응하고 Y를 정의역, X를 공역으로 갖는 함수를 f의 역함수라 하고 $x = f^{-1}(y)$ 로 나타낼 수 있습니다.
- *조건=> 일대일대응 / 증가함수($x \uparrow \rightarrow f(x) \uparrow$) or 감소함수($x \uparrow \rightarrow f(x) \downarrow$)

일대일 대응 $\rightarrow f(x)=y$ 인 x 는 유일 $\rightarrow x$ 에 대응하고, $\forall y \in Y$, y 공역의 원소로 역함수 $f^{-1}(y)$ 가 존재한다.

2. 멱집합(Power Set)

- 어떤 집합 A에 대하여 A의 모든 부분집합들로 이루어진 집합을 A의 멱집합이라고 합니다. A의 원소의 개수가 n개라면 멱집합의 개수는 2^n 개입니다. $P(A) = 2^A$ 로 나타낼 수 있습니다.
- *증명=> A의 어떤 부분집합을 만들 때 각 원소에 대해 그 원소가 들어가는 경우와 그렇지 않은 경우 2가지로 나눈다. 이렇게 원소마다 부분집합에 대한 2가지 경우의 수가 생기고 각 사건이 독립적이므로 각 원소 수만큼 곱한다.
- cf. 멱함수: ' $y=ax^n$ '과 같은 식으로 나타낼 수 있는 함수.

3. 수학적 귀납법

- 자연수 n에 대한 명제 함수 $p(n)$ 에 대해 $p(1)$ 참이고 임의의 자연수 k에 대해 $p(k)$ 가 참이라 가정합니다. 이 때 $k+1$ 에 대하여 $p(k+1)$ 이 참 일 때 모든 자연수 n은 함수 $p(n)$ 에 대해 참임을 증명하는 방법입니다.
- *조건=> 자연수
- cf. 귀류법(proof by contradiction) = 모든 경우가 성립하지 않을 때 자주 사용하는 증명법
(p가 거짓이면 q가 거짓이다. 가정이 잘못되었으므로 $\therefore p$ 가 참이어야함)

4. 릴레이션

- 두 집합의 원소들 사이에 어떤 관계를 이루는 부분집합을 릴레이션이라고 합니다.
(유사한 대상들을 관련시키는 성질들의 특정 조합을 릴레이션이라고 합니다.)
- *표현법=> 카르테시안 곱($A \times B$)의 부분집합 형태 / 화살표 선도 / 표 / 방향성 그래프 등으로 표현 가능
(directed graph=>인접행렬/리스트)
- 이런 표현 형태들을 '집합 A에서 집합 B로의 관계'라고 말합니다.

5. 동치관계(equivalence relation)

- 어떤 집합 A에 대한 이진관계가 반사적, 대칭적, 전이적이면 집합 A에 대한 동치 관계라고 합니다.
- 이진관계(집합 A의 순서쌍 집합)
- 반사적(모든 x가 자기 자신과 관계됨) / 대칭적(x와 y와 관계되면 y는 x와 관계됨)
- 전이적(x가 y와 관계되고 y와 z와 관계되면 x는 z와 관계됨) / 비대칭적(x와 y가 같을 때만 대칭 관계가 성립)

6. 집합의 상등(Equality of Sets)

- 서로 같은 두 집합을 말합니다. 두 집합 A, B가 있을 때 A의 원소가 모두 B에 속하고, B의 원소도 모두 A에 속하면 집합 A와 집합 B는 서로 같다고 하고 이를 집합의 상등이라 말합니다.

수차 접근 ① ② 만지 해킹권만국하는 적외선 감도출 제리만수행

*파일시스템: 파일을 생성하여 자료들은 추가, 수정, 삭제, 검색 등의 조작할수있는 SIW시스템

• 데이터(파일)의존 ↑ 데이터변경시 영향도 2배에 증가됨

독립된 파일 단위로 (업무와 관련된) 데이터를 저장하므로 데이터 중복성과 데이터 종속성 발생

독립성

②

*데이터베이스시스템: 파일시스템의 문제점을 해결하며 데이터의 독립성, 데이터의 일관성 및 무결성 유지

차이점

특징: 실시간 접근 / 계속적인 변화 / 공유성 / 내용에 의한 참조
(빈게이트로 연결해결) (신뢰, 속내 명단) (여러사람들) (주요시작조각이나난 곳에 따라참조)

동행관리

DBMS: 응용프로그램과 데이터베이스 사이의 중재자.

장점: 독립성 보장, 무결성 보장, 일관성 유지, 데이터통합관리, 데이터실시간제

단점: 비용 ↑, 자료처리 복잡, 시스템 복잡 (병행, 한계), 전문인력 부족

Random Access

정의, 조작, 제어 기능을 함

(사용자 R 제어해 임의데이터 읽기) → ④ 안정성과 편리성 제공에 도움
∴ 데이터의 추가, 수정, 삭제 등이 용이함

*스키마 = 데이터베이스의 구조와 제약조건에 대한 명세를 기술한 것.

(=메타데이터)

데이터사전 (=시스템카탈로그)에 저장됨

외부스키마: 사용자 관련, 여러개 존재
- 개념스키마: 인본적스키마, 전체 뷰, DB전체논리구조, 무결성 규칙
내부스키마: 실제데이터 저장, DB 물리적구조

① 캡슐화(Encapsulation): 데이터와 함수 등 객체와 관련된 것들을 하나로 묶는 것을 말한다.
흔히 정보 은닉과 함께 연관지어 사용되는 개념으로 외부에서 알 필요가 없는 데이터나 행위는 보이지

1~4 객체지향 언어의 특징? 양도록해 실행흐름이 외부에 드러나지 않도록 한다. / 재사용성 향상
공통작업은 한곳에 치어. → 작업은은 시스템유니티 사용 (복제된작업은 한곳에서 수정이 가능함, 관리도 편리해 제작기에 4등 가능)

② 추상화(Abstraction): 인터페이스와 구현을 분리하는 것 / 구현을 통해 객체가 가진 특성 중 필수속성만으로 객체를 묘사하고 유사성만을 표현하여 세부적 사항은 각 객체마다 다르게 구현하도록 할수있다.
(C++에서는 가상함수를 통해 (상속을 이용) 추상화 가능)

③ 상속(Inheritance): 클래스기능을 확장하는 (예)생물종 하나 / 기존 클래스를 수정하지 않으면서도 이미 정의되어 있는 내용을 확장해서 사용할수있는 방법을 제공한다. / 코드의 간결성 ↑ (구현시 비슷한 속성의 공통 구현 방식) / 부모클래스 → 자식클래스 관계

④ 다형성(Polymorphism): 서로 다른 타입에 대해 동일한 방식으로 접근할 수 있도록 하나의 인터페이스를 제공하는 것이다.
하나의 형태로 여러개의 내용을 가짐 (즉, 동일한 타입의 객체의 동일한 메소드가 다른일을 하는 것)
재사용성과 유연성 확장 (인터페이스를 통한 유연한 기능 확장 및 클래스 추가 가능) / 동일한 메세지에 대해 서로 다르게 행동할
C언어에서.. ex. int + double 처럼 서로 다른 타입이나, + 연산 자체는 모든 기본타입에 대해 동일하게 자원하고있음
C++에서 더 다양한 다형성 제공. ex. 가상함수, 템플릿, 오버로딩 등... 다. 메세지 특정 선택을 통해, 리소스가 → 부동소수점 등 5등 정도

*오버로딩(overloading): 중복정의의라고도 할수 있다. 클래스 안에 같은 이름을 가진 함수를 여러개 정의할 수 있다.
단, 매개변수의 타입, 개수가 달라야 함.

*오버라이딩(overriding): 상속관계에 있는 클래스들 간 같은 이름의 함수를 정의 하는 것. 함수의 재정의라고도 한다.
자식클래스에서 부모클래스의 함수 재정의함. / + 동적바인딩 방식

데이터베이스

- 외부스키마: 사용자가 보는 관점 / 개념스키마: DB전체 논리구조, 제약조건, 무결성 / 내부스키마: DB물리적구조
- DDL: DB정의 및 변경 / DML: 사용자 요구에 따라 검색, 갱신, 삽입 등 지원 / DCL: 정확성, 안정성 유지
- 관계대수: 절차적언어, How, select, project, join, division / 실행할 연산을 명시
- / 관계해석: 비절차적언어, What, 관계대수로 표현가능 / 원하는 결과의 집합론적인 서술을 제시

- DB설계: 요구조건분석 -> 개념적설계(ERD) -> 논리적설계(RDB관계형데이터베이스/정규화) -> 물리적설계 -> DB구축
- E-R(Entity-Relationship) 모델: 개체와 개체 간의 관계를 도식화 한 것 (1:1 / 1:N / N:M)

- 관계형 데이터베이스: 도메인(속성이 가질 수 있는 값들의 집합) / 차수(속성 수) / 카디널리티(튜플 수)

- 슈퍼키: 속성들의 집합으로 구성된 키 / 유일성 O / 최소성 X
- 후보키: 개체들을 고유하게 식별할 수 있는 속성 / 유일성 O / 최소성 O
- 기본키: 주키 / 후보키 중 선택한 키 / 중복 X / NULL X
- 외래키: 다른 릴레이션의 기본키 참조하는 키

- 제약조건(무결성, Integrity): 개체(entity) 무결성: 기본키를 구성하는 키 값은 NULL X, 중복 X

- 참조 무결성: 참조할 수 없는 외래키를 가질 수 없음
- 도메인 무결성: 각 속성 값은 반드시 정의된 도메인에 속한 값이어야 함

- 정규화(Normalization): 종속성, 중복성의 배제, 릴레이션의 분해(이상현상 줄임)가 목적 / 단점: 연산 시간의 증가
- 1NF(도메인이 원자값) / 2NF(부분적합수종속성제거) / 3NF(이행적합수종속성제거) / BCNF(결정자O, 후보키X제외) / 4NF(다치종속성제거) / 5NF(조인종속성이용)

1NF->5NF 정규화 과정

- JOIN 연산: 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산
- => DB에 부담을 주므로 가급적 사용하지 않는 게 좋음(외래키로 데이터 검색하므로 성능 저하)

- 병행 제어: 목적: DB 공유 최대화 / 일관성 유지 / 시스템 활용도 최대화 / 사용자 응답시간 최소화

- ↳ 기법= Locking(로킹): 하나의 트랜잭션이 데이터 액세스하는 동안 다른 트랜잭션은 액세스 불가

로킹 단위 대: 로크 수 소 -> 관리 수월 / 병행성 수준 ↑ / 공유도 ↓
소: 로크 수 대 -> 관리 복잡 / 병행성 수준 ↓ / 오버헤드 ↑

기타

* 함수에서 파라미터 패싱 방법

- call by value (값에 의한 호출)

: 어떤 함수를 호출할 때 전달인자로 그 데이터 값 자체를 전달하는 호출 방법이다.

=> 인자를 넘겨줄 때 마다 메모리 공간을 할당하기 때문에 공간을 소모하나, 원본 데이터에 손상을 주지 않는다.

- call by reference (참조에 의한 호출)

: 어떤 함수를 호출할 때 전달인자로 어떤 특정한 데이터 값이 아닌 그 값에 대한 정보를 전달하는 호출방법이다.

=> 인자를 넘겨줄 때 주소 값을 참조하므로 메모리 공간을 따로 할당하지 않지만, 원본 데이터에 손실을 준다.

* Big O 표기법

- 시간 복잡도 함수에서 불필요한 정보를 제거하여 알고리즘 분석을 쉽게 할 목적으로 시간 복잡도를 표시하는 방법을 빅오표기법(big-oh notation)이라 한다.

(f와 g를 각각 양의 정수를 갖는 함수라 할 때, 만일 어떤 두 양의 상수 a와 b가 존재하고,

모든 $n \geq b$ 에 대하여, $f(n) \leq a \cdot g(n)$ 이면 $f(n) = O(g(n))$ 이다.)

* UNIX에서 프로세스 생성 함수

- fork() => 부모프로세스를 복제하는 함수 => 부모 -> 자식 (부모) -> 자식

=> 결국 트리형태가 된다. / 각각의 프로세스마다 고유의 pid(process id)를 부여한다.

fork() == 0

- ① 순차상환수로 선언한 클래스는 인스턴스 생성 불가. (= 추상클래스는 인스턴스 생성 불가.)
- (추상클래스를 상속한 자식클래스는 순차상환수를 구현해야 인스턴스 생성 가능.)

- ② 런타임 시에 공유 객체는 공유 객체로, 정적 멤버 함수는 정적 멤버 함수로, 정적 멤버 변수는 정적 멤버 변수로 선언한다.

③ STL (Standard Template Language)

템플릿으로 함수 작성 후 호출 시에 인스턴스화되어 호출부의 인자타입을 받아 그 타입에 맞는 함수를 (공과인식) 컴파일러가 자동으로 작성한다.